

AWS Foundational services

Amazon EC2 (Elastic Compute Cloud):

Description: Provides scalable compute capacity in the cloud.

Use Cases: Running virtual servers for various applications.

Amazon S3 (Simple Storage Service):

Description: Object storage service for scalable and secure storage of data.

Use Cases: Storing and retrieving any amount of data, hosting static websites.

Amazon RDS (Relational Database Service):

Description: Managed relational database service that supports multiple database engines.

Use Cases: Running databases without managing the infrastructure.

Amazon VPC (Virtual Private Cloud):

Description: A logically isolated section of the AWS Cloud where you can launch resources.

Use Cases: Building a secure and scalable network infrastructure.

Amazon IAM (Identity and Access Management):

Description: Enables you to manage access to AWS services and resources securely.

Use Cases: Setting permissions for users and resources.

Amazon CloudWatch:

Description: Monitoring service for AWS resources and applications.

Use Cases: Collecting and tracking metrics, setting alarms, and reacting to changes.

AWS Lambda:

Description: Serverless compute service that runs code in response to events.

Use Cases: Running code without provisioning or managing servers.

Amazon SNS (Simple Notification Service):

Description: Fully managed messaging service for publishing messages.

Use Cases: Sending messages or notifications to a distributed set of recipients.

Amazon SQS (Simple Queue Service):

Description: Fully managed message queuing service.

Use Cases: Decoupling components of a cloud application, distributed computing.

Amazon DynamoDB:

Description: Fully managed NoSQL database service.

Use Cases: Storing and retrieving any amount of data with low-latency access.

Amazon CloudFront:

Description: Content delivery network service for securely delivering data.

Use Cases: Accelerating delivery of websites, APIs, and other content.

Amazon Route 53:

Description: Scalable domain name system (DNS) web service.

Use Cases: Registering and managing domain names, routing traffic.

AWS Elastic Beanstalk:

Description: Fully managed service for deploying and running applications.

Use Cases: Quickly deploying and managing applications in multiple languages.

AWS Auto Scaling:

Description: Automatically adjusts the number of EC2 instances in a scaling group.

Use Cases: Ensuring performance and availability of applications.

Amazon EC2 Auto Scaling:

Description: Automatically adjusts the number of EC2 instances in a scaling group.

Use Cases: Ensuring performance and availability of applications.

AWS Well-Architected Framework

- The AWS Well-Architected Framework helps you understand the pros and cons of decisions you make while building systems on AWS.
- By using the Framework you will learn architectural best practices for designing and operating reliable, secure, efficient, cost-effective, and sustainable systems in the cloud.
- It provides a way for you to consistently measure your architectures against best practices and identify areas for improvement.
- The process for reviewing an architecture is a constructive conversation about architectural decisions, and is not an audit mechanism.
- [The Six Pillars](#) of AWS Well-Architected are:
 1. **Operational Excellence Pillar** - focuses on running and monitoring systems, and continually improving processes and procedures.
 2. **Security Pillar** - focuses on protecting information and systems.
 3. **Reliability Pillar** - focuses on workloads performing their intended functions and how to recover quickly from failure to meet demands.
 4. **Performance Efficiency Pillar** - focuses on structured and streamlined allocation of IT and computing resources.
 5. **Cost Optimization Pillar** - focuses on avoiding unnecessary costs.
 6. **Sustainability Pillar** - focuses on minimizing the environmental impacts of running cloud workloads.
- The AWS Well-Architected Tool (AWS WA Tool) is a 'free' service in the cloud that provides a consistent process for you to review and measure your architecture using the AWS Well-Architected Framework.
- The AWS WA Tool provides recommendations for making your workloads more reliable, secure, efficient, and cost-effective.

General design principles:

The Well-Architected Framework identifies a set of general design principles to facilitate good design in the cloud:

- **Stop guessing your capacity needs:** If you make a poor capacity decision when deploying a workload, you might end up sitting on expensive idle resources or dealing with the performance implications of limited capacity. With cloud computing, these problems can go away. You can use as much or as little capacity as you need, and scale up and down automatically.
- **Test systems at production scale:** In the cloud, you can create a production-scale test environment on demand, complete your testing, and then decommission the resources. Because you only pay for the test environment when it's running, you can simulate your live environment for a fraction of the cost of testing on premises.
- **Automate with architectural experimentation in mind:** Automation permits you to create and replicate your workloads at low cost and avoid the expense of manual effort. You can track changes to your automation, audit the impact, and revert to previous parameters when necessary.
- **Consider evolutionary architectures:** In a traditional environment, architectural

decisions are often implemented as static, onetime events, with a few major versions of a system during its lifetime. As a business and its context continue to evolve, these initial decisions might hinder the system's ability to deliver changing business requirements. In the cloud, the capability to automate and test on demand lowers the risk of impact from design changes. This permits systems to evolve over time so that businesses can take advantage of innovations as a standard practice.

- **Drive architectures using data:** In the cloud, you can collect data on how your architectural choices affect the behavior of your workload. This lets you make fact-based decisions on how to improve your workload. Your cloud infrastructure is code, so you can use that data to inform your architecture choices and improvements over time.
- **Improve through game days:** Test how your architecture and processes perform by regularly scheduling game days to simulate events in production. This will help you understand where improvements can be made and can help develop organizational experience in dealing with events.

AWS S3 bucket

AWS Storage Services:

Object, file, and block storage

- Amazon Simple Storage Service (S3)
 - Object storage with industry-leading scalability, availability and security for you to store & retrieve any amount of data from anywhere.
 - URL-based storage.
- Amazon Elastic File System (EFS)
 - A simple, serverless, elastic, set-and-forget file system for you to share file data without managing storage.
- Amazon FSx
 - Fully managed, cost-effective file storage offering the capabilities and performance of popular commercial and open-source file systems.
- Amazon Elastic Block Store (EBS)
 - Easy to use, high performance block storage service for both throughput and transaction-intensive workloads at any scale.
- Amazon File Cache
 - High-speed cache for datasets stored anywhere, accelerate cloud bursting workloads.

Amazon S3 Storage Classes:

There are several storage classes in Amazon. Those are:

- S3 Standard
 - Frequently accessed data (more than once a month) with milliseconds access.
- S3 Intelligent-Tiering
 - Data with changing or unknown access patterns.
- S3 Standard-IA
 - Infrequently accessed data (once a month) with milliseconds access.
- S3 One Zone-IA
 - Re-creatable, infrequently accessed data (once a month) stored in a single Availability Zone with milliseconds access
- S3 Glacier Instant Retrieval
 - Long-lived archive data accessed once a quarter with instant retrieval in milliseconds.
- S3 Glacier Flexible Retrieval (formerly known as "Glacier")
 - Long-lived archive data accessed once a year with retrieval of minutes to hours.
- S3 Glacier Deep Archive
 - Long-lived archive data accessed less than once a year with retrieval of hours.

Amazon S3 Standard (S3 Standard)

- Low latency and high throughput performance
- Designed for durability of 99.999999999% of objects across multiple Availability Zones
- Resilient against events that impact an entire Availability Zone
- Designed for 99.99% availability over a given year
- Backed with the Amazon S3 SLA for availability
- Supports SSL for data in transit and encryption of data at rest
- S3 Lifecycle management for automatic migration of objects to other S3 Storage Classes

Amazon S3 Intelligent-Tiering (S3 Intelligent-Tiering)

S3 Intelligent-Tiering is the first cloud storage that automatically reduces your storage costs on a granular object level by automatically moving data to the most cost-effective access tier based on

access frequency, without performance impact, retrieval fees, or operational overhead.

- Frequent, Infrequent, and Archive Instant Access tiers have the same low-latency and high-throughput performance of S3 Standard
- The Infrequent Access tier saves up to 40% on storage costs
- The Archive Instant Access tier saves up to 68% on storage costs
- Deep Archive Access tier has the same performance as Glacier Deep Archive and saves up to 95% for rarely accessed objects
- Designed for durability of 99.999999999% of objects across multiple Availability Zones and for 99.9% availability over a given year
- Backed with the Amazon S3 Service Level Agreement for availability
- Small monthly monitoring and auto tiering charge
- No operational overhead, no lifecycle charges, no retrieval charges, and no minimum storage duration
- Objects smaller than 128KB can be stored in S3 Intelligent-Tiering but will always be charged at the Frequent Access tier rates, and are not charged the monitoring and automation charge.

S3 Standard-IA

S3 Standard-IA is for data that is accessed less frequently, but requires rapid access when needed.

- Same low latency and high throughput performance of S3 Standard
- Designed for durability of 99.999999999% of objects across multiple Availability Zones
- Resilient against events that impact an entire Availability Zone
- Data is resilient in the event of one entire Availability Zone destruction
- Designed for 99.9% availability over a given year
- Backed with the Amazon S3 Service Level Agreement for availability
- Supports SSL for data in transit and encryption of data at rest
- S3 Lifecycle management for automatic migration of objects to other S3 Storage Classes

S3 One Zone-IA

S3 One Zone-IA is for data that is accessed less frequently, but requires rapid access when needed.

- Same low latency and high throughput performance of S3 Standard
- Designed for durability of 99.999999999% of objects in a single Availability Zone†
- Designed for 99.5% availability over a given year
- Backed with the Amazon S3 Service Level Agreement for availability
- Supports SSL for data in transit and encryption of data at rest
- S3 Lifecycle management for automatic migration of objects to other S3 Storage Classes

Amazon S3 Glacier Instant Retrieval

Amazon S3 Glacier Instant Retrieval is an archive storage class that delivers the lowest-cost storage for long-lived data that is rarely accessed and requires retrieval in milliseconds.

- Data retrieval in milliseconds with the same performance as S3 Standard
- Designed for durability of 99.999999999% of objects across multiple Availability Zones
- Data is resilient in the event of the destruction of one entire Availability Zone
- Designed for 99.9% data availability in a given year
- 128 KB minimum object size
- Backed with the Amazon S3 Service Level Agreement for availability
- S3 PUT API for direct uploads to S3 Glacier Instant Retrieval, and S3 Lifecycle management for automatic migration of objects

Amazon S3 Glacier Flexible Retrieval (Formerly S3 Glacier)

S3 Glacier Flexible Retrieval delivers low-cost storage, up to 10% lower cost (than S3 Glacier Instant Retrieval), for archive data that is accessed 1—2 times per year and is retrieved asynchronously.

- Designed for durability of 99.999999999% of objects across multiple Availability Zones

- Data is resilient in the event of one entire Availability Zone destruction
- Supports SSL for data in transit and encryption of data at rest
- Ideal for backup and disaster recovery use cases when large sets of data occasionally need to be retrieved in minutes, without concern for costs
- Configurable retrieval times, from minutes to hours, with free bulk retrievals
- S3 PUT API for direct uploads to S3 Glacier Flexible Retrieval, and S3 Lifecycle management for automatic migration of objects

S3 on Outposts:

Amazon S3 on Outposts delivers object storage to your on-premises AWS Outposts environment. S3 on Outposts provides a single Amazon S3 storage class, named 'OUTPOSTS', which uses the S3 APIs, and is designed to durably and redundantly store data across multiple devices and servers on your Outposts.

- S3 Object compatibility and bucket management through the S3 SDK
- Designed to durably and redundantly store data on your Outposts
- Encryption using SSE-S3 and SSE-C
- Authentication and authorization using IAM, and S3 Access Points
- Transfer data to AWS Regions using AWS DataSync
- S3 Lifecycle expiration actions

S3 Terminology:

- **Bucket**:- A container for storing objects. All objects are stored in buckets. Buckets have a globally unique name across all of AWS.
- **Object**:- The fundamental entity stored in Amazon S3. An object consists of data, a key (unique within a bucket), and metadata.
- **Key**:- The unique identifier for an object within a bucket. The combination of a bucket, a key, and a version ID uniquely identify each object in a bucket.
- **Amazon Resource Name (ARN)**:- A way to identify AWS resources across services. S3 buckets and objects have ARNs that can be used in IAM policies to control access.
- **Access Control List (ACL)**:- A set of permissions attached to an S3 bucket or object that specify which AWS accounts or users are granted access and what type of access they have (e.g., read or write).
- **Bucket Policy**:- A resource-based AWS Identity and Access Management (IAM) policy that applies to an S3 bucket. It defines who can access the bucket and under what conditions.
- **Storage Class**:- S3 offers different storage classes, each with different performance, durability, and cost characteristics. Common storage classes include STANDARD, INTELLIGENT_TIERING, ONEZONE_IA, GLACIER, and DEEP_ARCHIVE.
- **Versioning**:- A feature that allows you to preserve, retrieve, and restore every version of every object stored in a bucket. Versioning helps protect against accidental deletion or overwrites.
- **Lifecycle Configuration**:- Defines rules for automatically transitioning objects between storage classes or deleting them when they are no longer needed.
- **Transfer Acceleration**:- A feature that enables fast, easy, and secure transfers of files to and from S3 using Amazon CloudFront's globally distributed edge locations.

- **Server-Side Encryption (SSE):-** A feature that helps protect data at rest by automatically encrypting objects when they are stored in S3. There are different SSE options, including SSE-S3, SSE-KMS, and SSE-C.
- **Multipart Upload:-** A feature that allows you to upload large objects in parts, which can improve performance and reliability, especially for large files.
- **Event Notifications:-** S3 can generate events when certain operations occur, such as object creation or deletion. You can configure event notifications to trigger AWS Lambda functions or SQS queues.
- **Presigned URL:-** A URL that provides temporary access to a specific S3 object. Presigned URLs are often used to grant temporary access to private objects without requiring the requester to have AWS credentials.

S3 Advance Features:

- **Versioning:-** S3 supports versioning, which allows you to preserve, retrieve, and restore every version of every object stored in a bucket. This feature is useful for data protection and recovery in case of accidental deletion or overwrites.
- **Cross-Region Replication (CRR):-** CRR enables automatic, asynchronous replication of objects across different AWS regions. This can be useful for disaster recovery, compliance, and low-latency access to objects.
- **Transfer Acceleration:-** Amazon S3 Transfer Acceleration allows for faster uploads and downloads of objects to and from S3 by using Amazon CloudFront's globally distributed edge locations.
- **Event Notifications:-** S3 can generate events in response to specific operations on objects, such as object creation, deletion, or loss of availability. You can configure event notifications to trigger AWS Lambda functions, SQS queues, or SNS topics.
- **Multipart Upload:-** Multipart upload enables you to upload large objects in parts, which can be uploaded in parallel and then combined into a single object. This can improve performance, especially for large files.
- **Transfer Manager:-** The S3 Transfer Manager is a client-side library that provides a simple interface for managing transfers to and from Amazon S3. It automatically breaks up large files into smaller parts and uploads them in parallel, improving efficiency.
- **Storage Classes:-** S3 offers different storage classes, each with different performance, durability, and cost characteristics. These include STANDARD, INTELLIGENT_TIERING, ONEZONE_IA, GLACIER, and DEEP_ARCHIVE.
- **Server-Side Encryption (SSE):-** S3 supports server-side encryption to help protect data at rest. SSE options include SSE-S3, SSE-KMS (Key Management Service), and SSE-C (Customer Provided Keys).
- **Access Control:-** S3 allows you to control access to your buckets and objects using bucket policies, IAM policies, Access Control Lists (ACLs), and query string authentication. Fine-grained access control can be achieved using IAM roles and policies.

- **Lifecycle Policies**:- You can define lifecycle policies to automatically transition objects between storage classes or delete them when they are no longer needed. This helps in optimizing costs and storage efficiency.
- **Intelligent-Tiering**:- Intelligent-Tiering is a storage class that automatically moves objects between two access tiers—frequent and infrequent access—based on changing access patterns. It is designed to optimize costs for data with unknown or changing access patterns.
- **Amazon S3 Select**:- S3 Select allows you to retrieve only a subset of data from an object by using simple SQL expressions. This can improve query performance and reduce data transfer costs.
- **Requester Pays**:- With Requester Pays buckets, the requester (the user accessing the data) pays for the data transfer and request costs associated with the transfer of data from a bucket.

Storage logging and monitoring:

- Automated monitoring tools
 - Amazon CloudWatch metrics for Amazon S3
 - Tracks health of S3 resources and configure billing alerts when it reaches defined threshold.
 - AWS CloudTrail
 - Record actions taken by a user, a role, or an AWS service in Amazon S3.
 - CloudTrail logs provide you with detailed API tracking for S3 bucket-level and object-level operations.
- Manual monitoring tools
 - Server access logging
 - Get detailed records for the requests that are made to a bucket.
 - You can use server access logs for many use cases, such as conducting security and access audits, learning about your customer base, and understanding your Amazon S3 bill.
 - AWS Trusted Advisor
 - Evaluate your account by using AWS best practice checks to identify ways to optimize your AWS infrastructure, improve security and performance, reduce costs, and monitor service quotas.

S3 features: <https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html#S3Features>

Creating, configuring, and working with Amazon S3 buckets:

<https://docs.aws.amazon.com/AmazonS3/latest/userguide/creating-buckets-s3.html>

Monitoring Amazon S3: <https://docs.aws.amazon.com/AmazonS3/latest/userguide/monitoring-overview.html>

Hosting a static website using Amazon S3:

<https://docs.aws.amazon.com/AmazonS3/latest/userguide/WebsiteHosting.html>

S3 on Outposts: <https://docs.aws.amazon.com/AmazonS3/latest/userguide/S3onOutposts.html>

To give public access of objects over internet, create a bucket policy & add this:

```
{
```

```
"Version": "2012-10-17",
"Statement": [
  {
    "Sid": "PublicReadGetObject",
    "Effect": "Allow",
    "Principal": "*",
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::wiprotts301/*"
  }
]
}
```

Amazon IAM

- Identity & Access Management.
- It helps in securely connect & access AWS resources.
- You can centrally manage permissions for users to access resources.
- Takes care of authentication & authorization.

IAM features:

- **Shared access to your AWS account** - You can grant other people permission to administer and use resources in your AWS account without having to share your password or access key.
- **Granular permissions** - You can grant different permissions to different people for different resources.
- **Secure access to AWS resources for applications that run on Amazon EC2** - You can use IAM features to securely provide credentials for applications that run on EC2 instances.
- **Multi-factor authentication (MFA)** - You can add two-factor authentication to your account and to individual users for extra security.
- **Identity federation** - You can allow users who already have passwords elsewhere.
- **Eventually Consistent** - IAM achieves high availability by replicating data across multiple servers within Amazon's data centers around the world. If a request to change some data is successful, the change is committed and safely stored. However, the change must be replicated across IAM, which can take some time.
- **Free to use** - AWS Identity and Access Management (IAM) and AWS Security Token Service (AWS STS) are features of your AWS account offered at no additional charge. You are charged only when you access other AWS services using your IAM users or AWS STS temporary security credentials.

Accessing IAM:

- AWS Management Console
- AWS Command Line Tools
- AWS SDKs
- IAM Query API

When you are performing different job functions:

- Service user
- Service administrator
- IAM Administrator

When you assume an IAM role:

- Federated user access.
- Temporary IAM user permissions
- Cross-account access - You can use an IAM role to allow someone (a trusted principal) in a different account to access resources in your account.
- Cross-service access - Some AWS services use features in other AWS services.
 - o *Principal permissions*
 - When you use an IAM user or role to perform actions in AWS, you are considered a principal. Policies grant permissions to a principal.
 - When you use some services, you might perform an action that then

triggers another action in a different service.

- *Service role*

- A service role is an IAM role that a service assumes to perform actions on your behalf.
- An IAM administrator can create, modify, and delete a service role from within IAM.

- *Service-Linked roles*

- A service-linked role is a type of service role that is linked to an AWS service.
- The service can assume the role to perform an action on your behalf.
- Service-linked roles appear in your AWS account and are owned by the service.
- An IAM administrator can view, but not edit the permissions for service-linked roles.

- Applications running on Amazon EC2

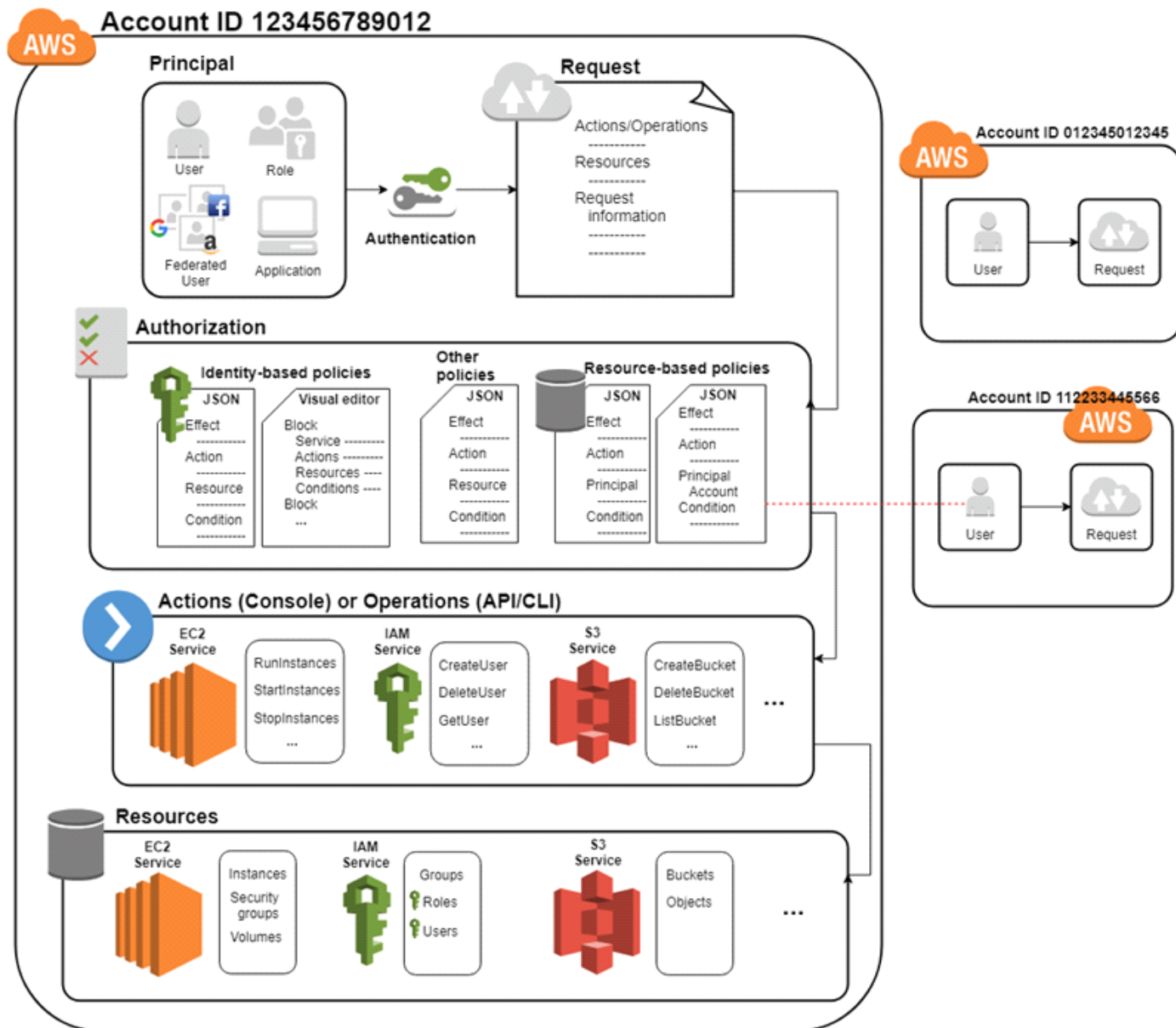
When you create policies and permissions

You grant permissions to a user by creating a policy, which is a document that lists the actions that a user can perform and the resources those actions can affect. Any actions or resources that are not explicitly allowed are denied by default. Policies can be created and attached to principals (users, groups of users, roles assumed by users, and resources).

These policies are used with an IAM role:

- **Trust policy** – Defines which principals can assume the role, and under which conditions. A trust policy is a specific type of resource-based policy for IAM roles. A role can have only one trust policy.
- **Identity-based policies** (inline and managed) – These policies define the permissions that the user of the role is able to perform (or is denied from performing), and on which resources.

How IAM works:



- First, a human user or an application uses their sign-in credentials to authenticate with AWS.
- Authentication is provided by matching the sign-in credentials to a principal (an IAM user, federated user, IAM role, or application) trusted by the AWS account.
- Next, a request is made to grant the principal access to resources.
- Access is granted in response to an authorization request.
- For example, when you first sign in to the console and are on the console Home page, you are not accessing a specific service.
- When you select a service, the request for authorization is sent to that service and it looks to see if your identity is on the list of authorized users, what policies are being enforced to control the level of access granted, and any other policies that might be in effect.
- Authorization requests can be made by principals within your AWS account or from another AWS account that you trust.
- Once authorized, the principal can take action or perform operations on resources in your AWS account.
- For example, the principal could launch a new Amazon Elastic Compute Cloud instance, modify IAM group membership, or delete Amazon Simple Storage Service buckets.

IAM Resource

IAM resources are stored in IAM. You can add, edit, and remove them from IAM.

- user
- group
- role
- policy
- identity-provider object

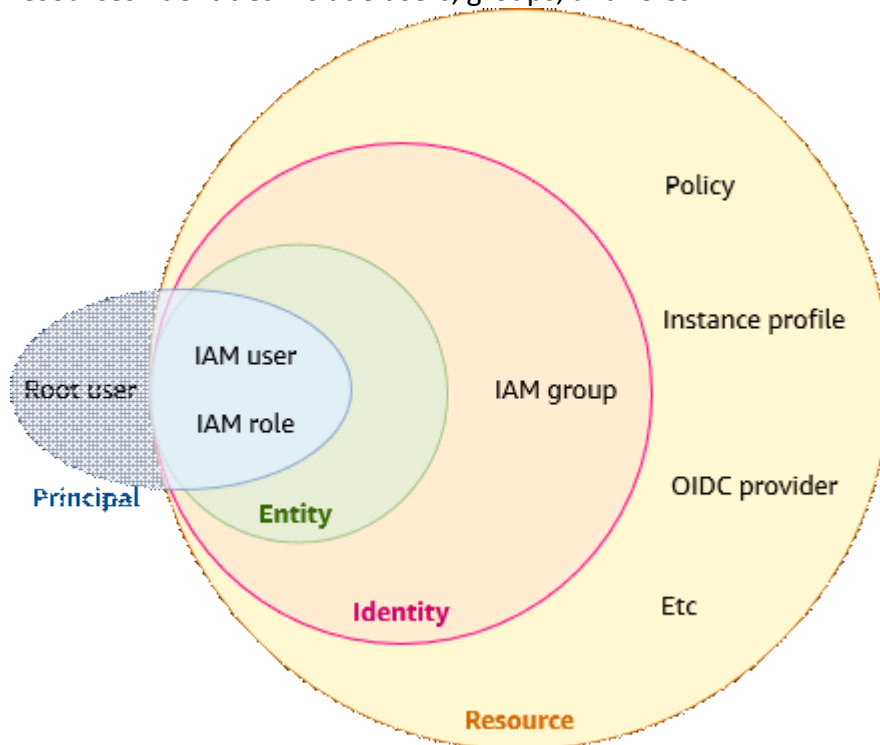
IAM Entity

IAM resources that AWS uses for authentication. Entities can be specified as a Principal in a resource-based policy.

- user
- Role

IAM Identity

An IAM resource that can be authorized in policies to perform actions and to access resources. Identities include users, groups, and roles.



Principals

A person or application that uses the AWS account root user, an IAM user, or an IAM role to sign in and make requests to AWS. Principals include federated users and assumed roles.

Human users

Also known as human identities; the people, administrators, developers, operators, and consumers of your applications.

Workload

A collection of resources and code that delivers business value, such as an application or backend process. Can include applications, operational tools, and components.

Principal

- A principal is a human user or workload that can make a request for an action or operation on an AWS resource.

- After authentication, the principal can be granted either permanent or temporary credentials to make requests to AWS, depending on the principal type.
- IAM users and root user are granted permanent credentials, while roles are granted temporary credentials.

Request

When a principal tries to use the AWS Management Console, the AWS API, or the AWS CLI, that principal sends a request to AWS. The request includes the following information:

- *Actions or operations* – The actions or operations that the principal wants to perform. This can be an action in the AWS Management Console, or an operation in the AWS CLI or AWS API.
- *Resources* – The AWS resource object upon which the actions or operations are performed.
- *Principal* – The person or application that used an entity (user or role) to send the request. Information about the principal includes the policies that are associated with the entity that the principal used to sign in.
- *Environment data* – Information about the IP address, user agent, SSL enabled status, or the time of day.
- *Resource data* – Data related to the resource that is being requested. This can include information such as a DynamoDB table name or a tag on an Amazon EC2 instance.

Amazon IAM - Permissions & Policies

- The access management portion of AWS Identity and Access Management (IAM) helps you define what a principal entity is allowed to do in an account.
- A principal entity is a person or application that is authenticated using an IAM entity (user or role). Access management is often referred to as authorization.
- You manage access in AWS by creating policies and attaching them to IAM identities (users, groups of users, or roles) or AWS resources.
- A policy is an object in AWS that, when associated with an identity or resource, defines their permissions.
- AWS evaluates these policies when a principal uses an IAM entity (user or role) to make a request.
- Permissions in the policies determine whether the request is allowed or denied.
- Most policies are stored in AWS as JSON documents.

Policies and accounts

- If you manage a single account in AWS, then you define the permissions within that account using policies.
- If you manage permissions across multiple accounts, it is more difficult to manage permissions for your users.
- You can use IAM roles, resource-based policies, or access control lists (ACLs) for cross-account permissions.
- However, if you own multiple accounts, we instead recommend using the [AWS Organizations](#) service to help you manage those permissions.

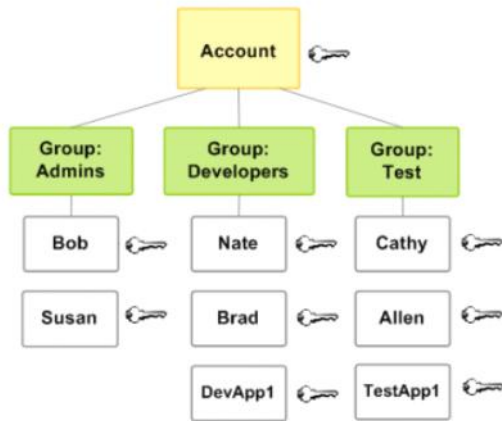
Policies and users

- IAM users are identities in the service.
- When you create an IAM user, they can't access anything in your account until you give them permission.
- You give permissions to a user by creating an identity-based policy, which is a policy that is attached to the user or a group to which the user belongs.

```
{
  "Version": "2012-10-17",
  "Statement": {
    "Effect": "Allow",
    "Action": "dynamodb:*",
    "Resource": "arn:aws:dynamodb:us-east-2:123456789012:table/Books"
  }
}
```

Policies and groups

- You can organize IAM users into IAM groups and attach a policy to a group.
- In that case, individual users still have their own credentials, but all the users in a group have the permissions that are attached to the group.



Identity-based and resource-based policies:

- Identity-based policies are permissions policies that you **attach to an IAM identity**, such as an IAM user, group, or role.
- Resource-based policies are permissions policies that you **attach to a resource** such as an Amazon S3 bucket or an IAM role trust policy.

Identity-based policies control what actions the identity can perform, on which resources, and under what conditions. Identity-based policies can be further categorized:

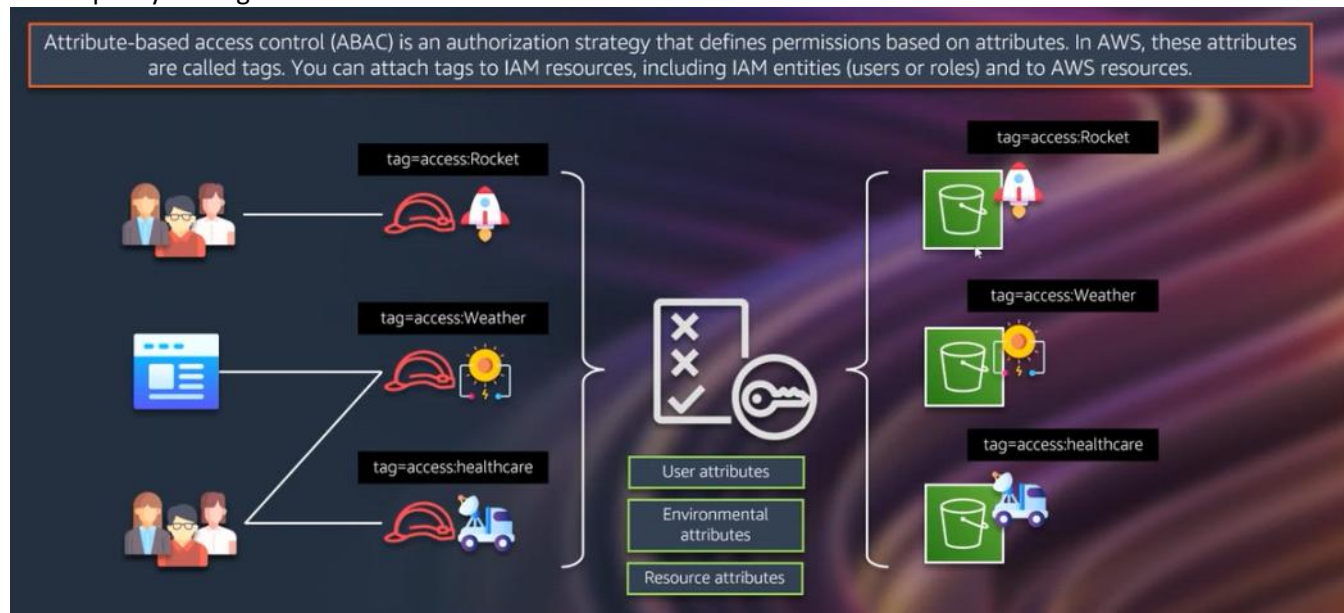
- **Managed policies** – Standalone identity-based policies that you can attach to multiple users, groups, and roles in your AWS account. You can use two types of managed policies:
 - **AWS managed policies** – Managed policies that are created and managed by AWS. If you are new to using policies, we recommend that you start by using AWS managed policies.
 - **Customer managed policies** – Managed policies that you create and manage in your AWS account. Customer managed policies provide more precise control over your policies than AWS managed policies. You can create, edit, and validate an IAM policy in the visual editor or by creating the JSON policy document directly. For more information, see [Creating IAM policies](#) and [Editing IAM policies](#).
- **Inline policies** – Policies that you create and manage and that are embedded directly into a single user, group, or role. In most cases, we don't recommend using inline policies.

Resource-based policies control what actions a specified principal can perform on that resource and under what conditions.

- Resource-based policies are inline policies, and there are no managed resource-based policies.
- To enable cross-account access, you can specify an entire account or IAM entities in another account as the principal in a resource-based policy.
- The IAM service supports only one type of resource-based policy called a role trust policy, which is attached to an IAM role.

AWS ABAC

- ABAC stands for Attribute-based access control.
- ABAC is an authorization strategy that defines permissions based on attributes.
- In AWS, these attributes are called **tags**.
- You can attach tags to IAM resources, including IAM entities (users or roles) and to AWS resources.
- You can create a single ABAC policy or small set of policies for your IAM principals.
- These ABAC policies can be designed to allow operations when the principal's tag matches the resource tag.
- ABAC is helpful in environments that are growing rapidly and helps with situations where policy management becomes cumbersome.



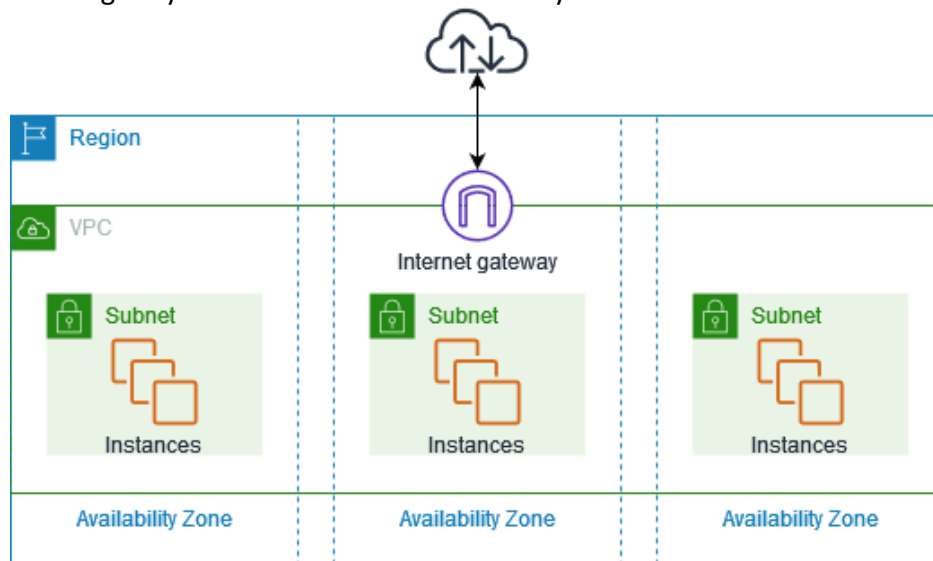
[Creating ABAC tutorial.](#)

ABAC provides the following advantages over the traditional RBAC model:

- ABAC permissions scale with innovation.
- ABAC requires fewer policies.
- Using ABAC, teams can change and grow quickly.
- Granular permissions are possible using ABAC.
- Use employee attributes from your corporate directory with ABAC.

Amazon VPC

- Virtual Private Cloud (VPC).
- It's a logically isolated virtual network that you've defined.



Features

The following features help you configure a VPC to provide the connectivity that your applications need:

- Virtual private clouds (VPC)
 - A VPC is a virtual network that closely resembles a traditional network that you'd operate in your own data center. After you create a VPC, you can add subnets.
- Subnets
 - A subnet is a range of IP addresses in your VPC. A subnet must reside in a single Availability Zone. After you add subnets, you can deploy AWS resources in your VPC.
- IP addressing
 - You can assign IP addresses, both IPv4 and IPv6, to your VPCs and subnets.
 - You can also bring your public IPv4 and IPv6 GUA addresses to AWS and allocate them to resources in your VPC, such as EC2 instances, NAT gateways, and Network Load Balancers.
- Routing
 - Use route tables to determine where network traffic from your subnet or gateway is directed.
- Gateways and endpoints
 - A gateway connects your VPC to another network. For example, use an internet gateway to connect your VPC to the internet.
 - Use a VPC endpoint to connect to AWS services privately, without the use of an internet gateway or NAT device.
- Peering connections
 - Use a VPC peering connection to route traffic between the resources in two VPCs.
- Traffic Mirroring
 - Copy network traffic from network interfaces and send it to security and monitoring appliances for deep packet inspection.
- Transit gateways
 - Use a transit gateway, which acts as a central hub, to route traffic between your

- VPCs, VPN connections, and AWS Direct Connect connections.
- VPC Flow Logs
 - A flow log captures information about the IP traffic going to and from network interfaces in your VPC.
- VPN connections
 - Connect your VPCs to your on-premises networks using AWS Virtual Private Network (AWS VPN).

Working with Amazon VPC:

- AWS Management Console
- AWS Command Line Interface (AWS CLI)
- AWS SDKs
- Query API

The types of public IPv4 addresses are:

- Elastic IP addresses (EIPs)
 - Static, public IPv4 addresses provided by Amazon that you can associate with an EC2 instance, elastic network interface, or AWS resource.
- EC2 public IPv4 addresses
 - Public IPv4 addresses assigned to an EC2 instance by Amazon (if the EC2 instance is launched into a default subnet or if the instance is launched into a subnet that's been configured to automatically assign a public IPv4 address).
- BYOIPv4 addresses
 - Public IPv4 addresses in the IPv4 address range that you've brought to AWS using Bring your own IP addresses (BYOIP).
- Service-managed IPv4 addresses
 - Public IPv4 addresses automatically provisioned on AWS resources and managed by an AWS service. For example, public IPv4 addresses on Amazon ECS, Amazon RDS, or Amazon Workspaces.

How Amazon VPC works:



VPCs and subnets

- A virtual private cloud (VPC) is a virtual network dedicated to your AWS account. It is logically isolated from other virtual networks in the AWS Cloud. You can specify an IP address range for the VPC, add subnets, add gateways, and associate security groups.
- A subnet is a range of IP addresses in your VPC. You launch AWS resources, such as Amazon EC2 instances, into your subnets. You can connect a subnet to the internet, other VPCs, and your own data centers, and route traffic to and from your subnets using route tables.

Default and nondefault VPCs

- If your account was created after December 4, 2013, it comes with a default VPC in each Region. A default VPC is configured and ready for you to use. For example, it has a default subnet in each Availability Zone in the Region, an attached internet gateway, a route in the main route table that sends all traffic to the internet gateway, and DNS settings that automatically assign public DNS hostnames to instances with public IP addresses and enable DNS resolution through the Amazon-provided DNS server. Therefore, an EC2 instance that is launched in a default subnet automatically has access to the internet. If you have a default VPC in a Region and you don't specify a subnet when you launch an EC2 instance into that Region, we choose one of the default subnets and launch the instance into that subnet.
- You can also create your own VPC, and configure it as you need. This is known as a nondefault VPC. Subnets that you create in your nondefault VPC and additional subnets that you create in your default VPC are called nondefault subnets.

Route tables

- A route table contains a set of rules, called routes, that are used to determine where network traffic from your VPC is directed. You can explicitly associate a subnet with a particular route table. Otherwise, the subnet is implicitly associated with the main route table.
- Each route in a route table specifies the range of IP addresses where you want the traffic to go (the destination) and the gateway, network interface, or connection through which to send the traffic (the target).

AWS private global network

- AWS provides a high-performance, and low-latency private global network that delivers a secure cloud computing environment to support your networking needs. AWS Regions are connected to multiple Internet Service Providers (ISPs) as well as to a private global network backbone, which provides improved network performance for cross-Region traffic sent by customers.
- The following considerations apply:
 - Traffic that is in an Availability Zone, or between Availability Zones in all Regions, routes over the AWS private global network.
 - Traffic that is between Regions always routes over the AWS private global network, except for China Regions.
- Network packet loss can be caused by a number of factors, including network flow collisions, lower level (Layer 2) errors, and other network failures. We engineer and operate our networks to minimize packet loss. We measure packet-loss rate (PLR) across the global backbone that connects the AWS Regions. We operate our backbone network to target a p99 of the hourly PLR of less than 0.0001%.

VPC resources

Each VPC automatically comes with the following resources:

- Default DHCP option set
- Default network ACL
- Default security group
- Main route table

You can create the following resources for your VPC:

- Network ACLs
- Custom route tables
- Security groups
- Internet gateway

- NAT gateways

Amazon EC2

Dive into EC2 (Elastic Compute Cloud) and understand instance types, AMIs, and security groups.

Learn about Auto Scaling, Elastic Load Balancing (ELB), and launch configurations.

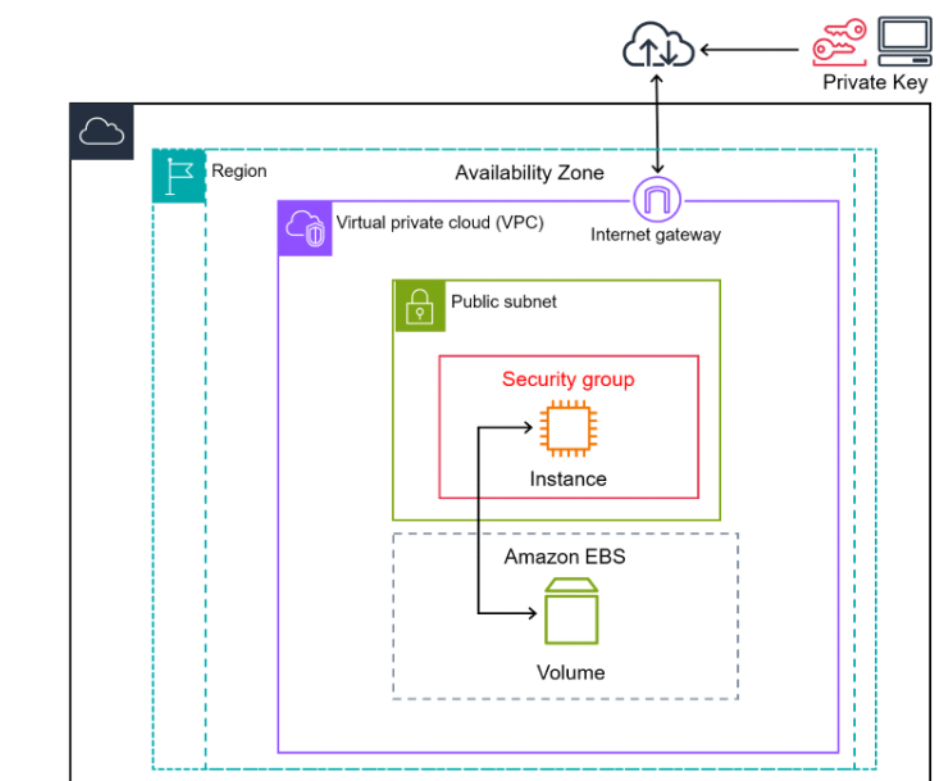
- Access reliable, scalable infrastructure on demand. Scale capacity within minutes with SLA commitment of 99.99% availability.
- Provide secure compute for your applications. Security is built into the foundation of Amazon EC2 with the AWS Nitro System.
- Optimize performance and cost with flexible options like AWS Graviton-based instances, Amazon EC2 Spot instances, and AWS Savings Plans.
- Migrate and build apps with ease using AWS Migration Tools, AWS Managed Services, or Amazon Lightsail. Learn how AWS can help.

Use cases

- Run cloud-native and enterprise applications
- Scale for HPC applications
- Develop for Apple platforms
- Train and deploy ML applications

Amazon Elastic Compute Cloud (Amazon EC2):

- Amazon Elastic Compute Cloud (Amazon EC2) provides on-demand, scalable computing capacity in the Amazon Web Services (AWS) Cloud.
- Using Amazon EC2 reduces hardware costs so you can develop and deploy applications faster.
- You can use Amazon EC2 to launch as many or as few virtual servers as you need, configure security and networking, and manage storage. You can add capacity (scale up) to handle compute-heavy tasks, such as monthly or yearly processes, or spikes in website traffic.
- When usage decreases, you can reduce capacity (scale down) again.



Features of Amazon EC2

Amazon EC2 provides the following high-level features:

- **Instances** - Virtual servers
- **Amazon Machine Images (AMIs)** - Preconfigured templates
- **Instance types** - Various configurations of CPU, memory, storage, networking capacity, and graphics hardware for your instances.
- **Key pairs** - Secure login information for your instances. AWS stores the public key and you store the private key in a secure place.
- **Instance store volumes** - Storage volumes for temporary data that is deleted when you stop, hibernate, or terminate your instance.
- **Amazon EBS volumes** - Persistent storage volumes for your data using Amazon Elastic Block Store (Amazon EBS).
- **Regions, Availability Zones, Local Zones, AWS Outposts, and Wavelength Zones** - Multiple physical locations for your resources, such as instances and Amazon EBS volumes.
- **Security groups** - A virtual firewall that allows you to specify the protocols, ports, and source IP ranges that can reach your instances, and the destination IP ranges to which your instances can connect.
- **Elastic IP addresses** - Static IPv4 addresses for dynamic cloud computing.
- **Tags** - Metadata that you can create and assign to your Amazon EC2 resources.
- **Virtual private clouds (VPCs)** - Virtual networks you can create that are logically isolated from the rest of the AWS Cloud. You can optionally connect these virtual networks to your own network.

Access Amazon EC2

You can create and manage your Amazon EC2 instances using the following interfaces:

- Amazon EC2 console
- AWS Command Line Interface
- AWS Tools for PowerShell
- AWS CloudFormation
- Query API

Pricing for Amazon EC2

Amazon EC2 provides the following pricing options:

- **Free Tier** - [free tier options](#)
- **On-Demand Instances** - Pay for the instances that you use by the second, with a minimum of 60 seconds
- **Savings Plans** - commitment to a consistent amount of usage, in USD per hour, for a term of 1 or 3 years.
- **Reserved Instances** - commitment to a specific instance configuration, including instance type and Region, for a term of 1 or 3 years.
- **Spot Instances** - Request unused EC2 instances, which can reduce your Amazon EC2 costs significantly.
- **Dedicated Hosts** - a physical EC2 server that is fully dedicated for your use
- **On-Demand Capacity Reservations** - Reserve compute capacity for your EC2 instances in a specific Availability Zone for any duration of time.
- **Per-second billing** - Removes the cost of unused minutes and seconds from your bill.

Create your EC2 resources and launch your EC2 instance: [follow here](#).

AWS Cloud Shell Commands:

IAM Commands:

To create an IAM group:

```
# aws iam create-group --group-name group_name
```

To add a user to the group that you created, use the following command:

```
# aws iam add-user-to-group --group-name east_coast --user-name username
```

To verify that the user is in the group, use the following command:

```
# aws iam get-group --group-name group_name
```

Alias Creation in AWS Account:

To list current alias:

```
aws iam list-account-aliases
```

To create an alias:

```
aws iam create-account-alias --account-alias new_alias_name
```

To delete an AWS account ID alias:

```
aws iam delete-account-alias --account-alias new_alias_name
```

list all user's info

```
aws iam list-users
```

list all user's usernames

```
aws iam list-users --output text | cut -f 2
```

list current user's info

```
aws iam get-user
```

list current user's access keys

```
aws iam list-access-keys
```

create multiple new users, from a file (need to upload user-names.txt file to CLI)

```
allUsers=$(cat ./user-names.txt)
for userName in $allUsers; do
    aws iam create-user \
        --user-name $userName
done
```

get a specific user's info

```
aws iam get-user --user-name aniltf
```

#####

Cheat sheet [URL](#)

Amazon Elastic Block Store (EBS)

- AWS EBS provides block level storage volumes for use with EC2 instances.
- EBS volumes behave like raw, unformatted block devices.
- You can mount these volumes as devices on your instances.
- EBS volumes that are attached to an instance are exposed as storage volumes that persist independently from the life of the instance.
- You can create a file system on top of these volumes, or use them in any way you would use a block device (such as a hard drive).

Features of Amazon EBS:

- <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/AmazonEBS.html>

Create an Amazon EBS volume

- <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ebs-creating-volume.html>

Attach an Amazon EBS volume to an instance

- <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ebs-attaching-volume.html>

How Amazon S3 works?

- Amazon S3 is an object storage service that stores data as objects within buckets.
- An object is a file and any metadata that describes the file.
- A bucket is a container for objects.
- To store your data in Amazon S3, you first create a bucket and specify a bucket name and AWS Region.
- Then, you upload your data to that bucket as objects in Amazon S3.
- Each object has a key (or key name), which is the unique identifier for the object within the bucket.

Topics:

Buckets

- A bucket is a container for objects stored in Amazon S3.
- You can store any number of objects in a bucket and can have up to 100 buckets in your account.

Objects

- Objects are the fundamental entities stored in Amazon S3. Objects consist of object data and metadata.
- The metadata is a set of name-value pairs that describe the object.
- These pairs include some default metadata, such as the date last modified, and standard HTTP metadata, such as Content-Type.
- An object is uniquely identified within a bucket by a key (name) and a version ID.

Keys

- An object key (or key name) is the unique identifier for an object within a bucket. Every object in a bucket has exactly one key.
- The combination of a bucket, object key, and optionally, version ID (if S3 Versioning is enabled for the bucket) uniquely identify each object.

S3 Versioning

- You can use S3 Versioning to keep multiple variants of an object in the same bucket.
- With S3 Versioning, you can preserve, retrieve, and restore every version of every object stored in your buckets.

Version ID

- When you enable S3 Versioning in a bucket, Amazon S3 generates a unique version ID for each object added to the bucket.

Bucket policy

- A bucket policy is a resource-based AWS Identity and Access Management (IAM) policy that you can use to grant access permissions to your bucket and the objects in it.
- Only the bucket owner can associate a policy with a bucket. The permissions attached to the bucket apply to all of the objects in the bucket that are owned by the bucket owner.
- Bucket policies are limited to 20 KB in size.

[AWS S3 Pricing](#)

AWS Elastic Load Balancer (ELB)

- Elastic Load Balancing automatically distributes your incoming traffic across multiple targets, such as EC2 instances, containers, and IP addresses, in one or more Availability Zones.
- It monitors the health of its registered targets, and routes traffic only to the healthy targets.
- Elastic Load Balancing scales your load balancer capacity automatically in response to changes in incoming traffic.
- 750 hours free per month between network and application load balancers with the free tier.

Load balancer benefits

- A load balancer distributes workloads across multiple compute resources, such as virtual servers.
- Using a load balancer increases the availability and fault tolerance of your applications.
- You can add and remove compute resources from your load balancer as your needs change, without disrupting the overall flow of requests to your applications.
- You can configure health checks, which monitor the health of the compute resources, so that the load balancer sends requests only to the healthy ones.
- You can also offload the work of encryption and decryption to your load balancer so that your compute resources can focus on their main work.

Accessing Elastic Load Balancing

- You can create, access, and manage your load balancers using any of the following interfaces:
 - o **AWS Management Console**— Provides a web interface that you can use to access Elastic Load Balancing.
 - o **AWS Command Line Interface (AWS CLI)** — Provides commands for a broad set of AWS services, including Elastic Load Balancing. The AWS CLI is supported on Windows, macOS, and Linux. For more information, see AWS Command Line Interface.
 - o **AWS SDKs** — Provide language-specific APIs and take care of many of the connection details, such as calculating signatures, handling request retries, and error handling. For more information, see AWS SDKs.
 - o **Query API**— Provides low-level API actions that you call using HTTPS requests. Using the Query API is the most direct way to access Elastic Load Balancing. However, the Query API requires that your application handle low-level details such as generating the hash to sign the request, and error handling. For more information, see the following:
 - Application Load Balancers and Network Load Balancers
 - Classic Load Balancers

Elastic Load Balancing pricing: <https://aws.amazon.com/elasticloadbalancing/pricing/>

Types of load balancers:

1. *Application Load Balancer*

- o **Layer:** Operates at Layer 7 (application layer) of the OSI model.
- o **Traffic:** Handles HTTP and HTTPS traffic.
- o **Routing:** Provides advanced request routing based on content (e.g., URL, host, headers, or query string).
- o **Use Cases:** Ideal for microservices architectures, API-based applications, and websites where routing decisions depend on the content of the request.

2. *Network Load Balancer*

- **Layer:** Operates at Layer 4 (transport layer) of the OSI model.
- **Traffic:** Designed for TCP and UDP traffic.
- **Routing:** Provides fast and efficient routing with low latency; routes traffic based on IP address and port.
- **Use Cases:** Suitable for high-performance applications requiring millions of connections, such as gaming servers or real-time messaging services.

3. Gateway Load Balancer

- **Layer:** Operates at Layer 3 (network layer) of the OSI model.
- **Traffic:** Handles IP protocol-based traffic.
- **Routing:** Allows you to deploy, scale, and manage virtual network appliances (e.g., firewalls, intrusion detection/prevention systems).
- **Use Cases:** Perfect for scenarios involving network security, monitoring, and traffic management across VPCs.

4. Classic Load Balancer

- **Layer:** Operates at both Layer 4 (TCP/UDP) and Layer 7 (HTTP/HTTPS) of the OSI model.
- **Traffic:** Supports basic routing capabilities for HTTP/HTTPS requests and TCP connections.
- **Features:** Provides simple load balancing across multiple EC2 instances, but lacks advanced features like content-based routing or WebSocket support.
- **Health Checks:** Monitors the health of instances and routes traffic only to healthy ones, using both ping-based and HTTP/HTTPS health checks.
- **Legacy Use:** Recommended for applications built on older architectures, or when transitioning to newer services like ALB or NLB.

Create an Application Load Balancer:

A load balancer takes requests from clients and distributes them across targets in a target group.

Before you begin, ensure that you have a virtual private cloud (VPC) with at least one public subnet in each of the zones used by your targets.

Tasks

- [Step 1: Configure a target group](#)
- [Step 2: Register targets](#)
- [Step 3: Configure a load balancer and a listener](#)
- [Step 4: Test the load balancer](#)

Monitor your Application Load Balancers:

- CloudWatch metrics
- Access logs
- Connection logs
- Request tracing
- CloudTrail logs

Troubleshoot your Application Load Balancers:

- [A registered target is not in service](#)
- [Clients cannot connect to an internet-facing load balancer](#)
- [Requests sent to a custom domain aren't received by the load balancer](#)
- [HTTPS requests sent to the load balancer return "NET::ERR_CERT_COMMON_NAME_INVALID"](#)
- [Load balancer shows elevated processing times](#)
- [The load balancer sends a response code of 000](#)
- [The load balancer generates an HTTP error](#)
- [A target generates an HTTP error](#)
- [An AWS Certificate Manager certificate is not available for use](#)
- [Multi-Line headers are not supported](#)

Creating ELB - step by step:

1. Login to AWS Console.
2. Search or switch to EC2 dashboard page.
3. Create 2 EC2 instances with user data given below:

```
#!/bin/bash
sudo su -
sudo yum install -y httpd php && sudo systemctl start httpd && sudo systemctl
enable httpd
echo "<html><body><h1 align='center'>Hostname: $(hostname)</h1></body>
</html>" > /var/www/html/index.html
```

4. And launch them.
5. And search for target groups:

The screenshot shows the AWS Management Console interface. On the left, the navigation sidebar is visible with categories like Savings Plans, Images, Elastic Block Store, Network & Security, Load Balancing, and Auto Scaling. The 'Target Groups' link under 'Load Balancing' is highlighted in yellow. The main console area displays the 'Resources' section for the 'US East (Ohio)' region, showing counts for various EC2 resources. Below this, there's a 'Launch instance' section with a prominent orange 'Launch instance' button. To the right, the 'Service health' section shows the region as 'US East (Ohio)' and lists available zones. The 'Scheduled events' section indicates no events are currently scheduled.

6. Click on "Create Target Group"

This screenshot shows the 'Target groups' page in the AWS console. At the top, there's a search bar and a 'Create target group' button highlighted in yellow. Below this is a table with columns: Name, ARN, Port, Protocol, Target type, Load balancer, and VPC ID. The table is currently empty, and a message in the center states 'No target groups' and 'You don't have any target groups in us-east-2'. A 'Create target group' button is located at the bottom of the page.

7. Fill required details:

Step 2
Register targets

Basic configuration

Settings in this section can't be changed after the target group is created.

Choose a target type

☒ **Instances**

- Supports load balancing to instances within a specific VPC.
- Facilitates the use of [Amazon EC2 Auto Scaling](#) to manage and scale your EC2 capacity.

☐ **IP addresses**

- Supports load balancing to VPC and on-premises resources.
- Facilitates routing to multiple IP addresses and network interfaces on the same instance.
- Offers flexibility with microservice-based architectures, simplifying inter-application communication.
- Supports IPv6 targets, enabling end-to-end IPv6 communication, and IPv4-to-IPv6 NAT.

☐ **Lambda function**

- Facilitates routing to a single Lambda function.
- Accessible to Application Load Balancers only.

☐ **Application Load Balancer**

- Offers the flexibility for a Network Load Balancer to accept and route TCP requests within a specific VPC.
- Facilitates using static IP addresses and PrivateLink with an Application Load Balancer.

Target group name

my-target-group-1

A maximum of 32 alphanumeric characters including hyphens are allowed, but the name must not begin or end with a hyphen.

Protocol : Port

Choose a protocol for your target group that corresponds to the Load Balancer type that will route traffic to it. Some protocols now include anomaly detection for the targets and you can set mitigation options once your target group is created. This choice cannot be changed after creation.

HTTP .80

8. Select as given below:

IP address type

Only targets with the indicated IP address type can be registered to this target group.

☒ **IPv4**

Each instance has a default network interface (eth0) that is assigned the primary private IPv4 address. The instance's primary private IPv4 address is the one that will be applied to the target.

☐ **IPv6**

Each instance you register must have an assigned primary IPv6 address. This is configured on the instance's default network interface (eth0). [Learn more](#)

VPC

Select the VPC with the instances that you want to include in the target group. Only VPCs that support the IP address type selected above are available in this list.

VPC-A
vpc-0c5df5c4375250e18
IPv4: 192.168.0.0/16

Protocol version

☒ **HTTP1**

Send requests to targets using HTTP/1.1. Supported when the request protocol is HTTP/1.1 or HTTP/2.

☐ **HTTP2**

Send requests to targets using HTTP/2. Supported when the request protocol is HTTP/2 or gRPC, but gRPC-specific features are not available.

☐ **gRPC**

Send requests to targets using gRPC. Supported when the request protocol is gRPC.

Health checks

The associated load balancer periodically sends requests, per the settings below, to the registered targets to test their status.

Health check protocol

HTTP

Health check path

Use the default path of "/" to ping the root, or specify a custom path if preferred.

/

Up to 1024 characters allowed.

9. Then, click on NEXT.

10. Include all available instances to register:

Register targets

This is an optional step to create a target group. However, to ensure that your load balancer routes traffic to this target group you must register your targets.

Available instances (2/2)

☒

i-077bf06be0a44a619

websvr-1

Running

websvr-launch-wizard-1

us-east-2a

192.168.1.30

subnet-087bac02edb80ad96

☒

i-0ddfcc9eef07bd929

websvr-2

Running

websvr-launch-wizard-1

us-east-2a

192.168.1.42

subnet-087bac02edb80ad96

2 selected

Ports for the selected instances

Ports for routing traffic to the selected instances.

1-65535 (separate multiple ports with comma)

Include as pending below

Review targets

Targets (0)

☒ Show only pending

Remove

Health status

Instance ID

Name

Port

State

Security groups

Zone

Private IPv4 address

Subnet ID

Launch time

No instances added yet

Specify instances above, or leave the group empty if you prefer to add targets later.

0 pending

Cancel

Previous

Create target group

Review targets

Targets (2)

☒ Show only pending

Remove

Health status

Instance ID

Name

Port

State

Security groups

Zone

Private IPv4 address

Subnet ID

Launch time

×

Pending

i-077bf06be0a44a619

websvr-1

80

Running

websvr-launch-wizard-1

us-east-2a

192.168.1.30

subnet-087bac02edb80ad96

December 1, 2023, 10:53 (UTC+05:30)

×

Pending

i-0ddfcc9eef07bd929

websvr-2

80

Running

websvr-launch-wizard-1

us-east-2a

192.168.1.42

subnet-087bac02edb80ad96

December 1, 2023, 10:53 (UTC+05:30)

2 pending

Cancel

Previous

Create target group

Successfully created the target group: my-target-group-1. Anomaly detection is automatically applied to all registered targets. Results can be viewed in the Targets tab.

EC2 > Target groups > my-target-group-1

Introducing Automatic Target Weights (ATW) to increase application availability

Automatic Target Weights is achieved by turning on anomaly mitigation, which provides responsive, dynamic distribution of traffic to targets based on anomaly detection results. All HTTP/HTTPS target groups now include anomaly detection by default. [Learn more](#)

my-target-group-1

Details

arn:aws:elasticloadbalancing:us-east-2:805116530761:targetgroup/my-target-group-1/68Hk4b30c4485

Target type

Instance

IP address type

IPv4

Protocol - Port

HTTP: 80

Load balancer

None associated

Protocol version

HTTP1

VPC

vpc-0c5df5c4375250e18

2

Total targets

0 Healthy

0 Unhealthy

0 Anomalous

2 Unused

0 Initial

0 Draining

Distribution of targets by Availability Zone (AZ)

Select values in this table to see corresponding filters applied to the Registered targets table below.

Targets

Monitoring

Health checks

Attributes

Tags

Registered targets (2)

☐

i-0ddfcc9eef07bd929

websvr-2

80

us-east-2a

Unused

Target group is not co...

Normal

☐

i-077bf06be0a44a619

websvr-1

80

us-east-2a

Unused

Target group is not co...

Normal

11.

12.

13. And search for "Load Balancer", on EC2 Dashboard.

EC2 > Load balancers

Load balancers

Elastic Load Balancing scales your load balancer capacity automatically in response to changes in incoming traffic.

Filter load balancers

Name	DNS name	State	VPC ID
No load balancers			
You don't have any load balancers in us-east-2			

Create load balancer

15. Select Application load balancer:

EC2 > Load balancers > Compare and select load balancer type

Compare and select load balancer type

A complete feature-by-feature comparison along with detailed highlights is also available. [Learn more](#)

Load balancer types

Application Load Balancer

Choose an Application Load Balancer when you need a flexible feature set for your applications with HTTP and HTTPS traffic. Operating at the request level, Application Load Balancers provide advanced routing and visibility features targeted at application architectures, including microservices and containers.

Create

Network Load Balancer

Choose a Network Load Balancer when you need ultra-high performance, TLS offloading at scale, centralized certificate deployment, support for UDP, and static IP addresses for your applications. Operating at the connection level, Network Load Balancers are capable of handling millions of requests per second securely while maintaining ultra-low latencies.

Create

Gateway Load Balancer

Choose a Gateway Load Balancer when you need to deploy and manage a fleet of third-party virtual appliances that support GENEVE. These appliances enable you to improve security, compliance, and policy controls.

Create

16. Fill basic configs:

Basic configuration

Load balancer name
Name must be unique within your AWS account and can't be changed after the load balancer is created.

my-lb-1

A maximum of 32 alphanumeric characters including hyphens are allowed, but the name must not begin or end with a hyphen.

Scheme [Info](#)
Scheme can't be changed after the load balancer is created.

☒ Internet-facing
An internet-facing load balancer routes requests from clients over the internet to targets. Requires a public subnet. [Learn more](#)

☐ Internal
An internal load balancer routes requests from clients to targets using private IP addresses.

IP address type [Info](#)
Select the type of IP addresses that your subnets use.

☒ IPv4

Basic configuration

Load balancer name
Name must be unique within your AWS account and can't be changed after the load balancer is created.

my-lb-1

A maximum of 32 alphanumeric characters including hyphens are allowed, but the name must not begin or end with a hyphen.

Scheme [Info](#)
Scheme can't be changed after the load balancer is created.

☒ **Internet-facing**
An internet-facing load balancer routes requests from clients over the internet to targets. Requires a public subnet. [Learn more](#)

☐ **Internal**
An internal load balancer routes requests from clients to targets using private IP addresses.

IP address type [Info](#)
Select the type of IP addresses that your subnets use.

☒ **IPv4**
Recommended for internal load balancers.

☐ **Dualstack**
Includes IPv4 and IPv6 addresses.

17. Select network. (ensure to have at least 2 subnets).

Network mapping [Info](#)
The load balancer routes traffic to targets in the selected subnets, and in accordance with your IP address settings.

VPC [Info](#)
Select the virtual private cloud (VPC) for your targets or you can [create a new VPC](#). Only VPCs with an internet gateway are enabled for selection. The selected VPC can't be changed after the load balancer is created. To confirm the VPC for your targets, view your [target groups](#).

VPC-A
vpc-0c5df5c4375250e18
IPv4: 192.168.0.0/16

Mappings [Info](#)
Select at least two Availability Zones and one subnet per zone. The load balancer routes traffic to targets in these Availability Zones only. Availability Zones that are not supported by the load balancer or the VPC are not available for selection.

☒ **us-east-2a (use2-az1)**

Subnet
subnet-087bac02edb80ad96 vpc-a-subnet-1

IPv4 address
Assigned by AWS

☒ **us-east-2b (use2-az2)**

Subnet
subnet-0a962769082dcd173 vpc-a-subnet-2

IPv4 address
Assigned by AWS

18. Select appropriate security group with port 80 & 443 enabled.

Security groups [Info](#)
A security group is a set of firewall rules that control the traffic to your load balancer. Select an existing security group, or you can [create a new security group](#).

Security groups
Select up to 5 security groups

websvr-launch-wizard-1
sg-0e05ae55ee6978f5d VPC: vpc-0c5df5c4375250e18

19. Select proper target group (you created earlier):

Listeners and routing [Info](#)
A listener is a process that checks for connection requests using the port and protocol you configure. The rules that you define for a listener determine how the load balancer routes requests to its registered targets.

▼ Listener HTTP:80 [Remove](#)

Protocol Port Default action [Info](#)

HTTP : 80 Forward to my-target-group-1 HTTP

Listeners and routing [Info](#)

A listener is a process that checks for connection requests using the port and protocol you configure. The rules that you define for a listener determine how the load balancer routes requests to its registered targets.

▼ Listener HTTP:80 Remove

Protocol
HTTP

Port
80
1-65535

Default action
Forward to my-target-group-1
Target type: Instance, IPv4
[Create target group](#)

Info

HTTP

↺

Listener tags - optional

Consider adding tags to your listener. Tags enable you to categorize your AWS resources so you can more easily manage them.

Add listener tag

You can add up to 50 more tags.

Add listener

20. Review everything you filled in summary & click on "Create Load Balancer":

Summary

Review and confirm your configurations. [Estimate cost](#)

Basic configuration [Edit](#)

my-lb-1

- Internet-facing
- IPv4

Security groups [Edit](#)

- websvr-launch-wizard-1
[sg-0e05ae55ee6978f5d](#)

Network mapping [Edit](#)

VPC [vpc-0c5df5c4375250e18](#)
VPC-A

- us-east-2a
[subnet-087bac02edb80ad96](#)
vpc-a-subnet-1
- us-east-2b
[subnet-0a962769082dcd173](#)
vpc-a-subnet-2

Listeners and routing [Edit](#)

- HTTP:80 defaults to [my-target-group-1](#)

Add-on services [Edit](#)

None

Tags [Edit](#)

None

Attributes

ⓘ

Certain default attributes will be applied to your load balancer. You can view and edit them after creating the load balancer.

Cancel

Create load balancer

21. Wait until provisioning state changes to ACTIVE:

Load balancers (1)

Elastic Load Balancing scales your load balancer capacity automatically in response to changes in incoming traffic.

Filter load balancers

1 match

my-lb-1

Clear filters

☐	Name	DNS name	State	VPC ID
☐	my-lb-1	my-lb-1-95310984.us-east-2.elb.amazonaws.com	Provisioning..	vpc-0c5df5c4375250e18

22. Then, Copy the "DNS name" and paste it to the browser to view of the instances are running and getting load balancing.

Load balancers (1)

Elastic Load Balancing scales your load balancer capacity automatically in response to changes in incoming traffic.

Filter load balancers 1 match

my-lb-1 X

Clear filters

☐	Name	DNS name	State	VPC ID
☐	my-lb-1	my-lb-1-95310984.us-east-2.elb.amazonaws.com	Active	vpc-0c5df5c4375250e18

23. After accessing the LB URL:

Sample Website x +

← → ↻ 🏠 ⚠ Not secure my-lb-1-95310984.us-east-2.elb.amazonaws.com ↗ ☆ 🛡 ⚙ 🎵

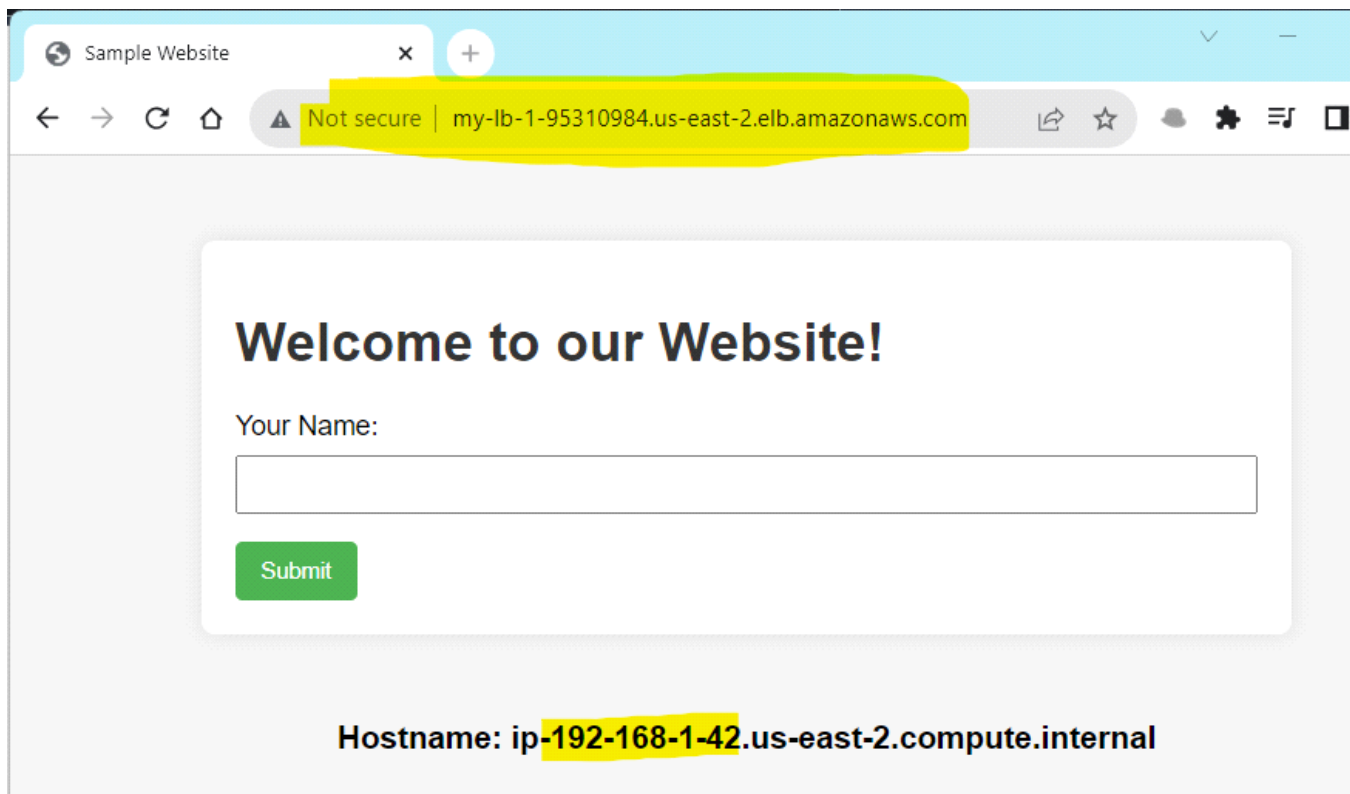
Welcome to our Website!

Your Name:

Submit

Hostname: ip-192-168-1-30.us-east-2.compute.internal

After refreshing the same URL:



24. If want to use the same in Auto scaling group practical, DO NOT DELETE.

25. Else,

- a. Unregister the EC2 instances
- b. Delete the load balancer
- c. Delete the target group.

The Top 12 Load Balancing Techniques to Keep on Your Radar in 2024

1. Round Robin (Weighted/Unweighted):

This classic method distributes incoming requests evenly across all servers. The weighted version takes into account each server's capacity, giving more capable servers a larger share of the load.

2. Least Connections (Weighted/Unweighted):

This technique routes new requests to the server with the fewest current connections, ensuring a balanced workload. Weighted least connections consider the server capacity, prioritizing more powerful servers.

3. Least Response Time:

Focuses on responsiveness, directing new requests to the server with the fastest response time, enhancing overall user experience.

4. Least Bandwidth Method:

This method sends traffic to the server utilizing the least bandwidth, helping to avoid overloading any single server.

5. Least Packets:

Aimed at balancing network load, this approach routes requests to the server that has processed the fewest packets, indicating a lighter workload.

6. IP Hash:

A client's IP address is used to determine which server will handle their request, ensuring a consistent user-server relationship.

7. Sticky Sessions (Session Persistence/Affinity):

Ensures that once a client is connected to a server, they stay connected to that server for the duration of their session, which is crucial for transactional websites.

8. Layer 7 Load Balancing:

Goes beyond simple load distribution to make routing decisions based on the content of the traffic, like URLs or data types in HTTP headers.

9. Geographical Load Balancing:

This method routes users to the nearest server based on their geographic location, reducing latency and improving speeds.

10. DNS Load Balancing:

Utilizes the DNS to direct traffic, often determining the best server even before a connection is initiated.

11. Transport Layer Protocol Load Balancing (TCP and UDP):

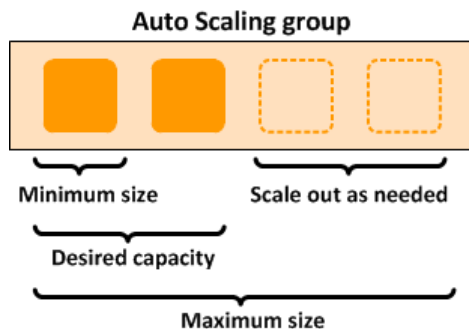
Balances traffic based on whether it's TCP (reliable, connection-oriented) or UDP (fast, connectionless).

12. Adaptive Load Balancing with AI:

Employs artificial intelligence to analyze real-time traffic and adaptively adjust routing based on traffic patterns and server health.

Auto Scaling Group

- Amazon EC2 Auto Scaling helps you ensure that you have the correct number of Amazon EC2 instances available to handle the load for your application.
- You create collections of EC2 instances, called Auto Scaling groups.
- You can specify the minimum number of instances in each Auto Scaling group, and Amazon EC2 Auto Scaling ensures that your group never goes below this size.
- You can specify the maximum number of instances in each Auto Scaling group, and Amazon EC2 Auto Scaling ensures that your group never goes above this size.
- If you specify the desired capacity, either when you create the group or at any time thereafter, Amazon EC2 Auto Scaling ensures that your group has this many instances.
- If you specify scaling policies, then Amazon EC2 Auto Scaling can launch or terminate instances as demand on your application increases or decreases.



Auto Scaling components

The following table describes the key components of Amazon EC2 Auto Scaling.

1. **Groups** - Your EC2 instances are organized into *groups* so that they can be treated as a logical unit for the purposes of scaling and management. When you create a group, you can specify its minimum, maximum, and, desired number of EC2 instances. [Read More](#).
2. **Configuration Templates** - Your group uses a *launch template*, or a *launch configuration* (not recommended, offers fewer features), as a configuration template for its EC2 instances. You can specify information such as the AMI ID, instance type, key pair, security groups, and block device mapping for your instances. see [Launch templates](#) and [Launch configurations](#)
3. **Scaling options** - Amazon EC2 Auto Scaling provides several ways for you to scale your Auto Scaling groups. For example, you can configure a group to scale based on the occurrence of specified conditions (dynamic scaling) or on a schedule. see [Choose your scaling method](#).

Pricing for Amazon EC2 Auto Scaling

- There are no additional fees with Amazon EC2 Auto Scaling, so it's easy to try it out and see how it can benefit your AWS architecture.
- You only pay for the AWS resources (for example, EC2 instances, EBS volumes, and CloudWatch alarms) that you use.

Creating Auto Scaling Group (with application load balancer):

To create an ASG, ensure to follow

1. Create a Target group (without any EC2 instances attached to it. Detach if any attached).
2. Create a load balancer (attached with target group)
3. Create a launch template.
4. Create an ASG using the same launch template (created before).

Target group:

EC2 > Target groups

Target groups (1/1) [Info](#)

Filter target groups

☒	Name	ARN	Port	Protocol	Target type	Load balancer	VPC ID
☒	my-target-group-1	arn:aws:elasticloadbalanci...	80	HTTP	Instance	my-lb-1	vpc-0c5df5c4375250e18

Target group: my-target-group-1

Details **Targets** Monitoring Health checks Attributes Tags

Registered targets (0) [Info](#) [Anomaly mitigation: Not applicable](#) [Deregister](#) [Register targets](#)

Target groups route requests to individual registered targets using the protocol and port number specified. Health checks are performed on all registered targets according to the target group's health check settings. Anomaly detection is automatically applied to HTTP/HTTPS target groups with at least 3 healthy targets.

Filter targets

☐	Instance ID	Name	Port	Zone	Health status	Health status details
No registered targets						

You have not registered targets to this group yet

[Register targets](#)

Load Balancer:

Load balancers (1/1)

Elastic Load Balancing scales your load balancer capacity automatically in response to changes in incoming traffic.

Filter load balancers

☒	Name	DNS name	State	VPC ID	Availability Zones	Type
☒	my-lb-1	my-lb-1-95310984.us-east-2.elb.amazonaws.com	Active	vpc-0c5df5c4375250e18	2 Availability Zones	application

Load balancer: my-lb-1

Details Listeners and rules Network mapping Security Monitoring Integrations Attributes Tags

Details

Load balancer type Application	Status Active	VPC vpc-0c5df5c4375250e18	IP address type IPv4
Scheme Internet-facing	Hosted zone Z3AADJGX6KTTL2	Availability Zones subnet-087bac02edb80ad96 us-east-2a (use2-az1) subnet-0a962769082dcd173 us-east-2b (use2-az2)	Date created December 1, 2023, 11:21 (UTC+05:30)
Load balancer ARN arn:aws:elasticloadbalancing:us-east-2:805116530761:loadbalancer/app/my-lb-1/c4c6bfa122176c09		DNS name Info my-lb-1-95310984.us-east-2.elb.amazonaws.com (A Record)	

Creating Launch Template:

1. Go to EC2 dashboard & select "Launch Template".
2. Fill launch template name & description:

Create launch template

Creating a launch template allows you to create a saved instance configuration that can be reused, shared and launched at a later time. Templates can have multiple versions.

Launch template name and description

Launch template name - *required*

my-launch-template

Must be unique to this account. Max 128 chars. No spaces or special characters like '&', '*', '@'.

Template version description

Ver1.0

Max 255 chars

3. Select the AMI for providing the OS:

▼ Application and OS Images (Amazon Machine Image) [Info](#)

An AMI is a template that contains the software configuration (operating system, application server, and applications) required to launch your instance. Search or Browse for AMIs if you don't see what you are looking for below

Q Search our full catalog including 1000s of application and OS images

Recents

Quick Start

Don't include in launch template

Amazon Linux

aws

macOS

Mac

Ubuntu

ubuntu

Windows

Microsoft

Red H

Red

Browse more AMIs

Including AMIs from AWS, Marketplace and the Community

Amazon Machine Image (AMI)

Amazon Linux 2023 AMI

Free tier eligible ▼

ami-06d4b7182ac3480fa (64-bit (x86)) / ami-0090be1905998682a (64-bit (Arm))
Virtualization: hvm ENA enabled: true Root device type: ebs

Description

Amazon Linux 2023 AMI 2023.2.20231113.0 x86_64 HVM kernel-6.1

Architecture

64-bit (x86)

AMI ID

ami-06d4b7182ac3480fa

Verified provider

4. Select instance type & key pair:

▼ Instance type
Info | Get advice
Advanced

Instance type

t2.micro
Free tier eligible

Family: t2 1 vCPU 1 GiB Memory Current generation: true
On-Demand Linux base pricing: 0.0116 USD per Hour
On-Demand SUSE base pricing: 0.0116 USD per Hour
On-Demand Windows base pricing: 0.0162 USD per Hour
On-Demand RHEL base pricing: 0.0716 USD per Hour

☐ All generations
[Compare instance types](#)

Additional costs apply for AMIs with pre-installed software

▼ Key pair (login)
Info

You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.

Key pair name

winkey

- Leave subnet entries & fill

▼ Network settings
Info

Subnet Info

Don't include in launch template

When you specify a subnet, a network interface is automatically added to your template.

Firewall (security groups)
Info

A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

☒ Select existing security group
☐ Create security group

Security groups Info

Select security groups

websvr-launch-wizard-1 sg-0e05ae55ee6978f5d ✕
VPC: vpc-0c5df5c4375250e18

- Set the user data to download files from S3 bucket, also ensure to attach EC2-to-S3-Full-access role to attach with this as IAM profile.

```
#!/bin/bash
sudo su -
sudo yum install -y httpd php && sudo systemctl start httpd && sudo systemctl
enable httpd
aws s3 cp s3://demobepac01/index.php /var/www/html/index.php
aws s3 cp s3://demobepac01/styles.css /var/www/html/styles.css
chmod u+x /var/www/html/index.php
systemctl restart httpd
```

Code to copy & paste

```
#!/bin/bash
sudo su -
sudo yum install -y httpd php && sudo systemctl start httpd && sudo systemctl enable httpd
aws s3 cp s3://demobeprac01/index.php /var/www/html/index.php
aws s3 cp s3://demobeprac01/styles.css /var/www/html/styles.css
chmod u+x /var/www/html/index.php
systemctl restart httpd
```

And

▼ Advanced details [Info](#)

IAM instance profile [Info](#)

EC2-to-S3-full-access

arn:aws:iam::805116530761:instance-profile/EC2-to-S3-full-access

[Create new IAM profile](#)

- Click on "Click launch template".

▼ Storage (volumes) [Info](#)

EBS Volumes [Hide details](#)

Volume 1 (AMI Root) (8 GiB, EBS, General purpose SSD (gp3))
AMI Volumes are not included in the template unless modified

[Add new volume](#)

▼ Resource tags [Info](#)

No resource tags are currently included in this template. Add a resource tag to include it in the launch template.

[Add new tag](#)

You can add up to 50 more tags.

Virtual server type (instance type)
t2.micro

Firewall (security group)
websvr-launch-wizard-1

Storage (volumes)
1 volume(s) - 8 GiB

[Cancel](#) [Create launch template](#)

Now creating Auto Scaling Group:

- Go to EC2 dashboard page and select "Auto Scaling Groups".

Amazon EC2 Auto Scaling

helps maintain the availability of your applications

Auto Scaling groups are collections of Amazon EC2 instances that enable automatic scaling and fleet management features. These features help you maintain the health and availability of your applications.

Create Auto Scaling group

Get started with EC2 Auto Scaling by creating an Auto Scaling group.

[Create Auto Scaling group](#)

How it works

Auto Scaling group

Pricing

Amazon EC2 Auto Scaling features have no additional fees beyond the service fees for Amazon EC2, CloudWatch (for scaling policies), and the other AWS resources that you use. Visit the pricing page of each service to learn more.

- Choose launch template:

EC2 > Auto Scaling groups > Create Auto Scaling group

Step 1
Choose launch template or configuration

Step 2
Choose instance launch options

Step 3 - optional
Configure advanced options

Step 4 - optional
Configure group size and scaling

Step 5 - optional
Add notifications

Step 6 - optional
Add tags

Step 7
Review

Choose launch template or configuration [info](#)

Specify a launch template that contains settings common to all EC2 instances that are launched by this Auto Scaling group. If you currently use launch configurations, you might consider migrating to launch templates.

Name

Auto Scaling group name
Enter a name to identify the group.

my-launch-template-1

Must be unique to this account in the current Region and no more than 255 characters.

Launch template [info](#) [Switch to launch configuration](#)

Launch template
Choose a launch template that contains the instance-level settings, such as the Amazon Machine Image (AMI), instance type, key pair, and security groups.

my-launch-template

[Create a launch template](#)

Version

Default (1)

3. Select required VPC & subnets (ensure to provide at least 2 subnets):

Step 2
Choose instance launch options

Step 3 - optional
[Configure advanced options](#)

Step 4 - optional
[Configure group size and scaling](#)

Step 5 - optional
[Add notifications](#)

Step 6 - optional
[Add tags](#)

Step 7
[Review](#)

Instance type requirements [info](#) [Override launch template](#)

You can keep the same instance attributes or instance type from your launch template, or you can choose to override the launch template by specifying different instance attributes or manually adding instance types.

Launch template	Version	Description
my-launch-template lt-02a4608a2625154a5	Default	Ver1.0

Instance type
t2.micro

Network [info](#)

For most applications, you can use multiple Availability Zones and let EC2 Auto Scaling balance your instances across the zones. The default VPC and default subnets are suitable for getting started quickly.

VPC
Choose the VPC that defines the virtual network for your Auto Scaling group.

vpc-0c5df5c4375250e18 (VPC-A)
192.168.0.0/16

[Create a VPC](#)

Availability Zones and subnets
Define which Availability Zones and subnets your Auto Scaling group can use in the chosen VPC.

Select Availability Zones and subnets

us-east-2a | subnet-087bac02edb80ad96 (vpc-a-subnet-1)
192.168.1.0/24

us-east-2b | subnet-0a962769082dcd173 (vpc-a-subnet-2)
192.168.2.0/24

[Create a subnet](#)

4. Select an existing load balancer (as created in previous step):

Step 1
[Choose launch template or configuration](#)

Step 2
[Choose instance launch options](#)

Step 3 - optional
Configure advanced options

Step 4 - optional
[Configure group size and scaling](#)

Step 5 - optional
[Add notifications](#)

Step 6 - optional
[Add tags](#)

Step 7
[Review](#)

Configure advanced options - optional [info](#)

Integrate your Auto Scaling group with other services to distribute network traffic across multiple servers using a load balancer or to establish service-to-service communications using VPC Lattice. You can also set options that give you more control over health check replacements and monitoring.

Load balancing [info](#)

Use the options below to attach your Auto Scaling group to an existing load balancer, or to a new load balancer that you define.

☐ No load balancer
Traffic to your Auto Scaling group will not be fronted by a load balancer.

☒ Attach to an existing load balancer
Choose from your existing load balancers.

☐ Attach to a new load balancer
Quickly create a basic load balancer to attach to your Auto Scaling group.

Attach to an existing load balancer
Select the load balancers that you want to attach to your Auto Scaling group.

☒ Choose from your load balancer target groups
This option allows you to attach Application, Network, or Gateway Load Balancers.

☐ Choose from Classic Load Balancers

Step 7
[Review](#)

☒ Choose from your load balancer target groups
This option allows you to attach Application, Network, or Gateway Load Balancers.

☐ Choose from Classic Load Balancers

Existing load balancer target groups
Only instance target groups that belong to the same VPC as your Auto Scaling group are available for selection.

Select target groups

my-target-group-1 | HTTP
(Application Load Balancer: myalb-1)

5. Just click on "NEXT".

Additional settings

Monitoring [Info](#)

☐ Enable group metrics collection within CloudWatch

Default instance warmup [Info](#)

The amount of time that CloudWatch metrics for new instances do not contribute to the group's aggregated instance metrics, as their usage data is not reliable yet.

☐ Enable default instance warmup

Cancel Skip to review Previous **Next**

6. Configuring the group size & scaling:

EC2 > Auto Scaling groups > Create Auto Scaling group

Step 1
[Choose launch template or configuration](#)

Step 2
[Choose instance launch options](#)

Step 3 - optional
[Configure advanced options](#)

Step 4 - optional
Configure group size and scaling

Step 5 - optional
[Add notifications](#)

Step 6 - optional
[Add tags](#)

Step 7
[Review](#)

Configure group size and scaling - optional [Info](#)

Define your group's desired capacity and scaling limits. You can optionally add automatic scaling to adjust the size of your group.

Group size [Info](#)

Set the initial size of the Auto Scaling group. After creating the group, you can change its size to meet demand, either manually or by using automatic scaling.

Desired capacity type
Choose the unit of measurement for the desired capacity value. vCPUs and Memory(GiB) are only supported for mixed instances groups configured with a set of instance attributes.

Units (number of instances)

Desired capacity
Specify your group size.

2

Scaling [Info](#)

You can resize your Auto Scaling group manually or automatically to meet changes in demand.

Scaling limits
Set limits on how much your desired capacity can be increased or decreased.

Min desired capacity
2
Equal or less than desired capacity

Max desired capacity
4
Equal or greater than desired capacity

7. Select the default:

Automatic scaling - optional

Choose whether to use a target tracking policy [Info](#)

You can set up other metric-based scaling policies and scheduled scaling after creating your Auto Scaling group.

☒ No scaling policies

Your Auto Scaling group will remain at its initial size and will not dynamically resize to meet demand.

☐ Target tracking scaling policy

Choose a CloudWatch metric and target value and let the scaling policy adjust the desired capacity in proportion to the metric's value.

Instance maintenance policy - new [Info](#)

Control your Auto Scaling group's availability during instance replacement events. This includes health checks, instance refreshes, maximum instance lifetime features and events that happen automatically to keep your group balanced, called rebalancing events.

Choose a replacement behavior depending on your availability requirements

Mixed behavior

☒ No policy

For rebalancing events, new instances will launch before terminating others. For all other events, instances terminate and launch at the same time.

Prioritize availability

☐ Launch before terminating

Launch new instances and wait for them to be ready before terminating others. This allows you to go above your desired capacity by a given percentage and may temporarily increase costs.

Control costs

☐ Terminate and launch

Terminate and launch instances at the same time. This allows you to go below your desired capacity by a given percentage and may temporarily reduce availability.

Flexible

☐ Custom behavior

Set custom values for the minimum and maximum amount of available capacity. This gives you greater flexibility in setting how far below and over your desired capacity EC2 Auto Scaling goes when replacing instances.

Instance scale-in protection

Scale-in protection prevents newly launched instances from being terminated by scaling activities. Make sure to remove scale-in protection for the group or individual instances when instances are ready to be terminated.

☐ Enable instance scale-in protection

8. & select NEXT.
9. Do not add notification.
10. Add tags as needed.
11. Review & Create auto scaling group.
12. Wait until the status changes:

Auto Scaling groups (1) Info									
Search your Auto Scaling groups									
☐	Name	Launch template/configuration	Instances	Status	Desired capacity	Min	Max	Availability Zones	
☐	my-launch-template-1	my-launch-template Version Default	0	Updating capacity	2	2	4	us-east-2a, us-east-2b	

13. Now copy the load balancer's URL & paste the URL in the web browser.

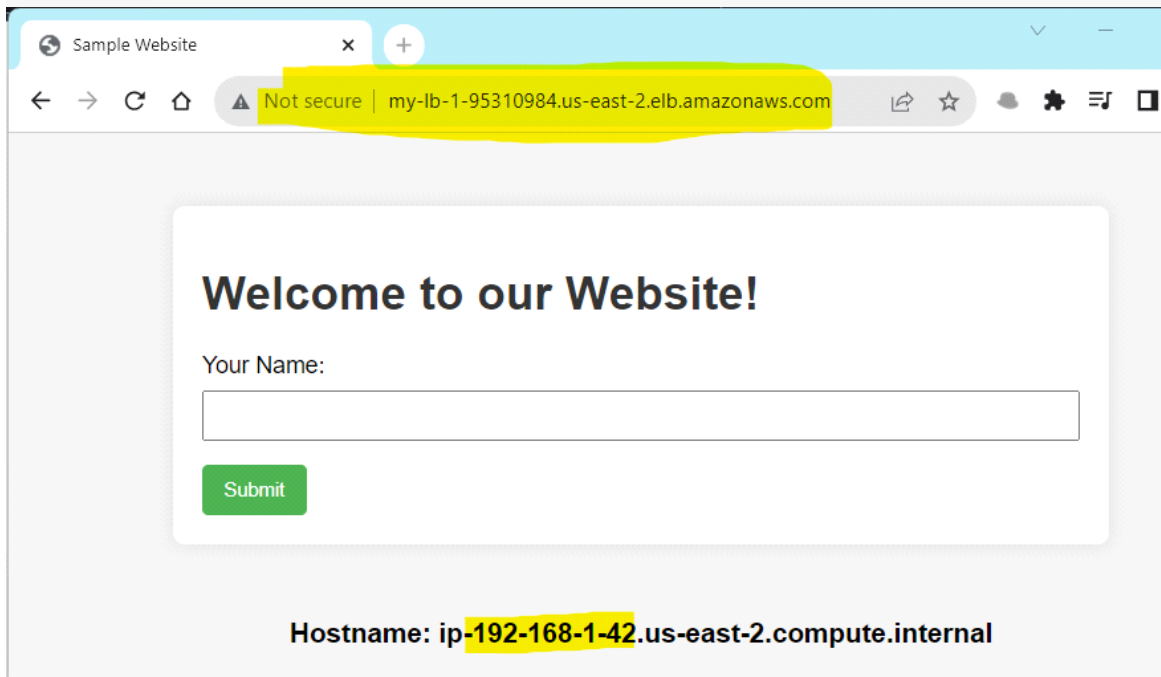
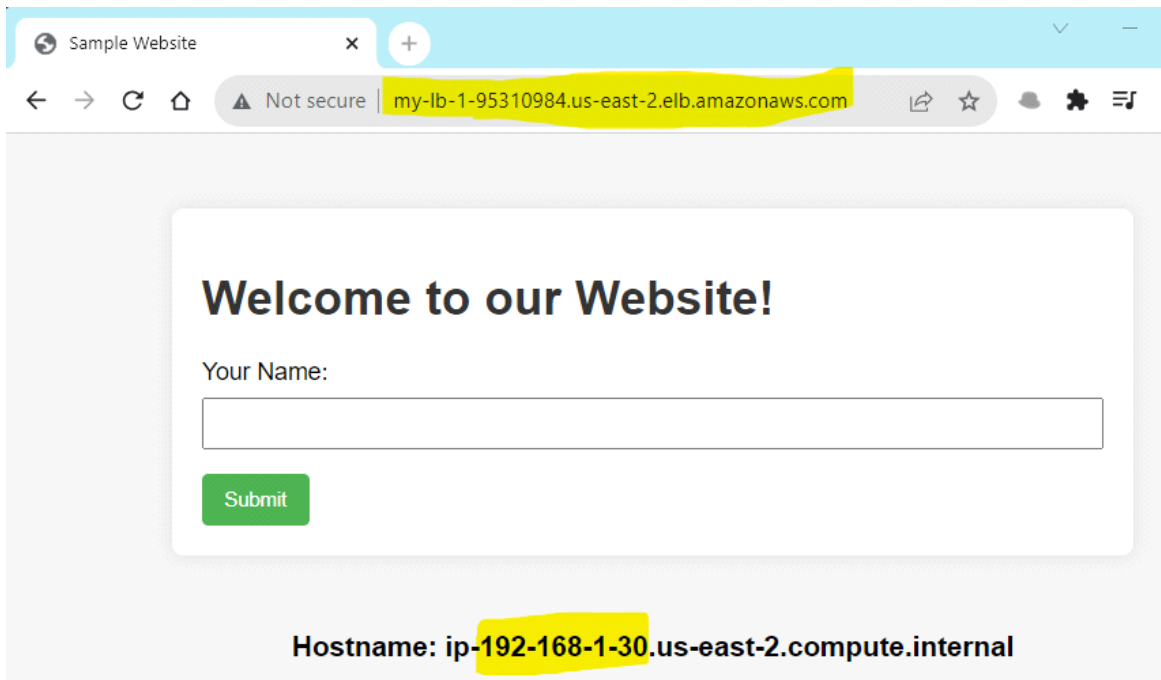
Load balancers (1)									
Elastic Load Balancing scales your load balancer capacity automatically in response to changes in incoming traffic.									
Filter load balancers									
☐	Name	DNS name	State	VPC ID	Availability Zones	Type	Date created		
☐	my-lb-1	my-lb-1-95310984.us-east-2.elb.amazonaws.com	Active	vpc-0c5df5c4375250e18	2 Availability Zones	application	December 1, 2023, 11:...		

Sample Website

Not secure | [my-lb-1-95310984.us-east-2.elb.amazonaws.com](#)

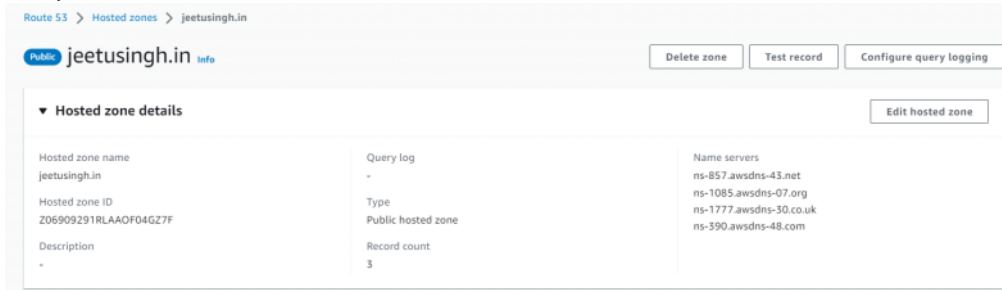
Welcome to our Website!

Your Name:



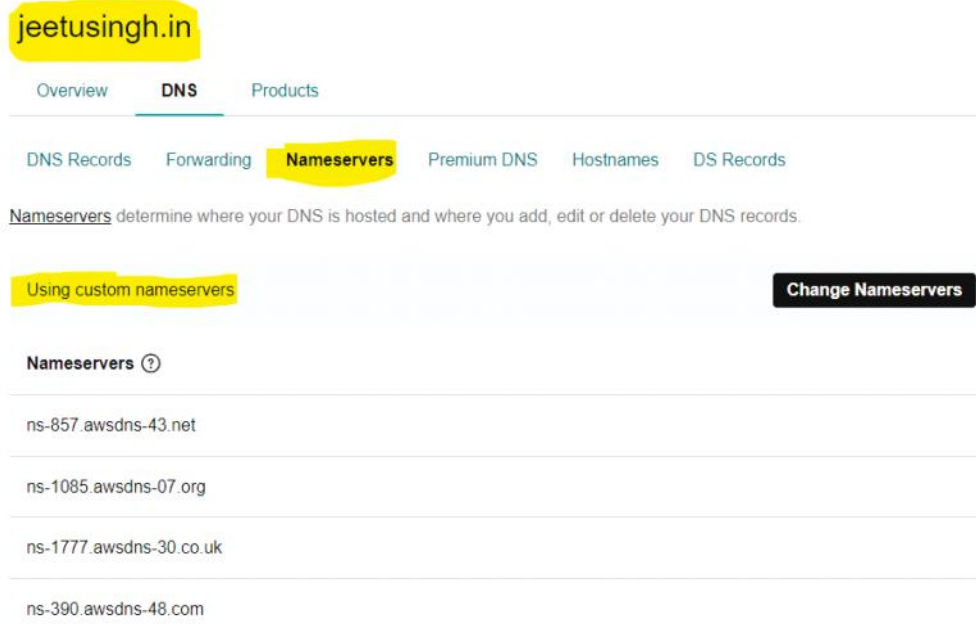
AWS Route 53 - Simple Routing

1. Purchase a domain from any registrar like:
 - a. Go Daddy
 - b. HostGator
 - c. Google Domain
 - d. Etc...
2. Switch to AWS Route 53 and create a new hosted zone (with the same domain name).



Once hosted zone is ready, copy the nameserver(s).

3. Once domain is ready, create a custom nameserver with the entries on step 2:



4. Create a new EC2 instance with the following:
 - a. Ubuntu server
 - b. Public IP Enabled (in this case 3.144.134.10)
 - c. Default VPC (in this case)
 - d. User data
 - e. SG enabled with 80, 22, 443.

Instances (1/1) [Info](#)

Find Instance by attribute or tag (case-sensitive)

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 DNS
websvr01	i-0c8943cf555d5e834	Running	t2.micro	2/2 checks passed	No alarms	us-east-2c	ec2-3-144-134-10.us-e...

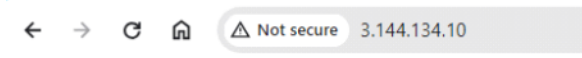
Instance: i-0c8943cf555d5e834 (websvr01)

Details | Security | Networking | Storage | Status checks | Monitoring | Tags

Instance summary [Info](#)

Instance ID i-0c8943cf555d5e834 (websvr01)	Public IPv4 address 3.144.134.10 open address	Private IPv4 addresses 172.31.32.158
IPv6 address -	Instance state Running	Public IPv4 DNS ec2-3-144-134-10.us-east-2.compute.amazonaws.com open address
Hostname type IP name: ip-172-31-32-158.us-east-2.compute.internal	Private IP DNS name (IPv4 only) ip-172-31-32-158.us-east-2.compute.internal	

5. Access this IP on web browser.



Server Details

Hostname: ip-172-31-32-158

IP Address: 172.31.32.158

6. Copy this Public IP & create a new "A" record.

Create record [Info](#)

Quick create record [Switch to wizard](#)

Record 1 [Delete](#)

Record name [Info](#): subdomain jeetusingh.in

Record type [Info](#): A - Routes traffic to an IPv4 address and some AWS resources

Value [Info](#): 3.144.134.10

TTL (seconds) [Info](#): 10

Routing policy [Info](#): Simple routing

[Add another record](#)

[Cancel](#) [Create records](#)

Records (3) | DNSSEC signing | Hosted zone tags (0)

Records (3) [Info](#)

Automatic mode is the current search behavior optimized for best filter results. [To change modes go to settings.](#)

Filter records by property or value

Record name	Type	Routin...	Differ...	Alias	Value/Route traffic to	TTL (s...
jeetusingh.in	A	Simple	-	No	3.144.134.10	10

7. Once record is created, wait for some time for the propagation. Then access the domain URL.



This may take more than 15mins. Try "<https://developers.google.com/speed/public-dns/cache>" to clear the cache memory (but not reliable).

Binding the domain name with the load balancer's URL.

8. Create a target group.

The screenshot displays the AWS Management Console 'Target groups' page. A table lists one target group: 'websvr-lb-01' with ARN 'arn:aws:elasticloadbalancing:us-east-2:805116530761:targetgroup/websvr-lb-01/950ef6f2cb29dedc', Port 80, Protocol HTTP, Target type Instance, Load balancer web-svr-lb-01, and VPC ID vpc-0146ba4a2e5e96f08. Below the table, the 'Details' tab for 'websvr-lb-01' is open, showing fields like Target type (Instance), Protocol : Port (HTTP: 80), Protocol version (HTTP1), VPC (vpc-0146ba4a2e5e96f08), IP address type (IPv4), Load balancer (web-svr-lb-01), and Target access analysis (Access issue detected).

9. Create an application load balancer.

The screenshot shows the AWS Management Console 'Load balancers' page. A table lists one load balancer: 'web-svr-lb-01' with DNS name 'web-svr-lb-01-467764825.us-east-2.elb.amazonaws.com', State Active, VPC ID vpc-0146ba4a2e5e96f08, Availability Zones 3, and Type application. Below the table, the 'Details' tab for 'web-svr-lb-01' is open, showing fields like Load balancer type (Application), Status (Active), VPC (vpc-0146ba4a2e5e96f08), IP address type (IPv4), Scheme (Internet-facing), Hosted zone (Z3AADJGX6KTTL2), Availability Zones (subnet-0e0d7bc19b0a390b1 us-east-2a (use2-az1)), and Date created (December 15, 2023, 12:08 (UTC+05:30)).

10. Delete the A record entry from the Route 53 console, as we will be using the LB's URL to access the server.

Public jeetusingh.in Info Delete zone Test record Configure query logging

► Hosted zone details Edit hosted zone

Records (3) DNSSEC signing Hosted zone tags (0)

Records (3) Info Refresh Delete record Import zone file Create record
Automatic mode is the current search behavior optimized for best filter results. To change modes go to settings.

Q Filter records by property or value Type ▼ Routing policy ▼ Alias ▼ < 1 > ⚙

☐	Record name ▼	Type ▼	Routin... ▼	Differ... ▼	Alias ▼	Value/Route traffic to ▼	TTL (s... ▼	Health ... ▼	Evalua... ▼	R.. ▼
☐	jeetusingh.in	A	Simple	-	No	3.149.236.119	10	-	-	-
☐	jeetusingh.in	NS	Simple	-	No	ns-1906.awsdns-46.co.uk. ns-795.awsdns-35.net. ns-1123.awsdns-12.org. ns-33.awsdns-04.com.	172800	-	-	-
☐	jeetusingh.in	SOA	Simple	-	No	ns-1906.awsdns-46.co.uk. a...	900	-	-	-

After deletion.

Public jeetusingh.in Info Delete zone Test record Configure query logging

► Hosted zone details Edit hosted zone

Records (2) DNSSEC signing Hosted zone tags (0)

Records (2) Info Refresh Delete record Import zone file Create record
Automatic mode is the current search behavior optimized for best filter results. To change modes go to settings.

Q Filter records by property or value Type ▼ Routing policy ▼ Alias ▼ < 1 > ⚙

☐	Record name ▼	Type ▼	Routin... ▼	Differ... ▼	Alias ▼	Value/Route traffic to ▼	TTL (s... ▼	Health ... ▼	Evalua... ▼	R.. ▼
☐	jeetusingh.in	NS	Simple	-	No	ns-1906.awsdns-46.co.uk. ns-795.awsdns-35.net. ns-1123.awsdns-12.org. ns-33.awsdns-04.com.	172800	-	-	-
☐	jeetusingh.in	SOA	Simple	-	No	ns-1906.awsdns-46.co.uk. a...	900	-	-	-

11. Create a new record with the "Switch to wizard" view:

Create record Info

Quick create record Switch to wizard Delete

▼ Record 1

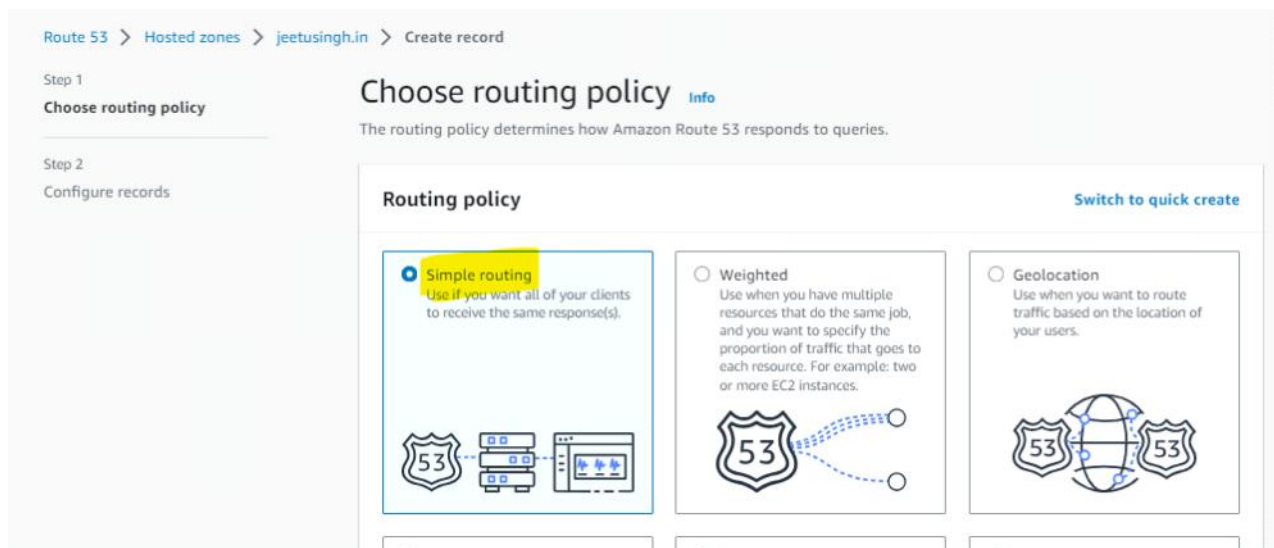
Record name Info jeetusingh.in Record type Info

Keep blank to create a record for the root domain.

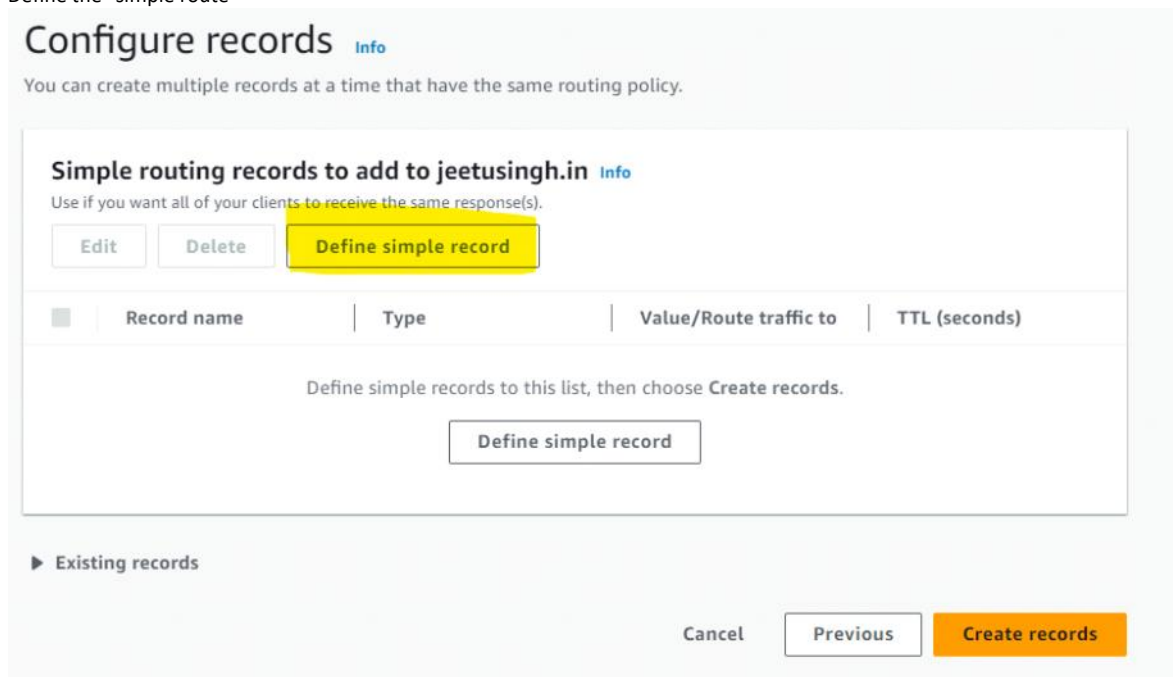
☐ Alias

Value Info

Once switched to new console,

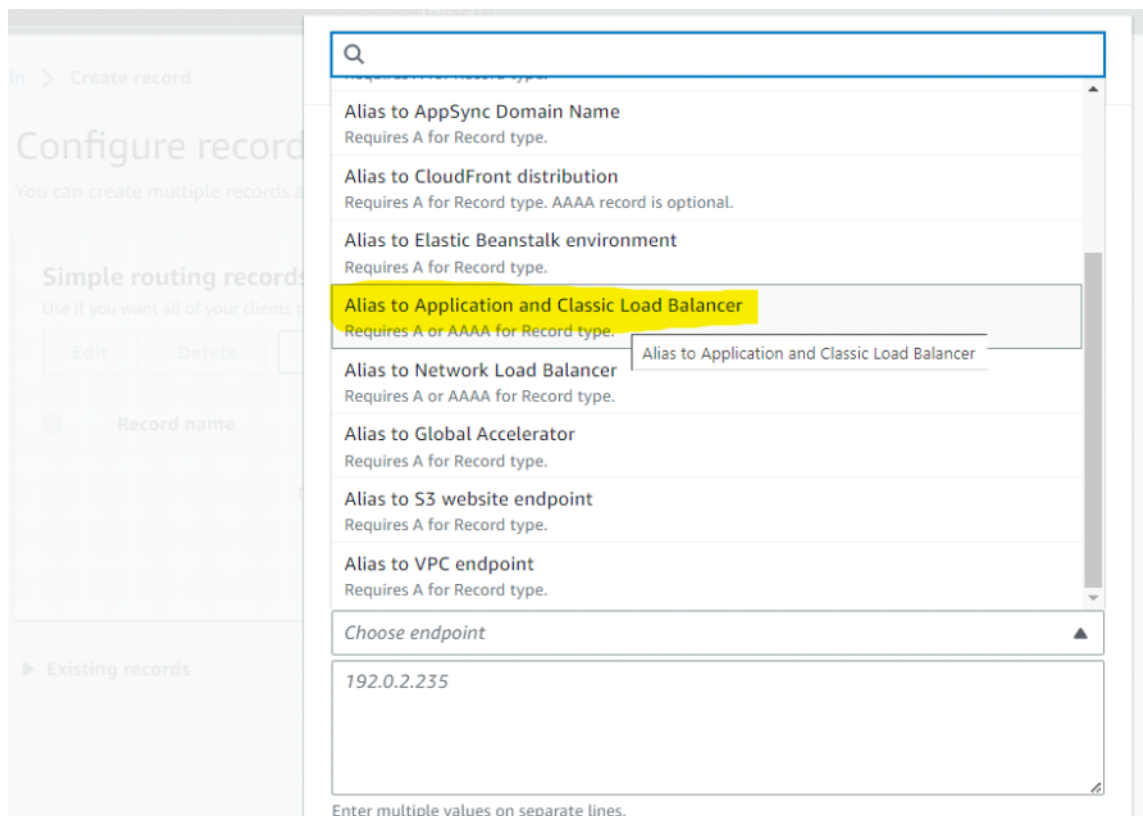


Define the "simple route"



In the record type, select "A" record.

In value/route traffic to, select endpoint: "Alias to application and classic load balancer".



Select the region, in which load balancer is created.

Value/Route traffic to [Info](#)

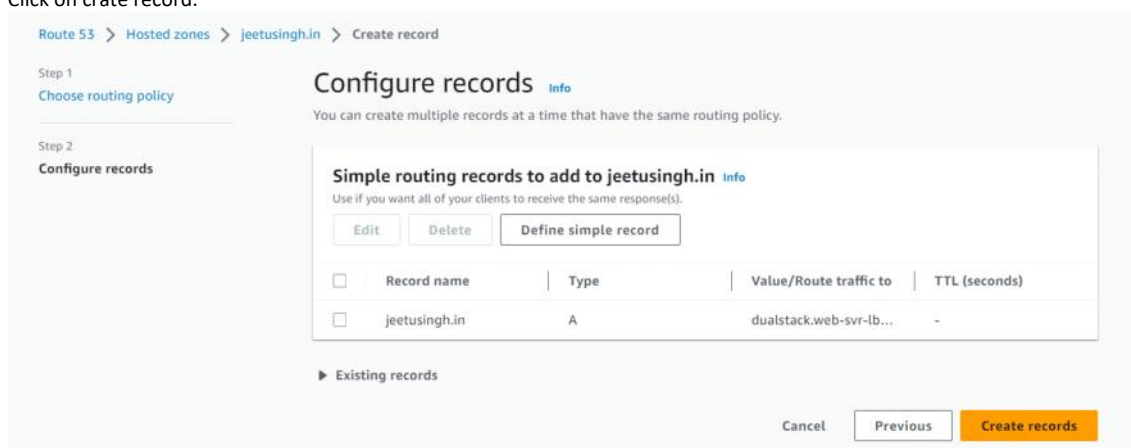
The option that you choose determines how Route 53 responds to DNS queries. For most options, you specify where you want to route internet traffic.

Alias to Application and Classic Load Balancer ▼

US East (Ohio) ▼

Alias hosted zone ID: Z3AADJGX6KTTL2

Click on crate record.



Verify the status:

Route 53 > Hosted zones > jeetusingh.in > Change Info

C0107421WTCR75KWPV29 [Info](#)

Change info details

ID
/change/C0107421WTCR75KWPV29

Submitted at
December 15, 2023, 13:21 (UTC+05:30)

Status
⏸ PENDING

Comment
-

Route 53 > Hosted zones > jeetusingh.in > Change Info

C0107421WTCR75KWPV29 [Info](#)

Change info details

ID
/change/C0107421WTCR75KWPV29

Status
✅ **INSYNC**

Accessing the URL:

jeetusingh.in

Not secure jeetusingh.in

Learnings works-stuff xtrass PBS blocking dates Udemy people+

Server Details

Hostname: ip-172-31-12-28

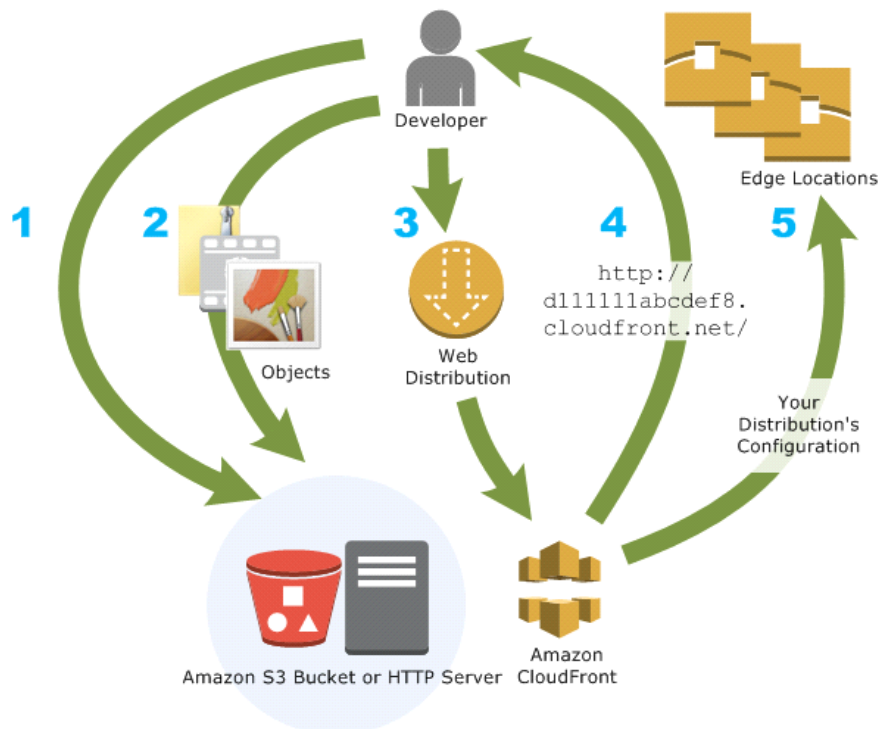
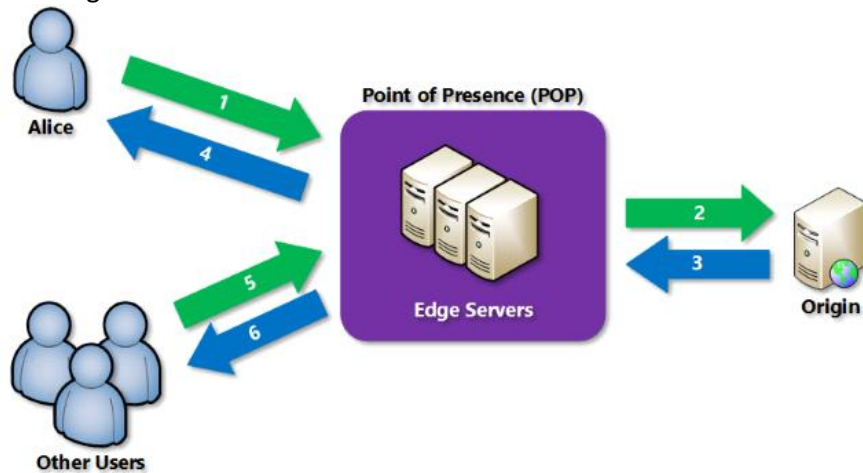
IP Address: 172.31.12.28

You can now create another EC2 instance, attach it to target group and refresh the browser with the domain name.

AWS CloudFront

- Starting with content delivery network (CDN).
- It provides the delivery of content like:
 - o Static content
 - o Dynamic content
 - o Web content like: html, css, JS, etc.
 - o Images
 - o Media files like video/audio.

Working on CDN:



Benefits:

- Reduced latency
- 600+ global POPs.
- 1TB of data transfer out free.
- Can be integrated with
 - o AWS shield

- AWS WAF

Use case

- Delivered fast, secure websites
- Stream live & on-demand video
- Distributed patches & updates.

Connecting S3 bucket with CloudFront:

1. Create S3 bucket

Create bucket Info

Buckets are containers for data stored in S3. [Learn more](#)

General configuration

AWS Region

US East (Ohio) us-east-2

Bucket name Info

mydemobuck01

Bucket name must be unique within the global namespace and follow the bucket naming rules. [See rules for bucket naming](#)

Copy settings from existing bucket - *optional*

Only the bucket settings in the following configuration are copied.

Choose bucket

Format: s3://bucket/prefix

Object Ownership Info

Control ownership of objects written to this bucket from other AWS accounts and the use of access control lists (ACLs). Object ownership determines who can specify access to objects.

☐ ACLs disabled (recommended)

All objects in this bucket are owned by this account. Access to this bucket and its objects is specified using only policies.

☒ ACLs enabled

Objects in this bucket can be owned by other AWS accounts. Access to this bucket and its objects can be specified using ACLs.

⚠ We recommend disabling ACLs, unless you need to control access for each object individually or to have the object writer own the data they upload. Using a bucket policy instead of ACLs to share data with users outside of your account simplifies permissions management and auditing.

Object Ownership

☒ Bucket owner preferred

If new objects written to this bucket specify the bucket-owner-full-control canned ACL, they are owned by the bucket owner. Otherwise, they are owned by the object writer.

☐ Object writer

The object writer remains the object owner.

 If you want to enforce object ownership for new objects only, your bucket policy must specify that the bucket-owner-full-control canned ACL is required for object uploads. [Learn more](#) 

Block Public Access settings for this bucket

Public access is granted to buckets and objects through access control lists (ACLs), bucket policies, access point policies, or all. In order to ensure that public access to this bucket and its objects is blocked, turn on Block all public access. These settings apply only to this bucket and its access points. AWS recommends that you turn on Block all public access, but before applying any of these settings, ensure that your applications will work correctly without public access. If you require some level of public access to this bucket or objects within, you can customize the individual settings below to suit your specific storage use cases. [Learn more](#) 

☐ Block *all* public access

Turning this setting on is the same as turning on all four settings below. Each of the following settings are independent of one another.

☐ Block public access to buckets and objects granted through *new* access control lists (ACLs)

S3 will block public access permissions applied to newly added buckets or objects, and prevent the creation of new public access ACLs for existing buckets and objects. This setting doesn't change any existing permissions that allow public access to S3 resources using ACLs.

☐ Block public access to buckets and objects granted through *any* access control lists (ACLs)

S3 will ignore all ACLs that grant public access to buckets and objects.

☐ Block public access to buckets and objects granted through *new* public bucket or access point policies

S3 will block new bucket and access point policies that grant public access to buckets and objects. This setting doesn't change any existing policies that allow public access to S3 resources.

☐ Block public and cross-account access to buckets and objects through *any* public bucket or access point policies

S3 will ignore public and cross-account access for buckets or access points with policies that grant public access to buckets and objects.



Turning off block all public access might result in this bucket and the objects within becoming public

Tags - optional (0)

You can use bucket tags to track storage costs and organize buckets. [Learn more](#)

No tags associated with this bucket.

Add tag

Default encryption [Info](#)

Server-side encryption is automatically applied to new objects stored in this bucket.

Encryption type [Info](#)

- ☒ Server-side encryption with Amazon S3 managed keys (SSE-S3)
- ☐ Server-side encryption with AWS Key Management Service keys (SSE-KMS)
- ☐ Dual-layer server-side encryption with AWS Key Management Service keys (DSSE-KMS)
Secure your objects with two separate layers of encryption. For details on pricing, see [DSSE-KMS pricing](#) on the **Storage** tab of the [Amazon S3 pricing page](#).

Bucket Key

Using an S3 Bucket Key for SSE-KMS reduces encryption costs by lowering calls to AWS KMS. S3 Bucket Keys aren't supported for DSSE-KMS. [Learn more](#)

- ☐ Disable
- ☒ Enable

► Advanced settings

After creating the bucket, you can upload files and folders to the bucket, and configure additional bucket settings.

Cancel

Create bucket

2. Giving the public access to the S3 bucket.

mydemobuck01 [Info](#)

[Objects](#) | [Properties](#) | [Permissions](#) | [Metrics](#) | [Management](#) | [Access Points](#)

Objects (0) [Info](#)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to s

Copy S3 URI

Copy URL

Download

Open

Delete

Actions

☐

Name

Type

Last modified

Creating bucket policy with public read access.

Bucket policy

The bucket policy, written in JSON, provides access to the objects stored in the bucket. Bucket policies don't apply to objects owned by other accounts. [Learn more](#)

```
{
  "Version": "2008-10-17",
  "Statement": [
    {
      "Sid": "AllowPublicRead",
      "Effect": "Allow",
      "Principal": {
        "AWS": "*"
      },
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::mydemobuck01/*"
    }
  ]
}
```

Public access code:

```
{
  "Version": "2008-10-17",
  "Statement": [
    {
      "Sid": "AllowPublicRead",
      "Effect": "Allow",
      "Principal": {
        "AWS": "*"
      },
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::mydemobuck01/*"
    }
  ]
}
```

3. Adding objects to the S3.

mydemobuck01 [Info](#)

[Objects](#) | [Properties](#) | [Permissions](#) | [Metrics](#) | [Management](#) | [Access Points](#)

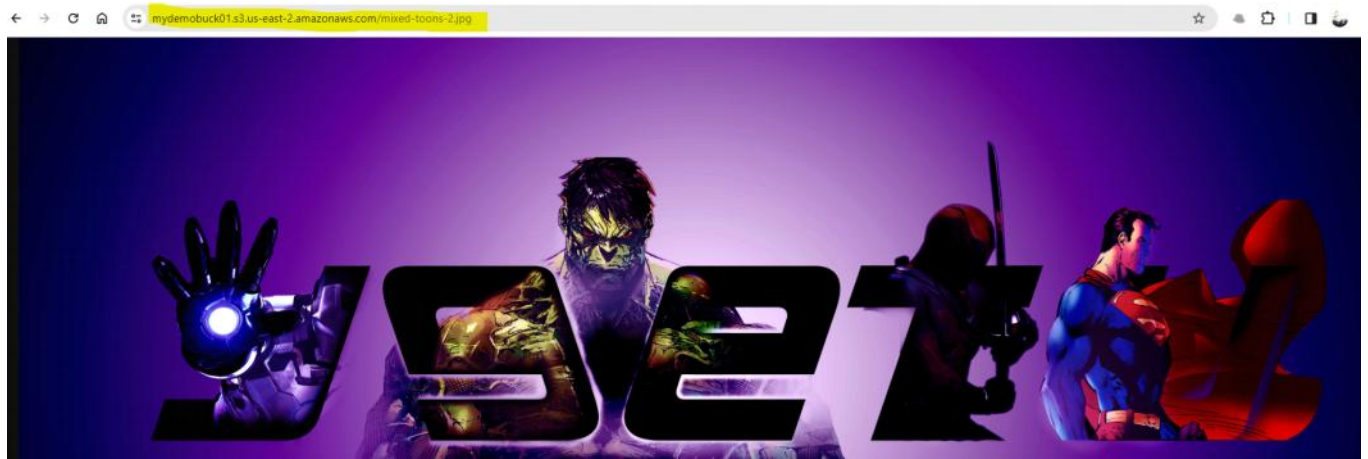
Objects (2) [Info](#)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them

[Refresh](#) [Copy S3 URI](#) [Copy URL](#) [Download](#) [Open](#) [Delete](#) [Actions](#) [Create folder](#) [Upload](#)

☐	Name	Type	Last modified	Size
☐	 mixed-toons-2.jpg	jpg	December 16, 2023, 13:13:55 (UTC+05:30)	
☐	 pexels-pratik-gupta-2748716.jpg	jpg	December 16, 2023, 13:14:07 (UTC+05:30)	

Accessing the object URL publicly:



4. Creating the cloud front:
AWS Console --> CloudFront

[CloudFront](#) > [Distributions](#) > Create

Create distribution

Origin

Origin domain
Choose an AWS origin, or enter your origin's domain name.

Origin path - optional [Info](#)
Enter a URL path to append to the origin domain name for origin requests.

Name
Enter a name for this origin.

Origin access [Info](#)

☒ **Public**
Bucket must allow public access.

☐ **Origin access control settings (recommended)**
Bucket can restrict access to only CloudFront.

☐ **Legacy access identities**
Use a CloudFront origin access identity (OAI) to access the S3 bucket.

Add custom header - optional
CloudFront includes this header in all requests that it sends to your origin.

Default cache behavior

Path pattern

Info

Default (*)

Compress objects automatically

Info

No

Yes

Viewer

Viewer protocol policy

HTTP and HTTPS

Redirect HTTP to HTTPS

HTTPS only

Allowed HTTP methods

GET, HEAD

GET, HEAD, OPTIONS

GET, HEAD, OPTIONS, PUT, POST, PATCH, DELETE

Restrict viewer access

If you restrict viewer access, viewers must use CloudFront signed URLs or signed cookies to access your content.

No

Creating a new policy (if required):

demo-policy-1

EditDelete

Details

Description

ID

Last modified

-

f503cc03-9fdd-48bb-8232-9009ad9cb90f

December 16, 2023 at 6:39:31 AM UTC

TTL settings

info

Minimum TTL (seconds)

Maximum TTL (seconds)

Default TTL (seconds)

1

604800

86400

Cache key settings

info

Headers - None

Cookies - None

Query strings - None

Compression support

info

Gzip

Brotli

Enabled

Enabled

SAA-C003 Page 62

☒ Cache policy and origin request policy (recommended)

☐ Legacy cache settings

Cache policy

Choose an existing cache policy or create a new one.

demo-policy-1



[Create cache policy](#)

[View policy](#)

Origin request policy - optional

Choose an existing origin request policy or create a new one.

Select origin policy



[Create origin request policy](#)

Response headers policy - optional

Choose an existing response headers policy or create a new one.

Select response headers



[Create response headers policy](#)

► Additional settings

Function associations - optional [Info](#)

Choose an edge function to associate with this cache behavior, and the CloudFront event that invokes the function.

	Function type	Function ARN / Name	Include body
Viewer request	No association		

Web Application Firewall (WAF) [Info](#)

☐ Enable security protections

Keep your application secure from the most common web threats and security vulnerabilities using AWS WAF. Blocked requests are stopped before they reach your web servers.

☒ Do not enable security protections

Select this option if your application does not need security protections from AWS WAF.

Settings

Price class [Info](#)

Choose the price class associated with the maximum price that you want to pay.

- ☒ Use all edge locations (best performance)
- ☐ Use only North America and Europe
- ☐ Use North America, Europe, Asia, Middle East, and Africa

Alternate domain name (CNAME) - optional

Add the custom domain names that you use in URLs for the files served by this distribution.

[Add item](#)

 To add a list of alternative domain names, use the [bulk editor](#).

Custom SSL certificate - optional

Associate a certificate from AWS Certificate Manager. The certificate must be in the US East (N. Virginia) Region (us-east-1).

[Choose certificate](#)



[Request certificate](#) 

Supported HTTP versions

Add support for additional HTTP versions. HTTP/1.0 and HTTP/1.1 are supported by default.

[Add support for additional HTTP versions](#)

Standard logging

Get logs of viewer requests delivered to an Amazon S3 bucket.

- ☒ Off
- ☐ On

IPv6

- ☐ Off
- ☒ On

Description - optional

[Cancel](#)

[Create distribution](#)

Wait until the last modified changes to date & time:

CloudFront > Distributions > E2PYAEGWADPA9A

E2PYAEGWADPA9A

General Security Origins Behaviors Error pages Invalidations Tags

Details

Distribution domain name dlsi1uptpea40.cloudfront.net	ARN arn:aws:cloudfront::805116530761:distribution/E2PYAEGWADPA9A	Last modified Deploying
--	---	----------------------------

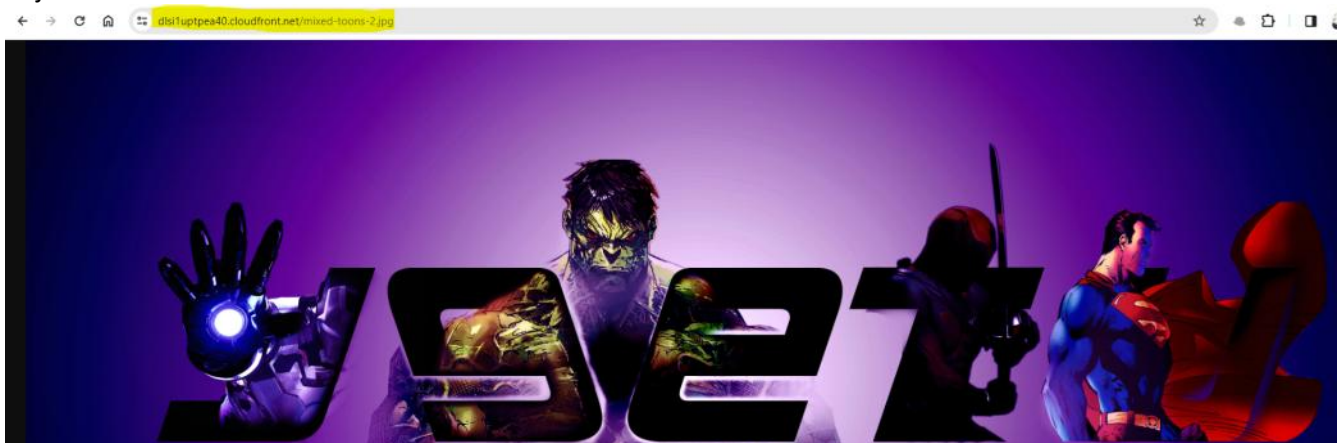
E2PYAEGWADPA9A

General Security Origins Behaviors Error pages Invalidations Tags

Details

Distribution domain name dlsi1uptpea40.cloudfront.net	ARN arn:aws:cloudfront::805116530761:distribution/E2PYAEGWADPA9A	Last modified December 16, 2023 at 7:49:39 AM UTC
--	---	--

Now copy the "Distribution domain name" & add object name after it to access the object over the browser:



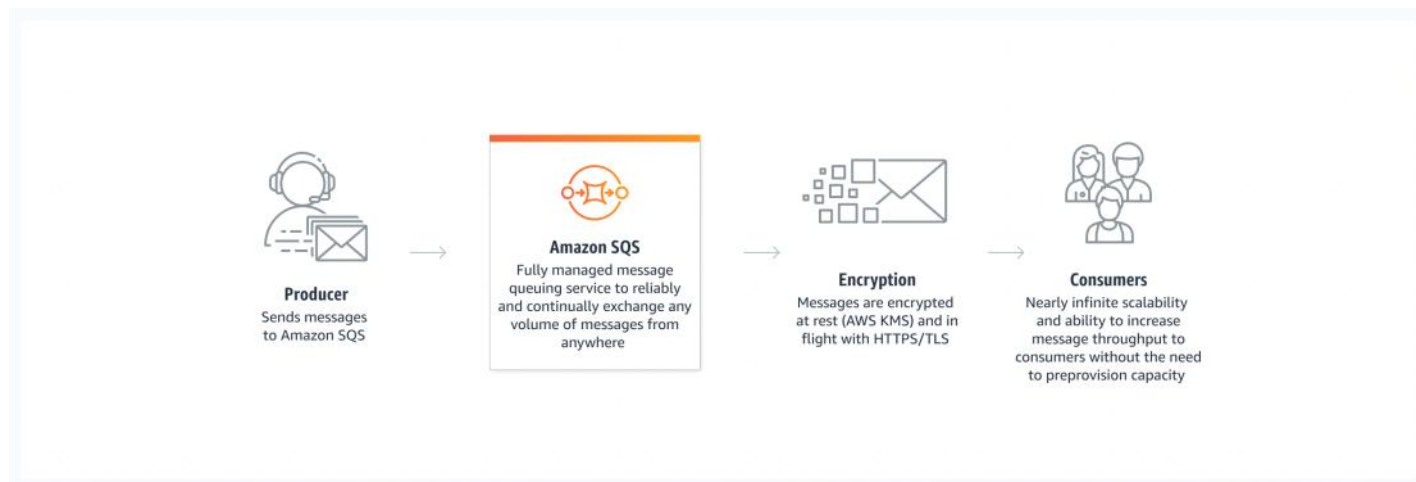
This Distribution domain name could be later bind with the Route53 for custom name.

Amazon Simple Queue Service (SQS)

- What is Amazon SQS?
 - A fully managed message queuing service.
 - Decouples and scales microservices, distributed systems, and serverless applications.
 - Enables components to scale independently.
 - What does it do?
 - Sends, stores, and receives messages between software components.
 - Ensures messages are delivered and not lost.
 - Types of queues:
 - Standard queues: at least once delivery.
 - FIFO queues: exactly once processing.
 - Benefits:
 - Ensures message safety and delivery.
 - Keeps sensitive data secure.
 - Scales elastically.
 - Cost-effective.
-
- Fully managed message queuing for microservices, distributed systems, and serverless applications
 - Amazon SQS lets you send, store, and receive messages between software components at any volume, without losing messages or requiring other services to be available.

SQS workflow:

Producer == sender
Consumer == receiver



Use case:

- **Increase application reliability and scale**
 - Amazon SQS provides a simple and reliable way for customers to decouple and connect components (microservices) together using queues.
- **Decouple microservices and process event-driven applications**
 - Separate frontend from backend systems, such as in a banking application. Customers immediately get a response, but the bill payments are processed in the background
- **Ensure work is completed cost-effectively and on time**
 - Place work in a single queue where multiple workers in an autoscale group scale up and down based on workload and latency requirements.
- **Maintain message ordering with deduplication**

- Process messages at high scale while maintaining the message order, allowing you to deduplicate messages.

Distributed queues

There are three main parts in a distributed messaging system:

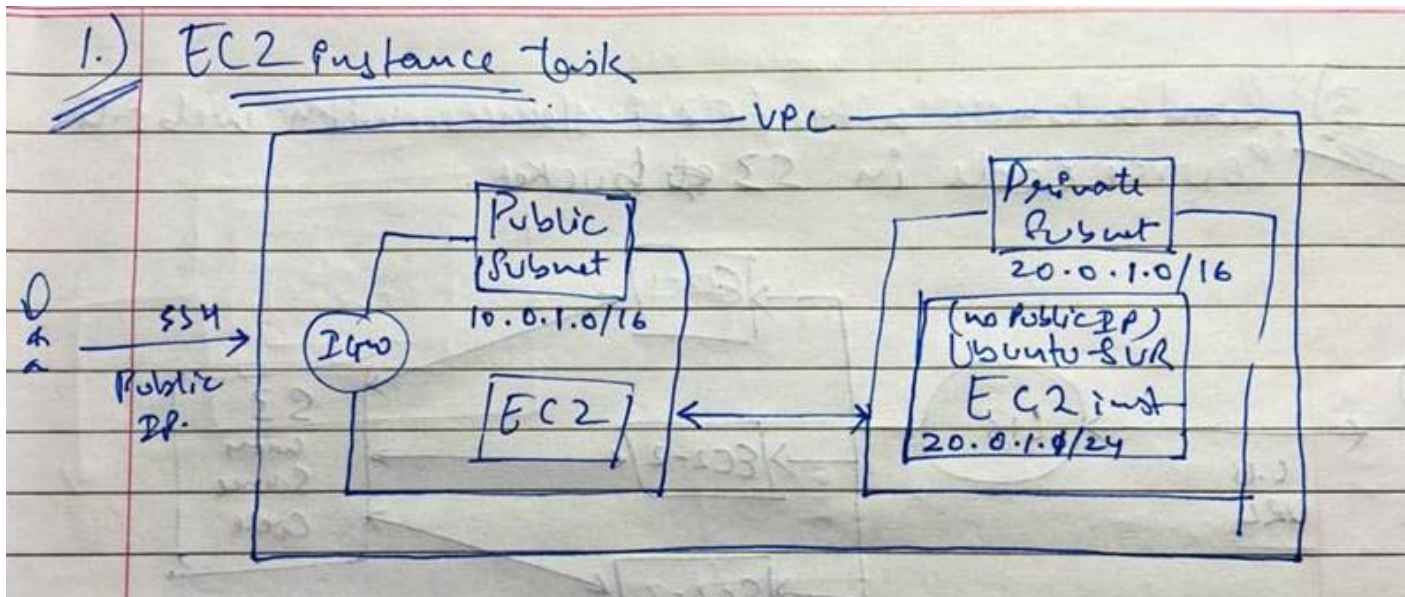
- The components of your distributed system,
- Your queue (distributed on Amazon SQS servers),
- and the messages in the queue.

Differences between Amazon SQS, Amazon MQ, and Amazon SNS:

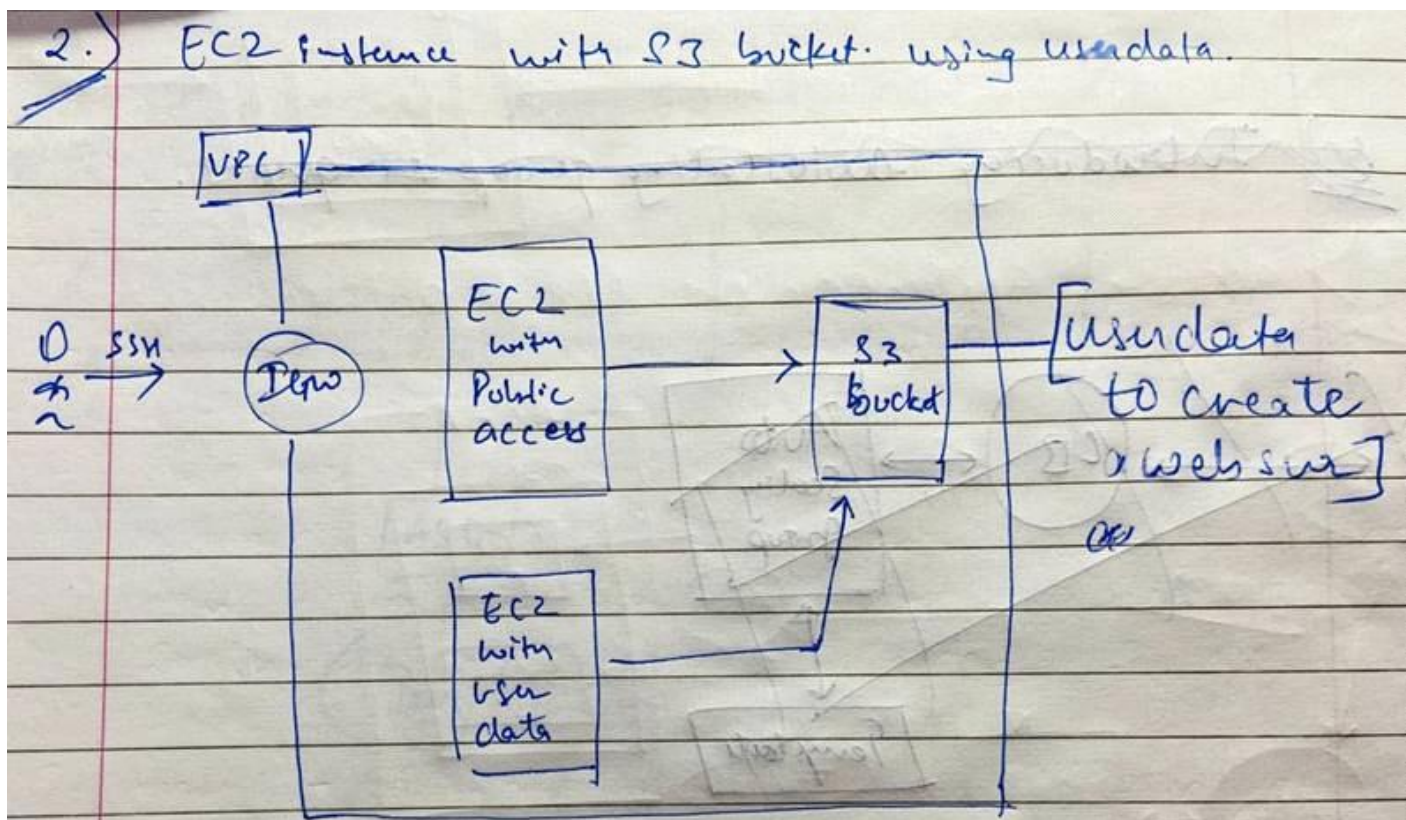
- **Amazon SQS** offers hosted queues that integrate and decouple distributed software systems and components. Amazon SQS provides a generic web services API that you can access using any programming language supported by AWS SDK. Messages in the queue are typically processed by a single subscriber. Amazon SQS and Amazon SNS are often used together to create a fanout messaging application.
- **Amazon SNS** is a publish-subscribe service that provides message delivery from publishers (also known as producers) to multiple subscriber endpoints(also known as consumers). Publishers communicate asynchronously with subscribers by sending messages to a topic, which is a logical access point and communication channel. Subscribers can subscribe to an Amazon SNS topic and receive published messages using a supported endpoint type, such as Amazon Kinesis Data Firehose, Amazon SQS, Lambda, HTTP, email, mobile push notifications, and mobile text messages (SMS). Amazon SNS acts as a message router and delivers messages to subscribers in real time. If a subscriber is not available at the time of message publication, the message is not stored for later retrieval.
- **Amazon MQ** is a managed message broker service that provides compatibility with industry standard messaging protocols such as Advanced Message Queueing Protocol (AMQP) and Message Queuing Telemetry Transport (MQTT). Amazon MQ currently supports Apache ActiveMQ and RabbitMQ engine types.

AWS scenario questions.

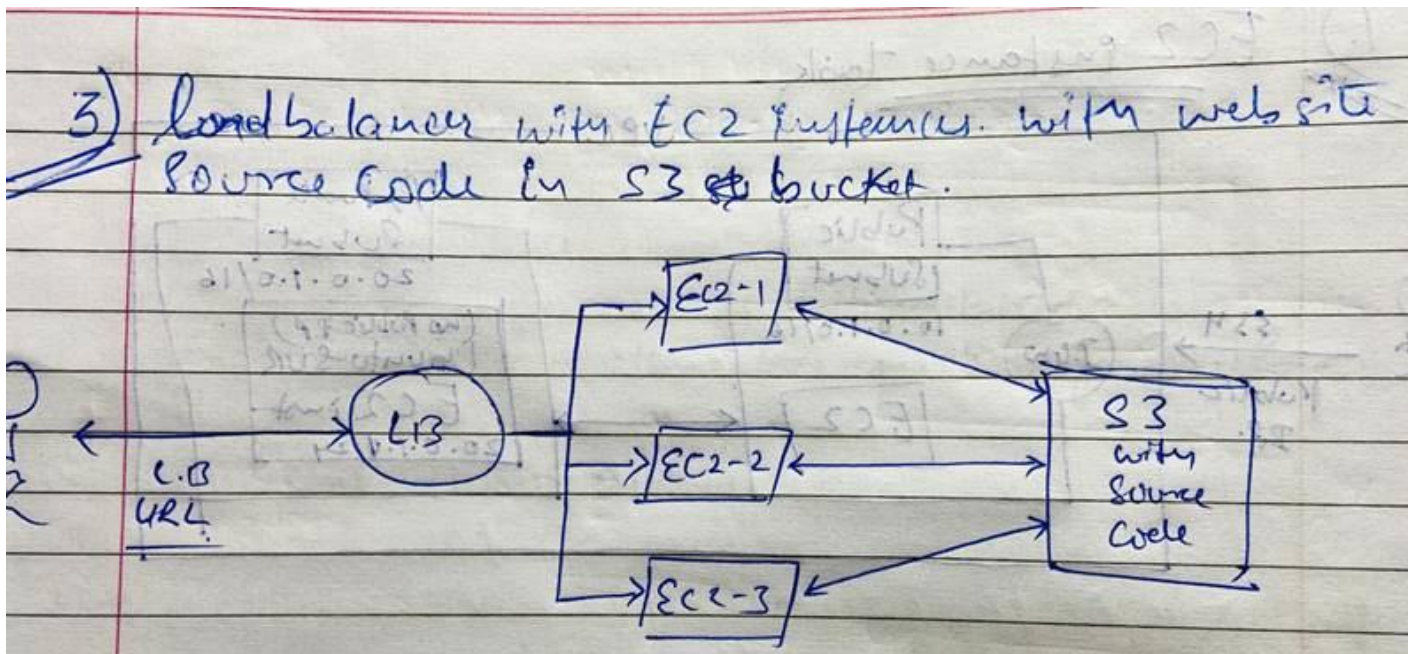
1. Create a VPC with a public & private subnet. Each subnet must have an ubuntu server with hosted website. Access the private instance with the help of public EC2 instance.



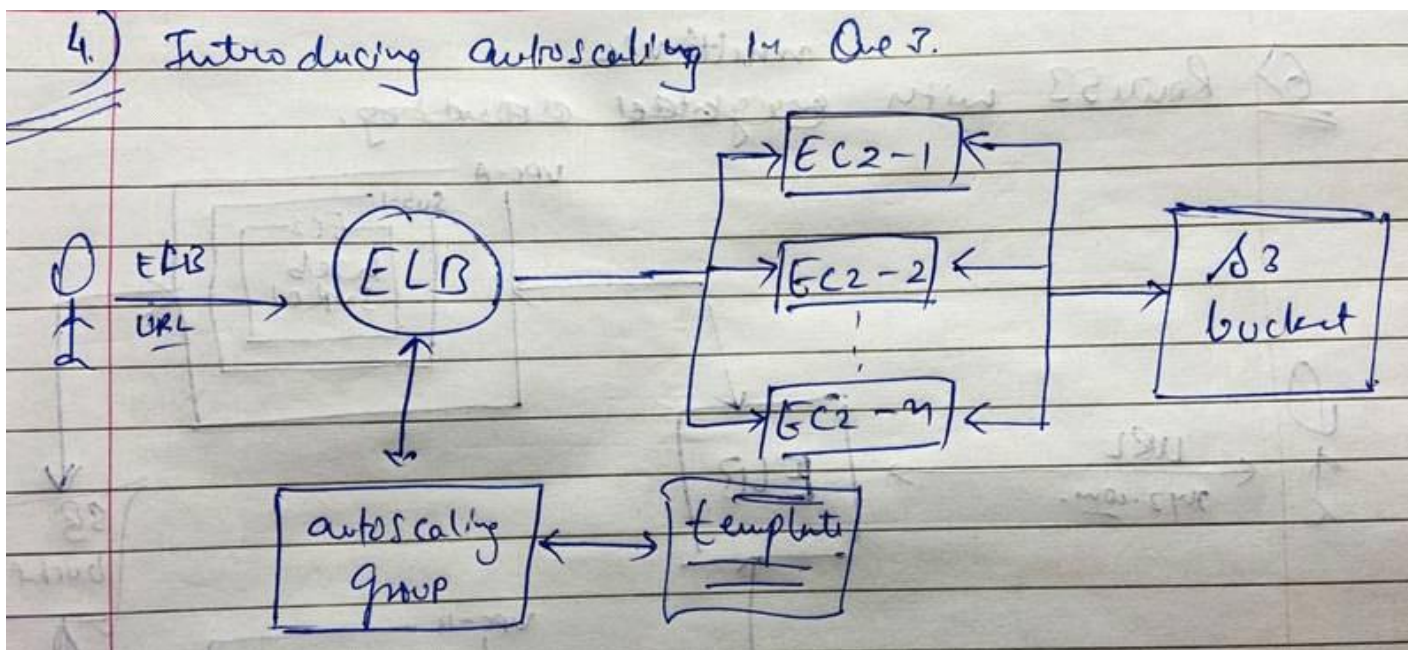
2. Create an 'index.html' file & upload it to a new S3 bucket. Use userdata script to fetch this S3 file & verify using browser with public IP address.



3. Create a load balancer in AWS and use userdata script to fetch data from S3 bucket & access the website using ELB URL.

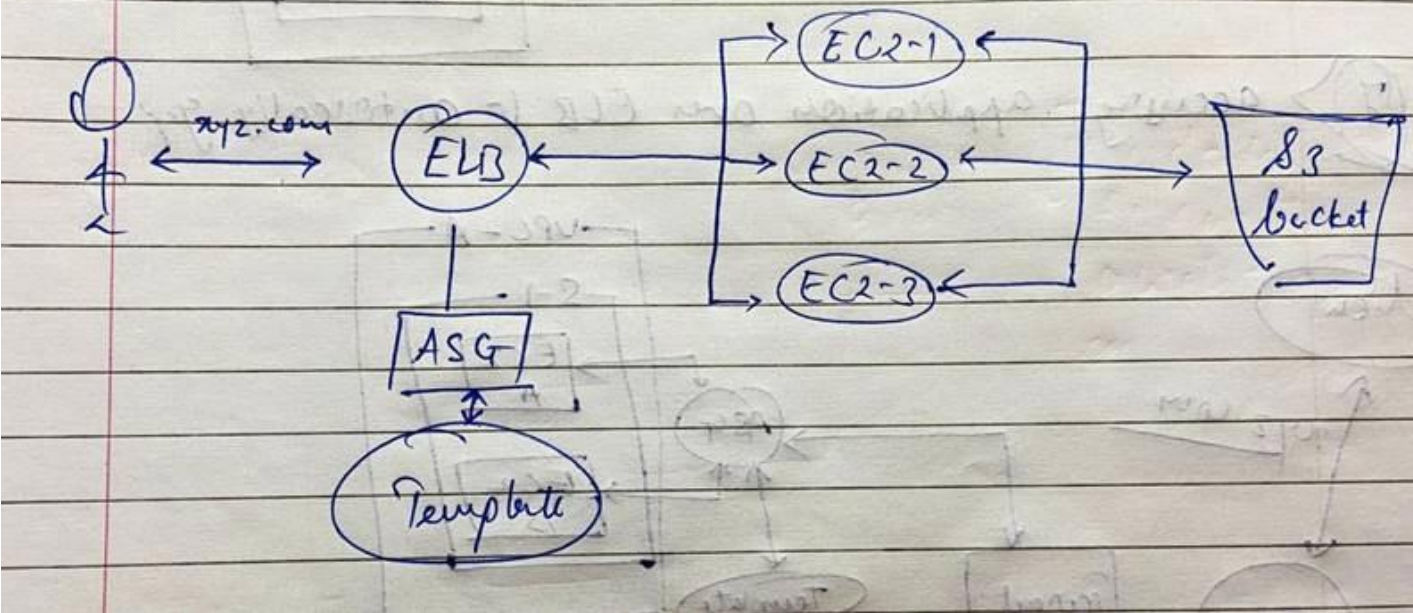


4. Extend question 3 and attach an autoscaling group with minimum of 2 & maximum of 5 instances. Use T2.Micro.

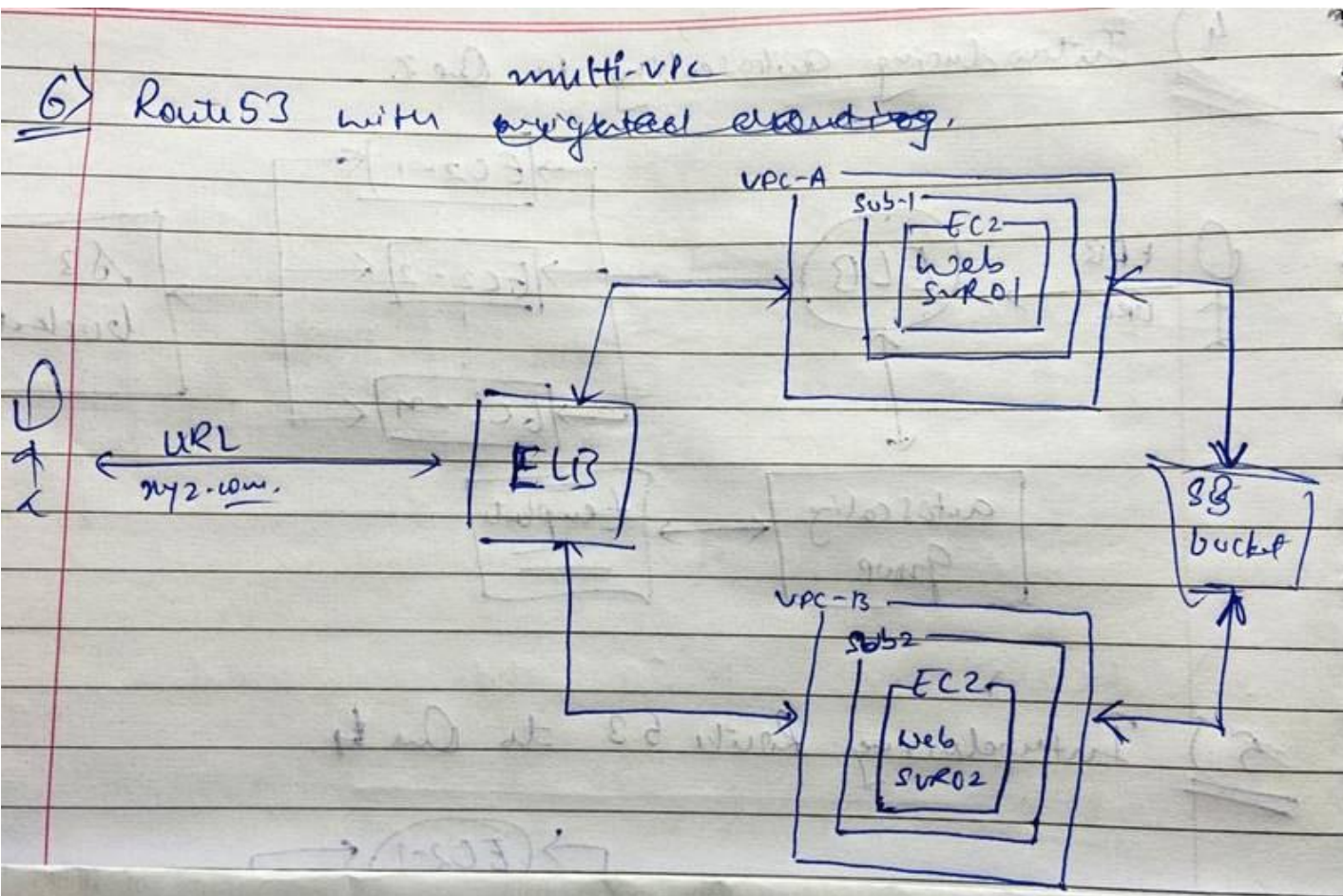


5. Use existing or purchase a new domain & attach it to an existing infrastructure.

5.) Integrating Route 53 to AWS

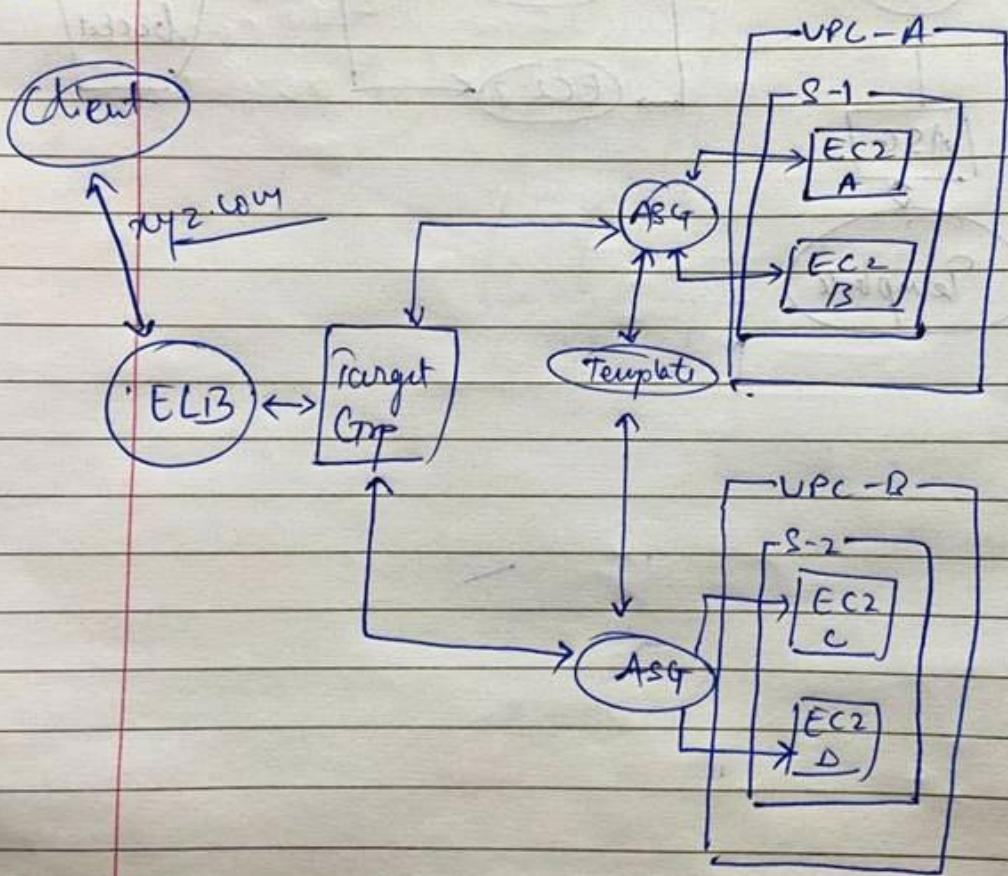


6. Create 2 VPCs with 2 public subnets. Create 2 EC2 instances each in separate subnets and connect both EC2 instances created in 2 different VPCs via ELB and access it using Route 53.



7. Create 2 VPCs. Each VPC contains 2 EC2 servers, connect all instances using an ELB & then access it via domain name.

⑦ accessing application over ELB to autoscaling grp.



AWS Lambda and its use cases

AWS Lambda is a serverless compute service offered by Amazon Web Services (AWS). It enables users to run code without the need to provision or manage servers. Instead, Lambda automatically scales and manages the infrastructure required to run the code in response to events or HTTP requests.

What it does?

- AWS Lambda allows developers to upload their code as functions, which are then triggered by various events or requests.
- When an event occurs, such as changes to data in an Amazon S3 bucket, updates to a DynamoDB table, or incoming HTTP requests through API Gateway, Lambda executes the corresponding function.
- Each function runs in an isolated environment, and the service automatically takes care of scaling, monitoring, and managing resources.

Use Cases:

- Event-Driven Processing:
 - Lambda functions can be triggered by events from various AWS services, such as object creation in S3, updates to a database in DynamoDB, or changes in an Amazon Kinesis stream. This makes Lambda ideal for building event-driven architectures.
- Microservices:
 - Developers can create small, independent functions for specific tasks, forming a microservices architecture. Each function can be independently deployed and scaled, promoting flexibility and modularity.
- Real-time File Processing:
 - Lambda can process files as they are uploaded to S3 or react to changes in a file system. This is useful for tasks such as image and video processing, log file analysis, or data validation.
- Backend for Mobile and Web Applications:
 - Lambda functions can serve as the backend logic for mobile or web applications, handling tasks like user authentication, data processing, or interacting with databases.
- IoT (Internet of Things):
 - Lambda can process and analyze data from IoT devices in real-time. For example, Lambda functions can react to sensor data, trigger alerts, or store information in a database.
- Scheduled Tasks:
 - Functions can be scheduled to run at specified intervals, making Lambda suitable for cron-like jobs, such as database backups, log rotation, or periodic data processing.

- Chatbots and Voice Assistants:
 - Lambda functions can be integrated with messaging platforms or voice services to power chatbots or voice-activated applications. This enables dynamic responses and interactions.
- Data Transformation and ETL (Extract, Transform, Load):
 - Lambda can be part of an ETL pipeline, transforming and loading data from one source to another. For example, data from a DynamoDB table can be transformed and loaded into a Redshift data warehouse.

Benefits:

- Scalability:
 - Lambda automatically scales based on the number of incoming requests. Whether you have a few requests or thousands, Lambda can handle the workload, ensuring optimal performance.
- Pay-Per-Use Model:
 - Users are billed for the actual compute time consumed by their functions, measured in milliseconds. This pay-per-use model can result in cost savings compared to traditional server-based models.
- No Server Management:
 - With Lambda, there is no need to provision or manage servers. This reduces operational overhead, allowing developers to focus solely on writing code.
- Automatic High Availability:
 - Lambda functions are automatically distributed across multiple availability zones, providing high availability and fault tolerance. AWS manages the infrastructure, ensuring robustness.
- Built-in Fault Tolerance:
 - Lambda automatically retries failed executions, and developers can configure error handling. This built-in fault tolerance enhances the reliability of applications.
- Quick Deployment:
 - Developers can quickly deploy code changes without dealing with the complexities of server provisioning. This facilitates rapid development and deployment cycles.
- Integration with Other AWS Services:
 - Lambda seamlessly integrates with various AWS services, allowing developers to build comprehensive solutions by combining different services.

When to use Lambda

Lambda is an ideal compute service for application scenarios that need to scale up rapidly, and scale down to zero when not in demand. For example, you can use Lambda for:

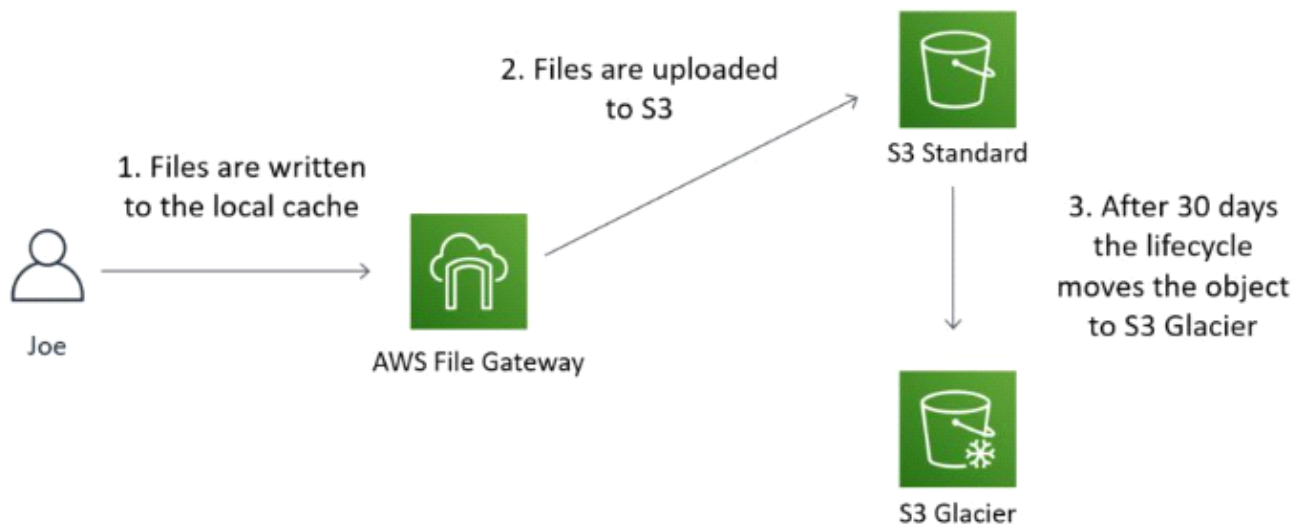
- **File processing:** Use Amazon Simple Storage Service (Amazon S3) to trigger Lambda data processing in real time after an upload.
- **Stream processing:** Use Lambda and Amazon Kinesis to process real-time streaming data for application activity tracking, transaction order processing, clickstream analysis, data cleansing, log filtering, indexing, social media analysis, Internet of Things (IoT) device data telemetry, and metering.
- **Web applications:** Combine Lambda with other AWS services to build powerful web

applications that automatically scale up and down and run in a highly available configuration across multiple data centers.

- **IoT backends:** Build serverless backends using Lambda to handle web, mobile, IoT, and third-party API requests.
- **Mobile backends:** Build backends using Lambda and Amazon API Gateway to authenticate and process API requests. Use AWS Amplify to easily integrate with your iOS, Android, Web, and React Native frontends.

AWS Glacier

Amazon Glacier is a low-cost, secure, and durable cloud storage service provided by Amazon Web Services (AWS). It is specifically designed for long-term data archival and retention. Amazon Glacier complements Amazon S3, providing an economical solution for storing data that is accessed infrequently and with a longer retrieval time.



What it does?

- Amazon Glacier is optimized for data that is rarely accessed and can tolerate retrieval times ranging from a few minutes to several hours.
- The service is designed to provide durable storage for archived data, making it suitable for compliance, regulatory, and backup purposes.
- Glacier achieves cost-effectiveness by utilizing low-cost storage media and by offering various retrieval options.

Use Cases:

- Data Archival:
 - Glacier is ideal for storing large amounts of data that needs to be archived for compliance or regulatory reasons. This could include financial records, legal documents, medical records, or any data that must be retained for an extended period.
- Backup and Recovery:
 - Organizations can use Glacier as part of their backup and recovery strategy. Backup archives can be stored in Glacier, ensuring the long-term retention of critical data while benefiting from the cost savings associated with infrequent access.
- Digital Preservation:
 - Glacier is well-suited for digital preservation efforts, such as archiving historical documents, images, videos, and other content that needs to be retained for future reference.
- Data Retention and Compliance:

- Industries with regulatory compliance requirements, such as finance and healthcare, can leverage Glacier to store data for the required retention periods. Glacier's low-cost storage options make it cost-effective for meeting compliance needs.
- **Media and Entertainment Archives:**
 - Media companies can use Glacier to store large libraries of media assets, including videos, audio recordings, and high-resolution images, that are not frequently accessed but need to be preserved for the long term.
- **Scientific Research Data:**
 - Glacier is suitable for storing scientific research data sets that may not be actively used but need to be preserved for future analysis and reference.

Types:

Amazon Glacier provides three retrieval options, each with different costs and retrieval times:

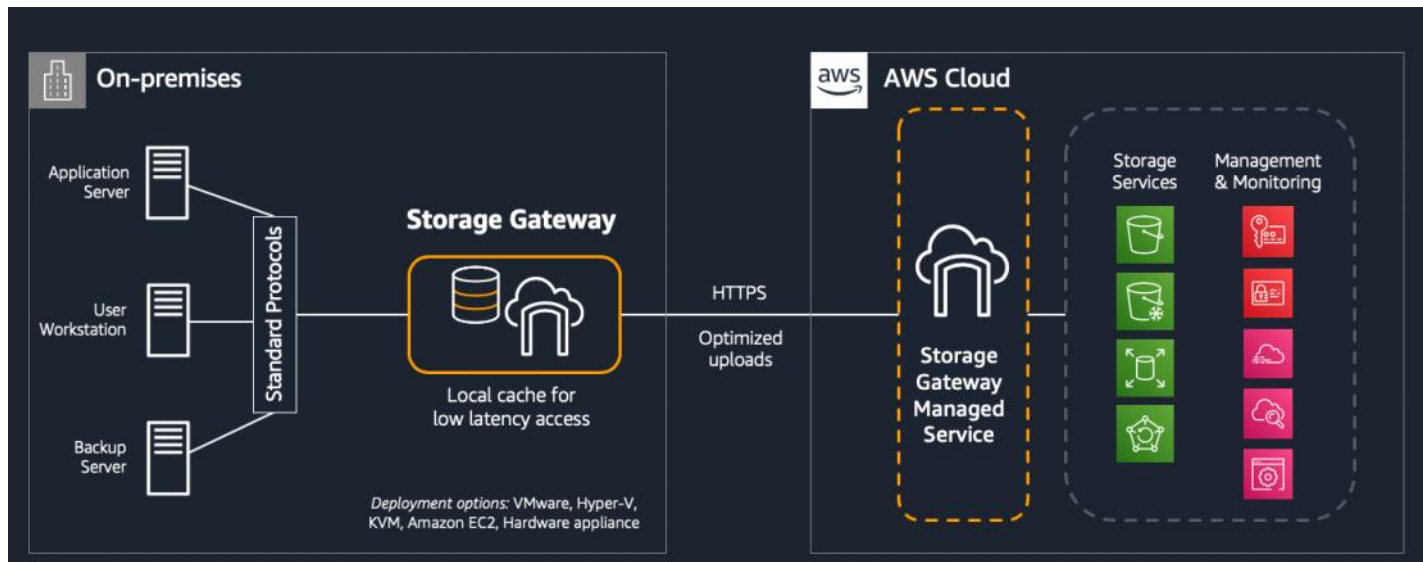
- **Expedited Retrieval:**
 - Suitable for situations where data needs to be retrieved within 1-5 minutes. This option comes with higher costs compared to others.
- **Standard Retrieval:**
 - Standard retrieval is appropriate for less time-sensitive scenarios where data can be retrieved within 3-5 hours. Costs are lower compared to expedited retrieval.
- **Bulk Retrieval:**
 - Bulk retrieval is the most cost-effective option, with retrieval times ranging from 5-12 hours. It is suitable for scenarios where data access speed is not critical.

Benefits:

- **Cost-Effective Storage:**
 - Glacier offers cost-effective storage options, making it significantly cheaper than more frequently accessed storage services like Amazon S3. This makes it an economical choice for archival needs.
- **Durability and Reliability:**
 - Amazon Glacier is designed for 99.999999999% (11 9's) durability, ensuring the long-term preservation and protection of archived data.
- **Security:**
 - Glacier provides built-in security features, including data encryption at rest and in transit. Access to stored data can be controlled using AWS Identity and Access Management (IAM) policies.
- **Scalability:**
 - Organizations can scale their storage needs easily by leveraging Glacier's scalable architecture. There are no limits on the amount of data that can be stored in Glacier.
- **Integration with Other AWS Services:**
 - Glacier can be seamlessly integrated with other AWS services, providing a comprehensive solution for archival storage within the AWS ecosystem.
- **Data Lifecycle Management:**
 - Glacier supports data lifecycle policies, allowing organizations to automate the transition of data from more expensive storage tiers to Glacier for long-term archival, further optimizing costs.
- **Low-Latency Retrieval Options:**
 - While Glacier is optimized for infrequent access, the service offers different retrieval options, allowing users to choose the level of speed and cost that aligns with their specific needs.

AWS Storage Gateway

AWS Storage Gateway is a hybrid cloud storage service that connects on-premises environments with cloud storage, providing a seamless and secure way to integrate on-premises IT infrastructure with the scalable and cost-effective storage solutions offered by Amazon Web Services (AWS). Storage Gateway allows businesses to extend their on-premises storage to the cloud, enabling a variety of use cases and facilitating data management and migration.



What it does?

- Storage Gateway acts as a bridge between on-premises data centers and cloud storage, providing a gateway that allows applications to securely store and retrieve data to and from AWS.
- It integrates with different storage protocols, making it compatible with existing on-premises applications.

Use Cases:

- **Cloud Backup and Restore:**
 - Storage Gateway allows businesses to back up on-premises data to AWS cloud storage, providing a scalable and cost-effective backup solution. In the event of data loss or corruption, organizations can easily restore data from the cloud.
- **Disaster Recovery:**
 - Storage Gateway facilitates disaster recovery strategies by enabling organizations to replicate on-premises data to AWS for storage. In case of a disaster or outage, data can be quickly recovered from the cloud to ensure business continuity.
- **Tiered Storage:**
 - Organizations can use Storage Gateway to implement tiered storage architectures. Frequently accessed data can reside on local storage, while less frequently accessed data can be moved to cost-effective AWS storage services, such as Amazon S3 or Amazon Glacier.
- **Data Migration:**
 - Storage Gateway simplifies data migration to the cloud. Businesses can seamlessly move data from on-premises environments to AWS, either for archival purposes or to take advantage of cloud-based processing and analytics services.
- **File Sharing and Collaboration:**

- By integrating Storage Gateway with AWS, organizations can create shared file systems that can be accessed by multiple users and applications both on-premises and in the cloud. This facilitates collaboration and data sharing across distributed teams.
- Content Distribution:
 - Storage Gateway supports caching of frequently accessed data on-premises, reducing latency for applications that require quick access to content. This is particularly useful for media and content distribution where low-latency access is crucial.
- Backup to Cloud:
 - Storage Gateway can be used to back up on-premises applications directly to AWS cloud storage, eliminating the need for on-premises tape libraries or other backup infrastructure. This simplifies backup processes and reduces operational overhead.

Types:

Storage Gateway offers different types to cater to various use cases:

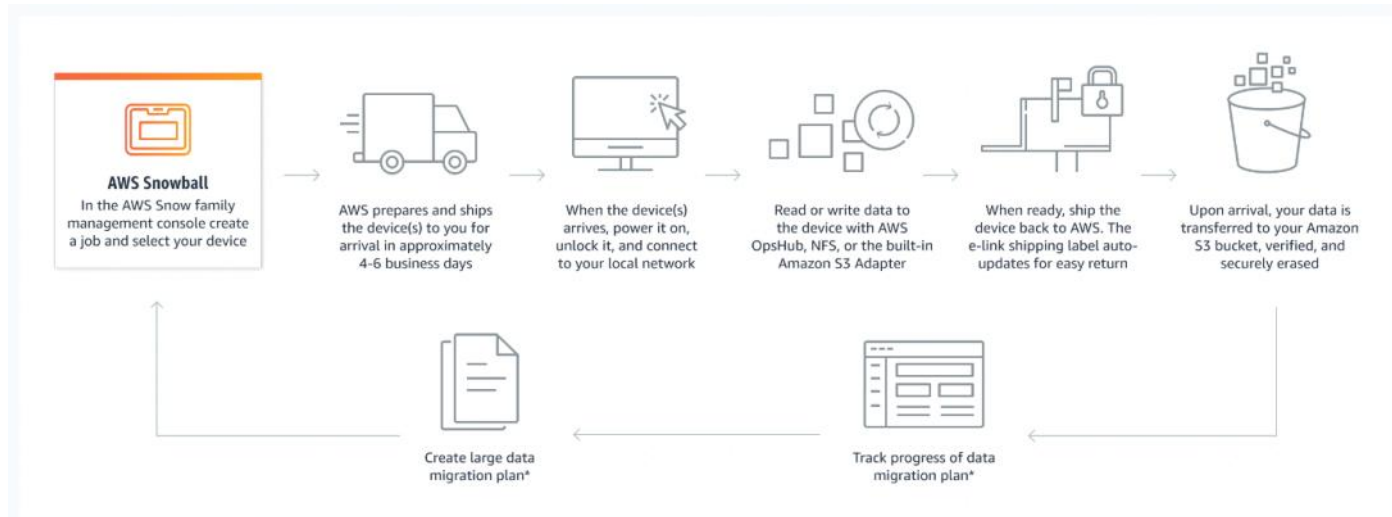
- File Gateway:
 - Presents a file interface to applications, allowing them to store files as objects in Amazon S3. It supports the Network File System (NFS) and Server Message Block (SMB) protocols.
- Volume Gateway:
 - Presents cloud storage volumes to applications as iSCSI devices. There are two configurations: Cached Volumes, where the primary data is stored in AWS, and Stored Volumes, where the entire dataset is stored on-premises and backed up to AWS.
- Tape Gateway:
 - Presents a virtual tape library interface to backup applications. Data written to the virtual tape library is stored in Amazon S3 or Glacier, providing a scalable and durable backup solution.

Benefits:

- Hybrid Cloud Integration:
 - Storage Gateway seamlessly integrates on-premises environments with AWS cloud storage, enabling a hybrid cloud architecture that provides flexibility and scalability.
- Cost-Effective Storage:
 - By leveraging cloud storage, organizations can reduce on-premises storage costs and take advantage of the cost-effectiveness of AWS storage services, paying only for the storage capacity they consume.
- Scalability:
 - Storage Gateway allows organizations to scale their storage infrastructure easily by leveraging the scalable nature of AWS cloud storage.
- Simplified Data Management:
 - Storage Gateway simplifies data management by providing a unified interface for on-premises and cloud storage. This reduces complexity and streamlines data access and retrieval.
- Data Security:
 - Data transmitted between on-premises environments and AWS is encrypted, ensuring the security and integrity of data during transfer. Additionally, AWS IAM policies can be used to control access to the stored data.
- Flexible Deployment Models:
 - Storage Gateway supports different deployment models, allowing organizations to choose the configuration that best fits their requirements, whether for backup, disaster recovery, or tiered storage.
- Reduced Infrastructure Maintenance:
 - With the offloading of data to the cloud, organizations can reduce the need for on-premises infrastructure maintenance, leading to operational efficiency.

AWS Snowball

AWS Snowball is a physical data transport solution provided by Amazon Web Services (AWS) to facilitate the transfer of large amounts of data between on-premises environments and the AWS cloud. Snowball devices are ruggedized, secure, and designed to simplify the process of moving data to and from AWS when high-speed internet connections are impractical or cost-prohibitive.



What it does?

- AWS Snowball allows customers to transfer massive datasets to and from AWS securely and quickly.
- The service provides a physical device, called a Snowball, which is a ruggedized storage appliance that customers can use to load their data.
- Once the data is loaded onto the Snowball device, it is shipped to an AWS data center, where the data is transferred to the customer's AWS S3 storage.

Use Cases:

- **Large-Scale Data Migration:**
 - Snowball is particularly useful when dealing with large-scale data migrations. Whether moving data to the cloud for the first time or transferring substantial datasets between AWS regions, Snowball provides a cost-effective and efficient solution.
- **Offline Data Transfer:**
 - In scenarios where transferring data over the internet would be time-consuming or expensive, Snowball offers an offline solution. Customers can load data onto the Snowball device at their location, and then the device is physically transported to an AWS data center.
- **Data Ingestion for Big Data and Analytics:**
 - Organizations with substantial datasets for big data analytics or machine learning can use Snowball to move data efficiently into AWS. This is especially beneficial for time-sensitive projects where waiting for data to be transferred over the network is not practical.
- **Content Distribution:**
 - Media and content providers can use Snowball to distribute large volumes of data, such as video libraries or high-resolution imagery, to AWS for content processing, storage, and distribution.

- **Data Backups and Recovery:**
 - Snowball can be employed for creating secure backups of large datasets. Organizations can load backup data onto Snowball devices, ensuring a quick and efficient method for offsite data storage and recovery.
- **Temporary Data Transfer for Events or Projects:**
 - For events, research projects, or temporary workloads requiring a large amount of data to be moved into or out of AWS, Snowball provides a temporary and scalable solution without the need for a long-term network connection.

Types:

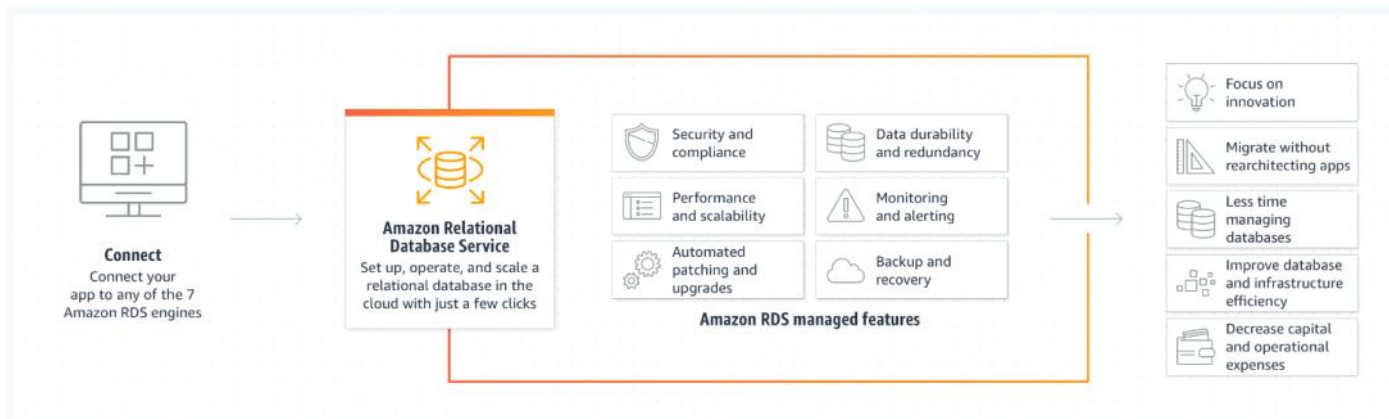
- **Snowball Edge:**
 - Snowball Edge is a more advanced version of Snowball that includes on-board compute capabilities. It supports local data processing with AWS Lambda functions, making it suitable for edge computing scenarios. It can also function as a storage appliance and supports clustering for high-availability use cases.
- **Snowmobile:**
 - Snowmobile is an even larger-scale data transport solution. Instead of a portable device, Snowmobile is a 45-foot long shipping container that can transfer up to 100 petabytes of data. It is used for massive data center migrations or large-scale data transfers.

Benefits:

- **Fast Data Transfer:**
 - Snowball provides a significantly faster data transfer mechanism compared to transferring data over the internet, especially for large datasets. This is particularly valuable for meeting tight deadlines and avoiding extended transfer times.
- **Security and Encryption:**
 - Data loaded onto Snowball devices is encrypted with 256-bit encryption, and AWS Key Management Service (KMS) can be used to manage encryption keys. This ensures the security of the data during transfer and storage.
- **Rugged Design:**
 - Snowball devices are designed to withstand harsh conditions during transit. They are ruggedized and have built-in environmental controls to protect the data during transportation.
- **Simplified Process:**
 - AWS Snowball simplifies the data transfer process by providing a physical device that customers can load with their data. This eliminates the need for complex network configurations and reduces the overall effort required for large-scale data migrations.
- **Cost-Effective:**
 - For large datasets, Snowball can be a cost-effective solution compared to using high-speed internet connections. The pricing model is transparent, and customers are billed based on the number of Snowball devices used.
- **Edge Computing with Snowball Edge:**
 - Snowball Edge extends the use cases beyond pure data transfer. Its on-board compute capabilities allow for edge computing scenarios, where data processing can occur at the edge, reducing the need to send all data back to the cloud.
- **Scalability with Snowmobile:**
 - Snowmobile is designed for massive-scale data transfers, making it suitable for organizations dealing with extremely large datasets, such as those in the petabyte range.

Amazon RDS (Relation Database Service)

Amazon RDS is a fully managed relational database service provided by Amazon Web Services (AWS). It enables users to set up, operate, and scale relational databases in the cloud without the need for manual intervention in common database administration tasks. RDS supports several popular relational database engines, making it a versatile and scalable solution for a wide range of applications.



What it does?

- Amazon RDS automates time-consuming administrative tasks such as hardware provisioning, database setup, patching, and backups.
- It allows users to focus on building and optimizing their applications rather than managing the underlying infrastructure.
- RDS supports multiple database engines, including MySQL, PostgreSQL, MariaDB, Oracle, and Microsoft SQL Server.

Use Cases:

- **Web and Mobile Applications:**
 - RDS is commonly used as a backend database for web and mobile applications. It provides a scalable and reliable storage solution for application data, allowing developers to focus on building features rather than managing databases.
- **E-commerce Applications:**
 - E-commerce platforms leverage Amazon RDS to store product catalogs, customer information, and transaction data. The managed service ensures high availability and reliability for online retail operations.
- **Content Management Systems (CMS):**
 - CMS platforms use RDS to store and manage content, user data, and configuration settings. The ease of scaling and automated backups make it a suitable choice for managing content-rich websites.
- **Data Warehousing:**
 - While Amazon RDS is primarily designed for transactional workloads, it can be used for smaller-scale data warehousing scenarios. For larger-scale data warehousing needs, Amazon Redshift is often a more suitable AWS service.
- **Business Applications:**
 - RDS is employed for hosting various business applications, including customer relationship management (CRM) systems, enterprise resource planning (ERP) solutions, and other business-critical applications.
- **Dev/Test Environments:**
 - Development and testing environments can benefit from the ease of provisioning and managing databases with RDS. It allows developers to quickly create

database instances for testing and development purposes.

- WordPress and Content-Based Websites:
 - WordPress websites and other content-based platforms often use RDS as the backend database. The managed service ensures that the database infrastructure is reliable and scalable as traffic and data requirements grow.

Database Engines:

Amazon RDS supports various database engines, each catering to specific use cases:

- MySQL:
 - An open-source relational database management system. It is known for its performance, reliability, and ease of use.
- PostgreSQL:
 - An open-source object-relational database system known for its extensibility and support for advanced data types.
- MariaDB:
 - A community-developed fork of MySQL, designed to remain open-source and maintain compatibility while introducing new features.
- Oracle:
 - A relational database management system developed by Oracle Corporation, widely used in enterprise applications.
- Microsoft SQL Server:
 - A relational database management system developed by Microsoft, commonly used in Windows environments.

Benefits:

- Automated Database Management:
 - RDS automates routine database management tasks, including software patching, backups, and hardware provisioning.
 - This allows users to focus on application development rather than database administration.
- High Availability and Reliability:
 - RDS provides high availability through automated backups, automated software patching, and multi-AZ deployments.
 - Multi-AZ deployments replicate databases across multiple availability zones to enhance reliability and fault tolerance.
- Scalability:
 - RDS supports horizontal and vertical scaling.
 - Users can easily scale their database instances vertically by choosing a larger instance type or horizontally by adding read replicas for read-intensive workloads.
- Security Features:
 - RDS includes built-in security features such as encryption at rest and in transit, network isolation, and automated software patching to enhance the security of the database environment.
- Cost-Efficiency:
 - With RDS, users pay only for the resources they consume, and the managed service eliminates the need for upfront investments in hardware and ongoing maintenance costs.
- Performance Monitoring and Metrics:
 - RDS provides performance monitoring and metrics through Amazon CloudWatch.
 - Users can set up alarms and notifications based on predefined metrics or custom performance thresholds.
- Automated Backups and Point-in-Time Recovery:
 - RDS automatically takes regular backups, and users can initiate point-in-time

recovery to restore their database to a specific point in time, minimizing data loss in case of errors or failures.

- Read Replicas:
 - RDS allows users to create read replicas for read scalability.
 - Read replicas can be used to offload read traffic from the primary database and improve overall performance.
- Easy Database Migration:
 - RDS supports easy migration of databases to and from the cloud.
 - Users can import/export data, use database snapshots, or utilize AWS Database Migration Service (DMS) for seamless migrations.

Amazon RDS - DB Instances

A DB instance is an isolated database environment in the AWS Cloud. The basic building block of Amazon RDS is the DB instance.

Your DB instance can contain one or more user-created databases. You can access your DB instance by using the same tools and applications that you use with a standalone database instance. You can create and modify a DB instance by using the AWS Command Line Interface (AWS CLI), the Amazon RDS API, or the AWS Management Console.

DB engines

A DB engine is the specific relational database software that runs on your DB instance. Amazon RDS currently supports the following engines:

- Db2
- MariaDB
- Microsoft SQL Server
- MySQL
- Oracle
- PostgreSQL

DB instance classes

A DB instance class determines the computation and memory capacity of a DB instance. A DB instance class consists of both the DB instance type and the size. Each instance type offers different compute, memory, and storage capabilities. For example, db.m6g is a general-purpose DB instance type powered by AWS Graviton2 processors. Within the db.m6g instance type, db.m6g.2xlarge is a DB instance class.

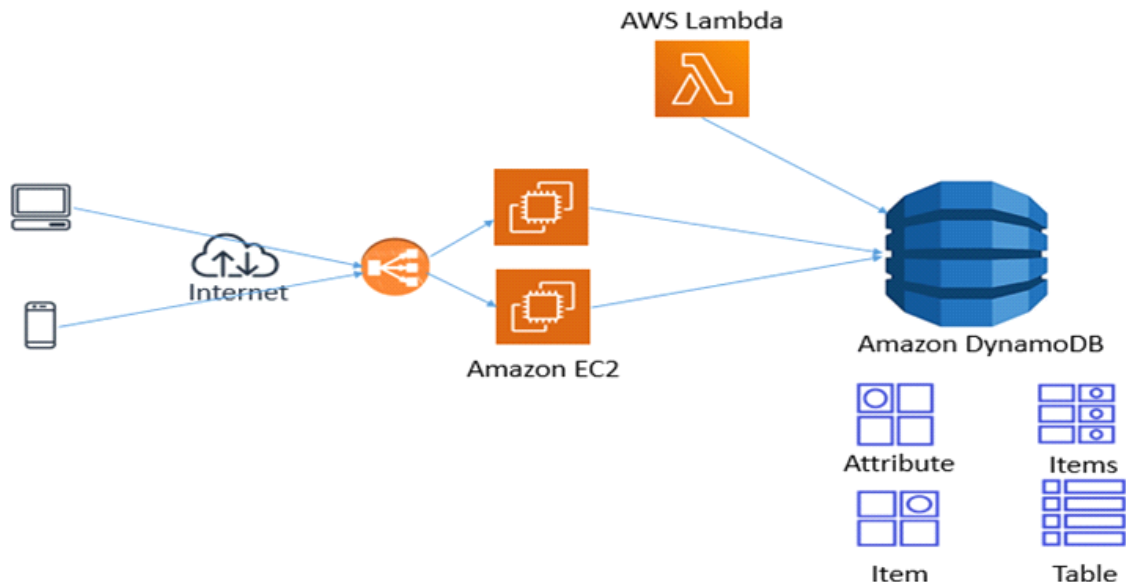
DB instance storage

Amazon EBS provides durable, block-level storage volumes that you can attach to a running instance. DB instance storage comes in the following types:

- General Purpose (SSD)
- Provisioned IOPS (PIOPS)
- Magnetic

Amazon DynamoDB

Amazon DynamoDB is a fully managed NoSQL database service provided by Amazon Web Services (AWS). It is designed to deliver high performance, scalability, and low-latency access to applications that require fast and predictable performance at any scale. DynamoDB is a serverless database, which means AWS takes care of the operational aspects, allowing developers to focus on building applications.



What it does:

- DynamoDB is a key-value and document database that provides single-digit millisecond latency at any scale.
- It supports both document and key-value data models, and its architecture is optimized for horizontal scalability.
- DynamoDB automatically replicates data across multiple Availability Zones within an AWS region to ensure high availability and fault tolerance.

Use Cases:

- Web and Mobile Applications:
 - DynamoDB is well-suited for web and mobile applications that require low-latency access to data. It can handle high volumes of read and write requests, making it suitable for dynamic and interactive applications.
- Gaming Applications:
 - Online gaming applications often require real-time access to player data, leaderboards, and game state. DynamoDB's low-latency and high-throughput capabilities make it a suitable choice for gaming scenarios.
- IoT (Internet of Things):
 - DynamoDB is used in IoT applications where devices generate a large volume of data. It can handle the ingestion and retrieval of IoT data efficiently, providing a scalable solution for storing and querying sensor data.
- Ad Tech:
 - Ad tech platforms that require quick and efficient storage and retrieval of user profiles, ad impressions, and campaign data can benefit from DynamoDB's

performance and scalability.

- E-commerce Platforms:
 - DynamoDB is commonly used in e-commerce applications for managing product catalogs, customer profiles, and order information. Its ability to handle high traffic and provide low-latency responses is crucial for e-commerce platforms.
- Content Management Systems (CMS):
 - CMS platforms that need to manage content metadata, user profiles, and dynamic content can leverage DynamoDB for its fast and scalable storage capabilities.
- Real-Time Analytics:
 - DynamoDB is used in scenarios where real-time analytics on large datasets are required. Its ability to handle high-throughput queries makes it suitable for applications that demand quick insights into data.
- Session Management:
 - Applications requiring session management, such as maintaining user sessions for web applications, can benefit from DynamoDB's ability to quickly store and retrieve session data.
- Caching and Metadata Storage:
 - DynamoDB can be used as a cache store or for storing metadata associated with other storage systems. Its low-latency access makes it suitable for scenarios where quick access to frequently accessed data is essential.

Key Features:

- Fully Managed:
 - DynamoDB is fully managed by AWS, taking care of operational aspects such as hardware provisioning, setup, configuration, and maintenance.
- Scalability:
 - DynamoDB provides seamless and automatic scalability. As the workload increases, the service automatically scales to handle higher read and write throughput.
- Low Latency:
 - DynamoDB offers single-digit millisecond latency for read and write operations, ensuring quick access to data for applications with low-latency requirements.
- Global Tables:
 - DynamoDB supports global tables, allowing data to be replicated across multiple AWS regions. This enables low-latency access for users distributed globally.
- Security:
 - DynamoDB supports encryption at rest and in transit. It integrates with AWS Identity and Access Management (IAM) for access control, providing a secure environment for data.
- On-Demand Capacity:
 - Users can choose on-demand capacity, which automatically scales based on the actual read and write requirements. This eliminates the need to provision and manage capacity in advance.
- Automatic Backups:
 - DynamoDB automatically creates and retains backups of tables, providing point-in-time recovery options in case of accidental data deletion or corruption.
- Event-Driven Programming:
 - DynamoDB Streams allows developers to capture changes to data in a table, enabling event-driven programming. This is useful for building applications with reactive and real-time features.
- DAX (DynamoDB Accelerator):
 - DAX is a fully managed, highly available, in-memory cache for DynamoDB that provides acceleration for read-heavy workloads, reducing response times.

Benefits:

- **Serverless Architecture:**
 - DynamoDB is a fully managed serverless database, eliminating the need for users to manage the underlying infrastructure. Developers can focus on building applications without worrying about database operations.
- **High Performance:**
 - DynamoDB is designed for high performance, providing low-latency access to data at any scale. This makes it suitable for applications with demanding performance requirements.
- **Scalability on Demand:**
 - DynamoDB scales automatically based on the workload, allowing applications to handle varying levels of traffic without manual intervention.
- **Global Reach:**
 - With global tables, DynamoDB enables low-latency access to data for users and applications distributed across different geographical regions.
- **Ease of Development:**
 - DynamoDB's simple and flexible data model, combined with SDKs for various programming languages, makes it easy for developers to interact with and integrate DynamoDB into their applications.
- **Cost-Effective:**
 - Users pay for the throughput and storage they consume, and DynamoDB's on-demand capacity option eliminates the need for capacity planning. This results in cost-effective pricing for varying workloads.
- **Managed Security:**
 - DynamoDB takes care of security aspects, including encryption, access control, and automatic backups, ensuring a secure and compliant environment for data storage.
- **Integration with AWS Ecosystem:**
 - DynamoDB seamlessly integrates with other AWS services, allowing users to build comprehensive solutions within the AWS ecosystem.
 - It integrates with AWS Lambda, AWS CloudFormation, AWS CloudWatch.

Amazon Redshift

Amazon Redshift is a fully managed, petabyte-scale data warehouse service provided by Amazon Web Services (AWS). It is designed to handle large-scale datasets and enable high-performance analysis of structured and semi-structured data. Amazon Redshift uses a columnar storage format and parallel processing capabilities to deliver fast query performance for analytics and business intelligence workloads.

What it does?

- Amazon Redshift allows organizations to analyze large volumes of data using SQL queries and business intelligence tools.
- It uses a massively parallel processing (MPP) architecture, where data is distributed across multiple nodes for parallel execution of queries.
- The columnar storage format enhances compression and accelerates query performance by reading only the necessary columns, optimizing for analytical workloads.

Use Cases:

- Data Warehousing:
 - Amazon Redshift is primarily designed for data warehousing and analytics. It is suitable for organizations that need to store and analyze large volumes of data for reporting, business intelligence, and decision-making.
- Business Intelligence (BI):
 - Redshift is commonly used in conjunction with business intelligence tools to analyze and visualize data. It allows organizations to derive insights from their data, create dashboards, and make data-driven decisions.
- Data Exploration and Ad Hoc Queries:
 - Redshift is well-suited for ad hoc queries and data exploration. Analysts and data scientists can interactively query large datasets to discover patterns, trends, and insights.
- ETL (Extract, Transform, Load) Processing:
 - Redshift can be part of ETL pipelines where data is extracted from various sources, transformed, and loaded into Redshift for analysis. Its ability to handle large-scale transformations and load operations makes it suitable for ETL workloads.
- Log Analysis:
 - Organizations with large volumes of log data, such as web logs or application logs, can use Redshift to analyze and derive insights from this data. This is particularly useful for monitoring and optimizing application performance.
- Time-Series Data Analysis:
 - Redshift is effective for analyzing time-series data, such as historical sales data, sensor data, or any data that evolves over time. Its capabilities support efficient querying and aggregation over time-based data.
- Predictive Analytics:
 - Redshift can be used as part of predictive analytics workflows, where historical data is analyzed to build models and make predictions.
 - The scalability and performance of Redshift support such analytical workloads.

Key Features:

- **Columnar Storage:**
 - Redshift uses a columnar storage format, storing data in columns rather than rows.
 - This enhances compression and improves query performance by reading only the necessary columns, reducing I/O.
- **Massively Parallel Processing (MPP):**
 - Redshift distributes data and query execution across multiple nodes, allowing parallel processing of queries.
 - This architecture enables high-performance analytics on large datasets.
- **Auto-scaling:**
 - Redshift supports automatic and manual resizing of clusters, allowing organizations to scale their data warehouse up or down based on changing workloads.
 - This ensures optimal performance and cost efficiency.
- **Managed Service:**
 - Redshift is fully managed by AWS, taking care of routine administrative tasks such as hardware provisioning, setup, configuration, and maintenance. This allows users to focus on data analysis rather than database management.
- **Integration with BI Tools:**
 - Redshift integrates seamlessly with popular business intelligence tools such as Tableau, Amazon QuickSight, Looker, and others.
 - This allows users to visualize and analyze data using their preferred BI tools.
- **Security:**
 - Redshift provides features such as encryption at rest and in transit, access controls through AWS Identity and Access Management (IAM), and Virtual Private Cloud (VPC) support to secure data and access.
- **Concurrency Scaling:**
 - Concurrency Scaling allows Redshift to automatically add and remove capacity to handle varying levels of concurrent queries.
 - This ensures consistent performance even during peak usage periods.
- **Data Compression and Encoding:**
 - Redshift uses advanced compression and encoding techniques to minimize storage requirements and improve query performance.
 - This is essential for optimizing the storage and processing of large datasets.

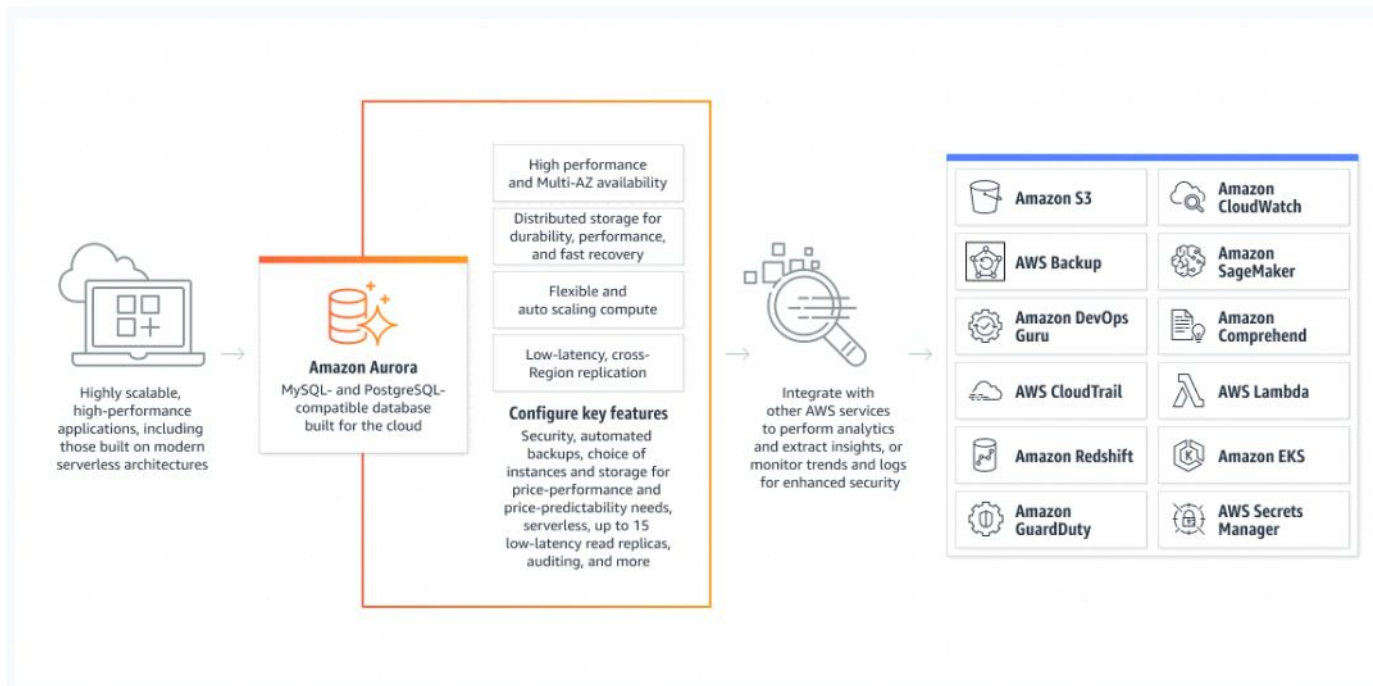
Benefits:

- **Scalability:**
 - Redshift is highly scalable, allowing organizations to scale their data warehouse up or down based on changing requirements. This ensures optimal performance and cost efficiency.
- **Performance:**
 - With its MPP architecture and columnar storage format, Redshift delivers fast query performance, making it suitable for complex analytical queries on large datasets.
- **Ease of Management:**
 - Redshift is a fully managed service, reducing the operational overhead for organizations.
 - AWS takes care of routine maintenance tasks, allowing users to focus on analytics.
- **Cost-Efficiency:**
 - Redshift's pay-as-you-go pricing model allows organizations to pay only for the resources they consume.

- This can result in cost savings compared to traditional on-premises data warehouses.
- Integration with BI Tools:
 - Redshift seamlessly integrates with popular BI tools, providing flexibility for users to analyze and visualize data using their preferred tools.
- Security and Compliance:
 - Redshift includes features for securing data, such as encryption, access controls, and VPC support.
 - It helps organizations meet security and compliance requirements.
- Flexibility:
 - Redshift supports various data types and offers flexibility in schema design.
 - It can handle both structured and semi-structured data, providing versatility for different analytical workloads.
- Automatic Backups and Snapshots:
 - Redshift automatically takes backups and snapshots, allowing users to recover data to a specific point in time.
 - This provides data protection and recovery capabilities.

Amazon Aurora

Amazon Aurora is a fully managed relational database service provided by Amazon Web Services (AWS). It is compatible with MySQL and PostgreSQL, offering high performance, availability, and durability. Amazon Aurora is designed to provide the benefits of commercial databases with the cost-effectiveness and simplicity of open-source databases.



What it does?

- Amazon Aurora provides a fully managed, highly available, and scalable relational database engine.
- It is designed to deliver high performance and reliability while reducing the operational overhead typically associated with managing traditional relational databases.
- Aurora uses a distributed and fault-tolerant architecture to ensure high availability and durability of data.

Use Cases:

- Transactional Workloads:
 - Aurora is well-suited for transactional workloads, such as those found in e-commerce platforms, financial applications, and various business applications where data consistency and reliability are critical.
- Content Management Systems (CMS):
 - CMS platforms that require a relational database for managing content, user data, and configurations can benefit from Aurora's performance and scalability.
- Enterprise Applications:
 - Aurora is suitable for a wide range of enterprise applications, including customer relationship management (CRM) systems, enterprise resource planning (ERP) solutions, and other business-critical applications.
- Online Analytical Processing (OLAP):
 - Aurora can be used for OLAP workloads, supporting analytical queries and reporting on large datasets. Its high-performance capabilities make it suitable for real-time analytics.
- Log and Event Processing:

- Applications that generate large volumes of log data or events can use Aurora to store and manage this data. Its scalability and reliability are beneficial for handling high-throughput workloads.
- Data Warehousing:
 - While Amazon Aurora is primarily designed for transactional workloads, it can be used for smaller-scale data warehousing scenarios where the focus is on real-time analytics and reporting.
- Microservices Architecture:
 - Aurora is well-suited for microservices architectures where multiple services need to interact with a shared database. Its compatibility with MySQL and PostgreSQL allows for easy integration with various development frameworks.

Key Features:

- High Availability:
 - Aurora provides high availability with automatic failover and replication across multiple Availability Zones within an AWS region. This ensures that applications remain operational even in the event of hardware or software failures.
- Performance:
 - Aurora offers high performance with low-latency read and write operations. It uses a distributed and parallel architecture to achieve efficient query execution.
- Auto-Scaling:
 - Aurora supports auto-scaling, allowing the database cluster to automatically adjust its compute and storage capacity based on the actual usage. This ensures optimal performance and cost efficiency.
- Global Databases:
 - Aurora Global Databases enable replication of data across multiple AWS regions. This allows for low-latency access to data for users distributed globally.
- Backups and Snapshots:
 - Aurora automatically takes continuous backups, and users can create manual snapshots. This provides point-in-time recovery options and allows users to restore data to a specific state.
- Security:
 - Aurora supports encryption at rest and in transit. It integrates with AWS Identity and Access Management (IAM) for access control, providing a secure environment for data.
- Compatibility:
 - Aurora is compatible with MySQL and PostgreSQL, allowing organizations to easily migrate existing applications and databases to Aurora with minimal code changes.
- Read Replicas:
 - Aurora supports read replicas, allowing users to offload read traffic from the primary database. This enhances performance for read-intensive workloads.

Benefits:

- Fully Managed:
 - Aurora is a fully managed service, reducing the operational burden on organizations. AWS takes care of routine administrative tasks, allowing users to focus on application development.
- High Performance:
 - Aurora delivers high performance with low-latency read and write operations. Its distributed architecture and parallel processing capabilities optimize query performance.
- Scalability:

- Aurora is highly scalable, supporting both manual and auto-scaling. This allows organizations to scale their database capacity based on changing workloads.
- Global Reach:
 - Aurora Global Databases enable data replication across multiple AWS regions, providing low-latency access for users distributed globally.
- Cost-Efficiency:
 - Aurora's pay-as-you-go pricing model allows organizations to pay only for the resources they consume. The automatic scaling feature ensures cost efficiency by adjusting capacity based on demand.
- Compatibility with MySQL and PostgreSQL:
 - Aurora is compatible with MySQL and PostgreSQL, making it easy for organizations to migrate existing applications and databases to Aurora without significant modifications.
- Security and Compliance:
 - Aurora provides features for securing data, such as encryption, access controls, and auditing capabilities. This helps organizations meet security and compliance requirements.
- High Availability and Reliability:
 - Aurora's architecture ensures high availability and durability of data. Automatic failover and continuous backups contribute to the reliability of the database service.

AWS ElastiCache

Amazon ElastiCache is a fully managed, in-memory caching service provided by Amazon Web Services (AWS). It is designed to improve the performance of web applications by allowing them to retrieve data from a fast, in-memory cache rather than relying on slower, disk-based databases. ElastiCache supports two popular open-source in-memory caching engines: Redis and Memcached.

What it does?

- ElastiCache enhances the performance and scalability of applications by providing a fully managed, in-memory caching layer. It stores frequently accessed data in-memory, reducing the need for repeated queries to the underlying databases. This results in faster response times and improved overall application performance.

Use Cases:

- **Caching Frequently Accessed Data:**
 - ElastiCache is used to store and retrieve frequently accessed data, such as database query results, API responses, and session data.
 - By caching this information in-memory, applications can respond more quickly to user requests.
- **Session Store:**
 - ElastiCache is often employed as a session store to manage user session data in web applications.
 - Storing session data in-memory allows for quick access and retrieval, improving the overall user experience.
- **Real-Time Analytics:**
 - Applications that require real-time analytics and data processing benefit from ElastiCache.
 - By caching frequently accessed datasets, analytics queries can be executed with lower latency.
- **Leaderboards and Counting:**
 - Gaming applications often use ElastiCache to store leaderboards and perform real-time counting operations.
 - This ensures that the information is readily available and can be quickly updated based on user interactions.
- **Pub/Sub Messaging:**
 - With Redis, ElastiCache supports Publish/Subscribe (Pub/Sub) messaging patterns.
 - This is useful for building real-time communication and notification systems where different components of an application can subscribe to and receive updates on specific events.
- **Geospatial Data:**
 - Applications that involve geospatial data, such as location-based services or mapping applications, can use Redis with ElastiCache to efficiently store and query geospatial information.
- **Content Caching:**
 - ElastiCache can be utilized to cache static content, such as images, CSS files, or API responses.
 - This reduces the load on backend servers and accelerates the delivery of content to end-users.

- **Application State Management:**
 - In scenarios where maintaining a shared state across multiple application instances is necessary, ElastiCache can be used to store and manage that shared state, ensuring consistency across the application environment.

Engines:

- **Redis:**
 - Redis is an open-source, in-memory data structure store. It supports various data structures, including strings, sets, lists, and more. Redis in ElastiCache is often chosen for its rich feature set, advanced data structures, and support for Pub/Sub messaging.
- **Memcached:**
 - Memcached is another open-source, in-memory key-value store. It is known for its simplicity and high-performance caching capabilities. Memcached in ElastiCache is suitable for straightforward caching use cases.

Key Features:

- **Managed Service:**
 - ElastiCache is fully managed by AWS, taking care of operational tasks such as hardware provisioning, setup, configuration, monitoring, and maintenance. This allows developers to focus on building applications.
- **Scalability:**
 - ElastiCache is scalable, supporting the addition or removal of nodes to accommodate changing workloads.
 - Both Redis and Memcached in ElastiCache offer horizontal scalability for improved performance.
- **High Availability:**
 - ElastiCache provides high availability through automatic data replication and failover mechanisms.
 - For Redis, Multi-AZ (Availability Zone) deployments are supported, ensuring data redundancy.
- **Security:**
 - ElastiCache supports encryption at rest and in transit. It integrates with Virtual Private Cloud (VPC) for network isolation and allows for secure access through IAM roles and security groups.
- **Monitoring and Logging:**
 - ElastiCache integrates with Amazon CloudWatch for monitoring and provides metrics related to cache performance.
 - It also supports the generation of logs for monitoring and troubleshooting purposes.
- **Automatic Backups:**
 - ElastiCache offers the option to enable automatic backups for Redis clusters. These backups provide point-in-time recovery options, ensuring data durability.
- **Parameter Groups:**
 - ElastiCache allows users to customize and tune various parameters to optimize the performance of Redis or Memcached clusters.
 - Parameter groups can be used to modify settings based on specific application requirements.

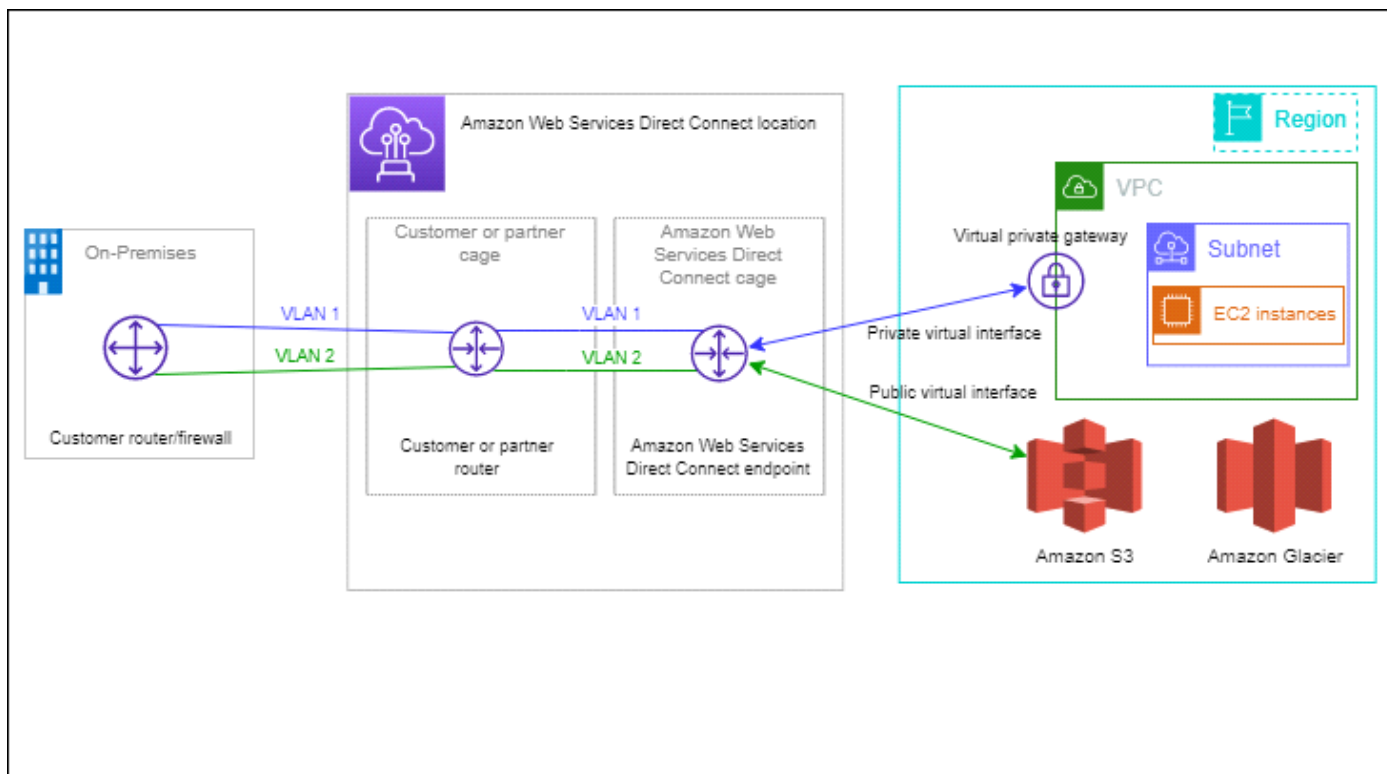
Benefits:

- **Improved Application Performance:**

- By storing frequently accessed data in-memory, ElastiCache significantly improves the performance of applications, reducing the need for repeated database queries.
- **Ease of Management:**
 - ElastiCache is a fully managed service, eliminating the operational overhead of maintaining an in-memory caching layer. AWS takes care of routine tasks, including setup, configuration, and maintenance.
- **Scalability and Flexibility:**
 - ElastiCache is scalable and can handle varying workloads by adding or removing nodes. This flexibility allows organizations to adapt to changing application requirements.
- **Cost-Efficiency:**
 - Caching frequently accessed data in-memory reduces the load on backend databases, leading to cost savings in terms of reduced database query and processing costs.
- **High Availability and Reliability:**
 - ElastiCache ensures high availability through features like automatic failover and data replication. This enhances the reliability of the caching layer.
- **Security and Compliance:**
 - ElastiCache provides security features such as encryption at rest and in transit, IAM integration, and support for VPCs, enabling organizations to meet security and compliance requirements.
- **Compatibility with Popular Engines:**
 - ElastiCache supports both Redis and Memcached, giving organizations the flexibility to choose the caching engine that best fits their specific use case and application requirements.

AWS Direct Connect

AWS Direct Connect is a network service provided by Amazon Web Services (AWS) that enables customers to establish dedicated and private network connections between their on-premises data centers or office locations and AWS. This dedicated network connection bypasses the public internet, providing a more reliable, low-latency, and secure link to AWS cloud services.



What it does?

- AWS Direct Connect establishes a direct physical connection between an organization's network and an AWS Direct Connect location, typically hosted in a colocation facility.
- This connection is then extended over a dedicated network link, which can be a dedicated line or a network service provided by AWS Direct Connect partners.
- This dedicated link provides more predictable and consistent network performance compared to internet-based connections.

Use Cases:

- Hybrid Cloud Deployments:
 - AWS Direct Connect is commonly used in hybrid cloud architectures where organizations maintain a combination of on-premises data centers and resources in the AWS cloud.
 - This allows for seamless integration and data exchange between on-premises infrastructure and AWS services.
- Data Migration:
 - When migrating large volumes of data to AWS, using AWS Direct Connect can significantly reduce the time required for data transfer compared to over-the-internet methods.

- This is particularly beneficial for organizations with high-volume data migration needs.
- Consistent Network Performance:
 - Applications that require consistent and low-latency network performance, such as real-time applications, video streaming, or voice over IP (VoIP), can benefit from AWS Direct Connect.
 - It provides a dedicated and reliable connection for such workloads.
- Private Connectivity to AWS Services:
 - AWS Direct Connect enables private connectivity to various AWS services, including Amazon S3, Amazon EC2, Amazon RDS, and others.
 - This can enhance security and compliance by avoiding data transmission over the public internet.
- Backup and Disaster Recovery:
 - Organizations can use AWS Direct Connect to establish a dedicated connection for backup and disaster recovery scenarios.
 - This ensures reliable and efficient data replication between on-premises environments and AWS.
- High-Performance Computing (HPC):
 - For applications that demand high-performance computing capabilities, such as scientific simulations, modeling, or rendering, AWS Direct Connect provides a dedicated and high-bandwidth connection to support these workloads.
- Improved Security:
 - Industries with stringent security and compliance requirements, such as finance, healthcare, or government, may use AWS Direct Connect to establish a private and dedicated connection to AWS, reducing exposure to potential security risks over the internet.
- Multiple Locations Connectivity:
 - Organizations with a presence in multiple geographic locations can use AWS Direct Connect to establish dedicated connections from each location to AWS.
 - This ensures efficient and reliable connectivity across the entire network.

Key Features:

- Dedicated Network Connection:
 - AWS Direct Connect provides a dedicated network connection, offering more predictable and consistent performance compared to internet-based connections.
- Multiple Speed Options:
 - AWS Direct Connect offers multiple speed options, ranging from 50 Mbps to 100 Gbps, allowing organizations to choose the level of network bandwidth that meets their specific requirements.
- Public and Private Virtual Interfaces:
 - AWS Direct Connect supports the creation of both public and private virtual interfaces.
 - Public interfaces can be used to connect to AWS services accessible over the public internet, while private interfaces are used for accessing services within an Amazon Virtual Private Cloud (VPC).
- Virtual LAN (VLAN) Tagging:
 - VLAN tagging allows organizations to use a single physical connection for multiple logical connections, enabling segmentation of traffic for different use cases or departments.
- Redundancy and High Availability:
 - To enhance reliability, AWS Direct Connect supports redundant connections.

Organizations can establish connections to multiple AWS Direct Connect locations or use the same location for redundant physical connections.

- Direct Connect Gateway:
 - The Direct Connect Gateway allows organizations to connect their virtual private clouds (VPCs) in different AWS regions to a single Direct Connect connection.
 - This simplifies the network architecture for multi-region deployments.
- Global Reach:
 - AWS Direct Connect has a global presence with numerous Direct Connect locations across different regions and cities worldwide.
 - This allows organizations to establish connections close to their on-premises locations.
- AWS Direct Connect Partners:
 - AWS Direct Connect partners provide network services that leverage the AWS Direct Connect infrastructure.
 - These partners offer additional options for connectivity, such as hosted connections, network functions, and expanded reach.

Benefits:

- Predictable and Consistent Performance:
 - By providing a dedicated and private connection, AWS Direct Connect ensures more predictable and consistent network performance compared to internet-based connections.
- Reduced Latency:
 - AWS Direct Connect reduces latency by establishing a direct physical connection, bypassing the public internet.
 - This is crucial for applications that require low-latency communication.
- Enhanced Security:
 - Direct connections between on-premises environments and AWS enhance security by avoiding data transmission over the public internet.
 - This is particularly important for sensitive data and compliance-sensitive workloads.
- Reliability and High Availability:
 - AWS Direct Connect provides redundancy and high availability options, ensuring a reliable connection.
 - This is crucial for critical workloads and applications that require constant connectivity.
- Cost Savings:
 - For organizations with significant data transfer requirements, using AWS Direct Connect can result in cost savings compared to over-the-internet data transfer methods.
- Efficient Data Migration:
 - AWS Direct Connect accelerates data migration by providing a dedicated and high-bandwidth connection between on-premises environments and AWS. This is beneficial for large-scale data transfer projects.
- Simplified Network Architecture:
 - AWS Direct Connect simplifies network architecture by providing a dedicated and direct link between on-premises environments and AWS.
 - This reduces the complexity associated with internet-based connections.
- Global Reach:
 - With a global network of Direct Connect locations, organizations can establish connections close to their on-premises locations, improving connectivity and performance on a global scale.

AWS VPN

Amazon Web Services (AWS) VPN provides a secure and scalable solution for connecting on-premises networks to AWS, enabling secure communication over the internet. AWS offers two types of VPN services: AWS Site-to-Site VPN and AWS Client VPN. These services cater to different use cases and scenarios where secure connectivity between on-premises networks and AWS resources is essential.

Use Cases:

- **Hybrid Cloud Deployments:**
 - AWS VPN is commonly used in hybrid cloud architectures where organizations maintain a combination of on-premises infrastructure and resources in the AWS cloud.
 - It allows secure and encrypted communication between the on-premises data center and AWS.
- **Remote Office Connectivity:**
 - Organizations with remote offices or branch locations can use AWS VPN to establish secure connections to AWS resources.
 - This is particularly useful for accessing applications, databases, and other services hosted in the AWS cloud.
- **Data Center Migration:**
 - During data center migration to AWS, organizations can use AWS VPN to maintain connectivity between the on-premises environment and AWS resources.
 - This ensures a smooth transition without disrupting business operations.
- **Backup and Disaster Recovery:**
 - AWS VPN is employed for backup and disaster recovery scenarios where secure communication between on-premises data centers and AWS is crucial for replicating data and ensuring business continuity.
- **Secure Access to AWS Resources:**
 - AWS VPN provides a secure way for remote users, partners, or third-party vendors to access AWS resources.
 - AWS Client VPN is specifically designed for secure remote access scenarios.
- **Development and Testing Environments:**
 - Development and testing environments hosted in AWS may need secure connectivity to on-premises systems for data exchange or integration testing. AWS VPN facilitates this connectivity securely.
- **Multi-Region Deployments:**
 - Organizations with a global presence and resources deployed in multiple AWS regions can use AWS VPN to establish secure connections between on-premises locations and AWS resources distributed across different regions.
- **Compliance and Security Requirements:**
 - Industries with strict compliance and security requirements, such as finance, healthcare, or government, use AWS VPN to ensure secure communication and adherence to regulatory standards.

AWS Site-to-Site VPN:

- **Secure Communication:**
 - AWS Site-to-Site VPN allows organizations to establish secure communication channels between their on-premises data center or network appliance and the virtual private cloud (VPC) in AWS.

- IPsec Encryption:
 - Site-to-Site VPN uses IPsec (Internet Protocol Security) to encrypt traffic between on-premises and AWS, ensuring the confidentiality and integrity of data during transmission.
- BGP (Border Gateway Protocol) Support:
 - BGP support in AWS Site-to-Site VPN enables dynamic routing between on-premises and AWS, allowing for efficient routing updates and automatic failover.
- High Availability:
 - AWS Site-to-Site VPN supports high availability configurations, including multiple VPN connections to a single virtual private gateway (VGW) or redundancy with multiple VGWs.
- Flexible Bandwidth Options:
 - Organizations can choose the appropriate bandwidth options for their Site-to-Site VPN connections, ranging from 1.25 Mbps up to 1 Gbps, based on their network requirements.

AWS Client VPN:

- Secure Remote Access:
 - AWS Client VPN provides a secure remote access solution, allowing users to connect to their AWS resources securely over the internet.
 - It is suitable for remote employees, partners, or third-party vendors.
- OpenVPN and TLS Support:
 - AWS Client VPN is built on OpenVPN, a widely used open-source VPN protocol, and supports TLS (Transport Layer Security) for secure communication between clients and the AWS VPN endpoint.
- Authentication Integration:
 - AWS Client VPN integrates with AWS Directory Service, Microsoft Active Directory, or other identity providers, allowing organizations to manage user authentication and access control.
- Split Tunneling:
 - AWS Client VPN supports split tunneling, allowing users to access both AWS resources and the internet simultaneously while connected to the VPN.
 - This optimizes bandwidth usage.
- Managed Certificates:
 - AWS Client VPN provides managed certificates, simplifying the process of certificate management for secure communication between clients and the VPN endpoint.

Key Features:

- Encryption and Security:
 - AWS VPN employs strong encryption and security protocols (IPsec for Site-to-Site VPN and OpenVPN/TLS for Client VPN) to ensure the confidentiality and integrity of data during transmission.
- Scalability:
 - AWS VPN services are scalable, allowing organizations to adapt their VPN configurations to accommodate changing network requirements, bandwidth needs, and user access.
- Compatibility:
 - AWS VPN is compatible with various VPN devices and client software, providing flexibility for organizations to choose the VPN solution that best fits their infrastructure and user requirements.
- Integration with AWS Services:

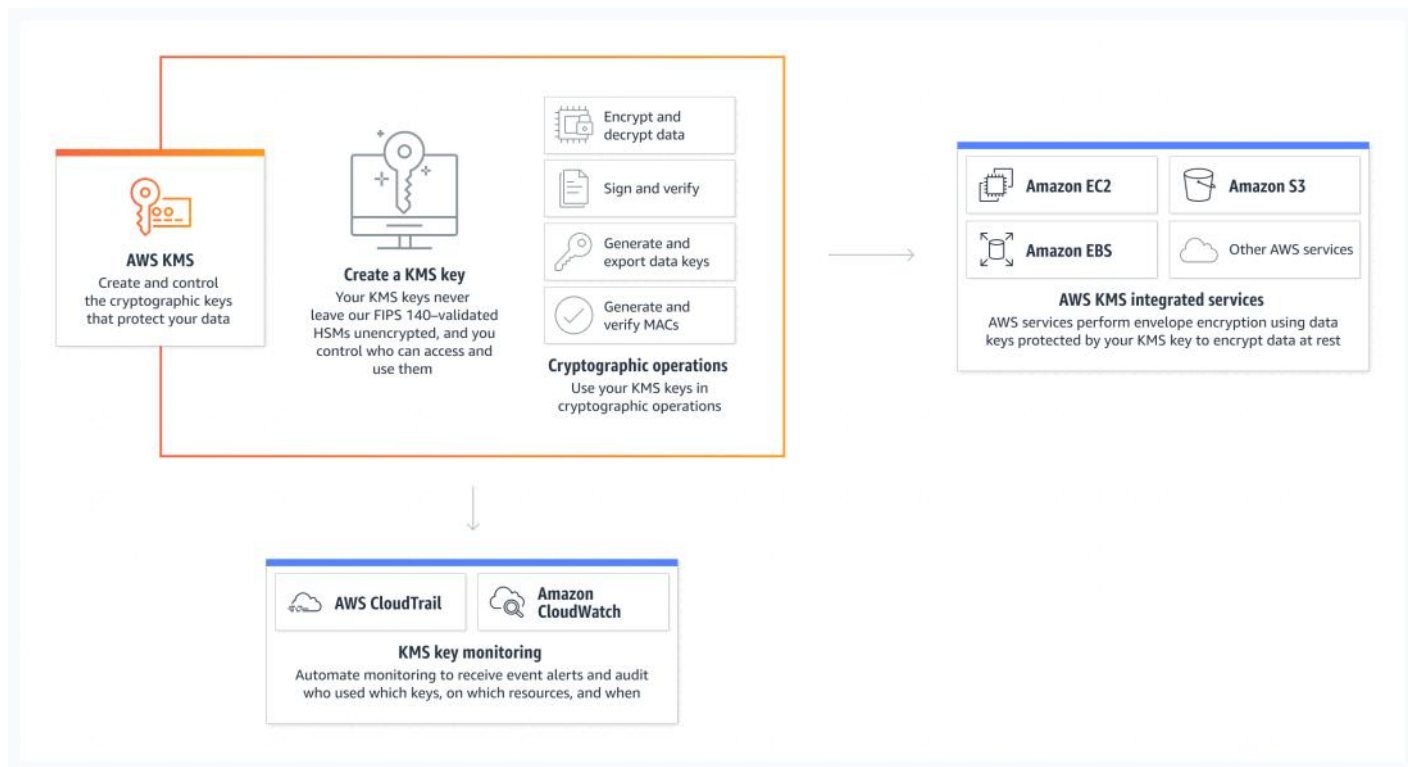
- AWS VPN seamlessly integrates with other AWS services, enabling secure communication between on-premises environments and AWS resources, such as Amazon EC2 instances, RDS databases, and more.
- Monitoring and Logging:
 - AWS VPN services integrate with Amazon CloudWatch for monitoring and logging, providing insights into VPN connection status, performance metrics, and diagnostic information.
- Ease of Management:
 - AWS VPN services are fully managed, reducing the operational burden on organizations. AWS takes care of routine maintenance tasks, updates, and scaling requirements.

Benefits:

- Secure Connectivity:
 - AWS VPN provides secure and encrypted connectivity between on-premises environments and AWS resources, ensuring the confidentiality and integrity of data.
- Flexibility and Scalability:
 - AWS VPN services are flexible and scalable, allowing organizations to adapt their VPN configurations based on changing requirements, such as network growth, bandwidth needs, or user access.
- Reduced Latency:
 - By establishing direct and dedicated connections, AWS VPN reduces latency compared to internet-based connections, ensuring optimal performance for applications and services.
- Remote Access Solution:
 - AWS Client VPN offers a secure remote access solution, allowing remote users to securely connect to AWS resources from anywhere, ensuring productivity and access control.
- Global Reach:
 - With AWS VPN services available in various regions, organizations with a global presence can establish secure connections close to their on-premises locations, ensuring efficient and reliable connectivity.
- Managed Service:
 - AWS VPN services are fully managed, reducing the operational overhead for organizations. AWS takes care of routine tasks, ensuring high availability, security, and reliability of VPN connections.

AWS Key Management Service (KMS)

- AWS Key Management Service (KMS) is a fully managed service that allows users to create, control, and manage cryptographic keys used for encrypting data at rest in AWS services and in their applications.
- KMS provides a secure and scalable solution for key management, making it easier for organizations to implement strong encryption practices and enhance the security of their data.



What it does?

AWS KMS helps users manage cryptographic keys securely, allowing them to create, rotate, disable, and delete keys as needed. It integrates seamlessly with various AWS services, enabling users to encrypt and decrypt data using these keys. KMS supports both symmetric and asymmetric keys and provides a central location for managing key access and permissions.

Use Cases:

- **Data Encryption for AWS Services:**
 - KMS is used to encrypt data at rest in various AWS services, such as Amazon S3, Amazon EBS, Amazon RDS, and more.
 - By default, many AWS services integrate with KMS for key management, providing a straightforward way to enhance data security.
- **Secure Key Storage:**
 - Organizations use KMS as a secure key storage solution. It provides a centralized location for managing keys, reducing the complexity of key management and ensuring the security of cryptographic keys.
- **Application-Level Encryption:**
 - Developers use KMS to implement application-level encryption, encrypting sensitive data before storing it in databases, file systems, or other storage solutions. KMS allows for easy integration into custom applications.
- **Key Rotation:**
 - KMS supports automatic key rotation, allowing organizations to regularly update

- cryptographic keys.
- This enhances security by reducing the risk associated with long-lived keys and improving compliance with security best practices.
- Multi-Tenant Applications:
 - In scenarios where multiple tenants or customers share the same infrastructure, KMS provides a way to isolate and manage encryption keys for each tenant securely.
 - This ensures data confidentiality and separation between tenants.
- Digital Rights Management (DRM):
 - KMS can be used in media and entertainment applications for implementing digital rights management.
 - It provides the necessary infrastructure to manage encryption keys securely, protecting copyrighted content.
- Cross-Account Key Sharing:
 - Organizations with multiple AWS accounts can use KMS for cross-account key sharing. This allows one AWS account to create and manage keys that are used by resources in another AWS account, facilitating secure data sharing.
- Compliance Requirements:
 - Industries with strict compliance requirements, such as healthcare (HIPAA), finance (PCI DSS), or government (FISMA), use KMS to meet encryption and key management standards, ensuring the confidentiality and integrity of sensitive data.

Key Features:

- Key Creation and Deletion:
 - KMS allows users to create and delete cryptographic keys. Key creation involves specifying key material or letting KMS generate it. Key deletion is a two-step process to prevent accidental key deletion.
- Key Rotation:
 - KMS supports automatic key rotation, allowing users to configure a rotation schedule for their keys. This enhances security by regularly updating keys without manual intervention.
- Symmetric and Asymmetric Keys:
 - KMS supports both symmetric and asymmetric key types. Symmetric keys are used for encryption and decryption, while asymmetric keys are used for digital signatures and key management operations.
- Integration with AWS Services:
 - KMS integrates seamlessly with various AWS services, providing a central key management solution for encrypting data at rest. Services like S3, EBS, and RDS can leverage KMS for key management.
- Key Policies and Permissions:
 - KMS allows users to set key policies to define who can use and manage keys. Key policies are JSON documents that specify permissions for AWS Identity and Access Management (IAM) users and roles.
- Audit Trails:
 - KMS provides audit trails through AWS CloudTrail, enabling users to track key usage, management operations, and changes to key policies. This supports compliance and security monitoring.
- Custom Key Stores:
 - KMS offers the option to use custom key stores, allowing users to create and control their own key management infrastructure. This is useful for organizations with specific security and compliance requirements.
- Tagging:
 - Users can tag their KMS keys with metadata to categorize and organize keys based on their use case or ownership. Tags help in resource management and tracking.

Benefits:

- **Simplified Key Management:**
 - KMS provides a centralized and fully managed solution for key management, simplifying the complexities associated with cryptographic key generation, storage, and rotation.
- **Integration with AWS Services:**
 - KMS seamlessly integrates with various AWS services, making it easy for users to implement encryption in their applications and data storage solutions without managing the underlying infrastructure.
- **Security Best Practices:**
 - By using KMS, organizations can adhere to security best practices, including regular key rotation, secure key storage, and enforcing access controls through key policies, thereby enhancing data security.
- **Automatic Key Rotation:**
 - KMS supports automatic key rotation, ensuring that cryptographic keys are regularly updated. This reduces the risk associated with long-lived keys and enhances overall security.
- **Scalability:**
 - KMS scales with the needs of the organization, allowing users to create and manage a large number of keys as their applications and data storage requirements grow.
- **Auditing and Compliance:**
 - KMS provides audit trails through CloudTrail, enabling organizations to monitor key usage, changes, and management operations. This supports compliance with regulatory requirements and internal security policies.
- **Custom Key Stores:**
 - For organizations with specific security and compliance requirements, KMS offers custom key stores, allowing users to control their own key management infrastructure while still benefiting from KMS features.
- **Cost-Effective:**
 - KMS follows a pay-as-you-go pricing model, where users are charged based on the number of key requests and key operations. This allows organizations to manage costs effectively based on their usage.

AWS CloudHSM

- Amazon Web Services (AWS) CloudHSM (Hardware Security Module) is a cloud-based hardware security module service that provides dedicated hardware-based cryptographic key storage and operations within the AWS Cloud.
- CloudHSM offers a secure and tamper-resistant environment for managing cryptographic keys used for data encryption, digital signatures, and other sensitive operations.

What it does?

AWS CloudHSM provides secure and high-performance hardware-based key storage and cryptographic operations. It is designed to help organizations meet compliance requirements and enhance the security of sensitive data by allowing them to manage their own cryptographic keys in a dedicated hardware security module.

Use Cases:

- Key Management for Sensitive Data:
 - CloudHSM is often used to manage cryptographic keys for sensitive data such as customer information, financial records, or intellectual property.
 - By keeping keys within a dedicated hardware module, organizations can enhance the security of their critical assets.
- Data Encryption for Compliance:
 - Organizations subject to regulatory requirements, such as those in the finance or healthcare industries, use CloudHSM to meet encryption and key management compliance standards.
 - CloudHSM helps organizations control and protect access to sensitive information.
- Secure Communication:
 - CloudHSM is employed in scenarios where secure communication channels, such as SSL/TLS, are required.
 - By providing a dedicated hardware module for cryptographic operations, CloudHSM enhances the security of encrypted communication.
- Digital Signatures:
 - Applications that require digital signatures for data integrity and authenticity, such as secure document signing or certificate issuance, can leverage
 - CloudHSM for secure key storage and cryptographic operations.
- Secure Key Storage and Retrieval:
 - CloudHSM is used for securely storing and retrieving cryptographic keys used by applications and services.
 - The dedicated hardware module provides a tamper-resistant environment, reducing the risk of key compromise.
- Payment Processing Security:
 - In the financial industry, CloudHSM can be utilized for securing payment processing systems.
 - The dedicated hardware security module helps protect cryptographic keys involved in payment transactions.
- Public Key Infrastructure (PKI):
 - Organizations implementing PKI for digital certificates, identity management, or secure communication can benefit from CloudHSM to secure and manage the private keys associated with their certificate authorities.

- Securing Cryptographic Operations in Cloud Environments:
 - CloudHSM addresses security concerns related to cryptographic operations in cloud environments.
 - By providing dedicated hardware in the cloud, organizations can maintain control over their cryptographic keys.

Key Features:

- Dedicated Hardware Security Module:
 - CloudHSM provides a dedicated hardware security module for each customer, ensuring isolation and security for cryptographic keys.
 - This module is tamper-resistant and FIPS 140-2 Level 3 compliant.
- Customizable Access Control Policies:
 - Organizations can define custom access control policies to control and restrict access to CloudHSM resources.
 - This includes policies for key usage, deletion, and administrative actions.
- High Availability and Redundancy:
 - CloudHSM supports high availability configurations with multiple HSMs in a cluster.
 - This provides redundancy and ensures continuous access to cryptographic keys, even in the event of a hardware failure.
- Integration with AWS Services:
 - CloudHSM integrates with various AWS services, allowing organizations to use CloudHSM keys for encryption in services like Amazon RDS, Amazon S3, and other AWS resources that support external key management.
- Cryptographic Algorithms and Operations:
 - CloudHSM supports a range of cryptographic algorithms and operations, including symmetric and asymmetric encryption, digital signatures, key generation, and more.
 - This flexibility accommodates diverse use cases.
- CloudWatch Metrics and Monitoring:
 - CloudHSM provides CloudWatch metrics for monitoring the performance and usage of the hardware security module.
 - Organizations can set up alarms and use CloudWatch logs for auditing and troubleshooting.
- Key Backup and Restore:
 - CloudHSM supports key backup and restore operations, allowing organizations to create backups of their keys for recovery purposes.
 - This helps prevent data loss in case of accidental key deletion.

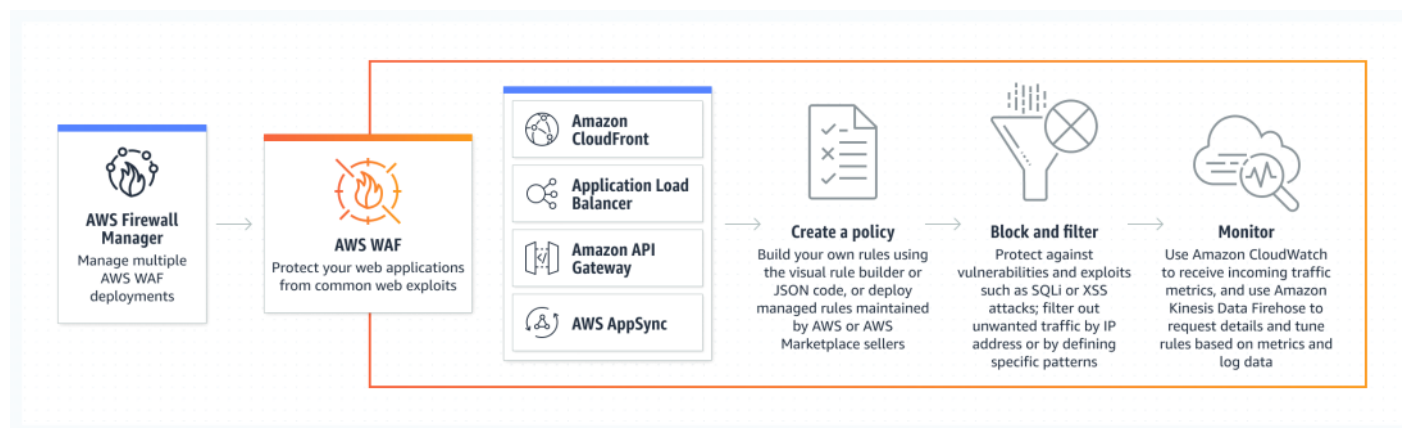
Benefits:

- Enhanced Security:
 - CloudHSM provides a dedicated hardware security module, enhancing the security of cryptographic keys by isolating them from other resources in the AWS Cloud.
- Compliance:
 - CloudHSM helps organizations meet compliance requirements by providing a FIPS 140-2 Level 3 compliant hardware security module. This is particularly important for industries with strict regulatory standards.
- Customizable Access Control:
 - Organizations can define and enforce custom access control policies, ensuring that only authorized users and applications have access to cryptographic keys stored in CloudHSM.

- High Availability:
 - CloudHSM supports high availability configurations, providing redundancy and ensuring continuous access to cryptographic keys even in the presence of hardware failures.
- Integration with AWS Services:
 - CloudHSM seamlessly integrates with various AWS services, allowing organizations to use their keys for encrypting data in other AWS resources, promoting consistency and security.
- Flexible Cryptographic Operations:
 - CloudHSM supports a wide range of cryptographic algorithms and operations, providing flexibility to accommodate different encryption and key management requirements.
- Secure Key Storage and Retrieval:
 - CloudHSM offers secure key storage and retrieval, reducing the risk of key compromise. Organizations can trust the dedicated hardware module to protect their cryptographic keys.
- Scalability:
 - CloudHSM is scalable, allowing organizations to add or remove HSMs based on their cryptographic key management needs. This ensures that the service can adapt to changing requirements and workloads.

AWS Web Application Firewall (WAF)

- AWS WAF is a web application firewall service provided by Amazon Web Services (AWS).
- It helps protect web applications from common web exploits and malicious attacks by allowing customers to configure rules and policies to filter and monitor HTTP and HTTPS traffic.
- AWS WAF can be deployed in front of web applications hosted on AWS or on-premises.



What it does:

- AWS WAF allows users to define rules and conditions to filter, monitor, and protect web applications from various web security threats.
- It inspects web requests and applies rules to allow, block, or monitor traffic based on defined criteria.
- This proactive security approach helps mitigate risks associated with common web vulnerabilities.

Use Cases:

- **Protection Against OWASP Top 10 Threats:**
 - AWS WAF can be configured to protect web applications against the OWASP (Open Web Application Security Project) Top 10 threats, including SQL injection, cross-site scripting (XSS), and other common vulnerabilities.
- **Mitigation of DDoS Attacks:**
 - WAF helps in mitigating Distributed Denial of Service (DDoS) attacks by allowing users to set rate-based rules to limit the number of requests from a single IP address.
 - This helps prevent overwhelming the web application with excessive traffic.
- **Bot and Scraping Protection:**
 - WAF can be used to identify and block malicious bots and scraping activities.
 - By setting rules to detect and filter bot traffic, organizations can protect their web applications from automated threats.
- **Geographic Access Control:**
 - Organizations can use WAF to implement geographic access control policies.
 - For example, they can block traffic from specific countries or regions to reduce the risk of attacks from known malicious sources.
- **API Security:**
 - For web applications with APIs, WAF can be used to secure and protect API endpoints.
 - It helps prevent common API-related vulnerabilities and ensures the integrity

and availability of API services.

- Protection of Sensitive Data:
 - WAF allows organizations to implement rules to identify and block attempts to access sensitive data, such as credit card information or personally identifiable information (PII), preventing data breaches.
- Custom Rule Sets:
 - Organizations can create custom rule sets tailored to their specific application and security requirements.
 - This flexibility enables them to address unique threats and vulnerabilities in their web applications.
- Integration with CloudFront and Application Load Balancers:
 - AWS WAF seamlessly integrates with Amazon CloudFront and Application Load Balancers (ALBs).
 - This integration allows users to deploy WAF at the edge locations, providing low-latency and distributed protection for web applications.
- Compliance and Regulatory Requirements:
 - WAF can be used to enforce compliance with regulatory standards by implementing security controls that align with industry-specific requirements.
 - This is crucial for organizations in sectors like finance, healthcare, and government.

Key Features:

- Web ACLs (Access Control Lists):
 - AWS WAF uses Web ACLs to define rules and conditions for filtering web traffic.
 - Web ACLs enable users to specify the ruleset to be applied to incoming requests and responses.
- Rule Groups:
 - Rule Groups are collections of rules that address specific types of threats or vulnerabilities.
 - AWS provides managed rule groups, and users can also create custom rule groups to suit their application's security needs.
- Managed Rules:
 - AWS offers managed rule sets that include pre-configured rules for common threats, such as SQL injection and cross-site scripting.
 - Users can use these rules as a starting point and customize them as needed.
- Rate-Based Rules:
 - Rate-based rules allow users to set thresholds for the number of requests from a single IP address within a specified time period.
 - This helps protect against DDoS attacks and other forms of abuse.
- IP Reputation Lists:
 - WAF can use IP reputation lists to block traffic from known malicious IP addresses.
 - AWS provides managed threat intelligence feeds, and users can also integrate custom IP reputation lists.
- Logging and Monitoring:
 - WAF provides detailed logging of web requests and actions taken based on defined rules.
 - Users can use Amazon CloudWatch to monitor WAF metrics and set up alarms for suspicious activity.
- Integration with AWS Services:
 - AWS WAF integrates seamlessly with other AWS services, including CloudFront, ALBs, and AWS Shield.
 - This allows users to deploy WAF across their infrastructure for comprehensive web application protection.

Benefits:

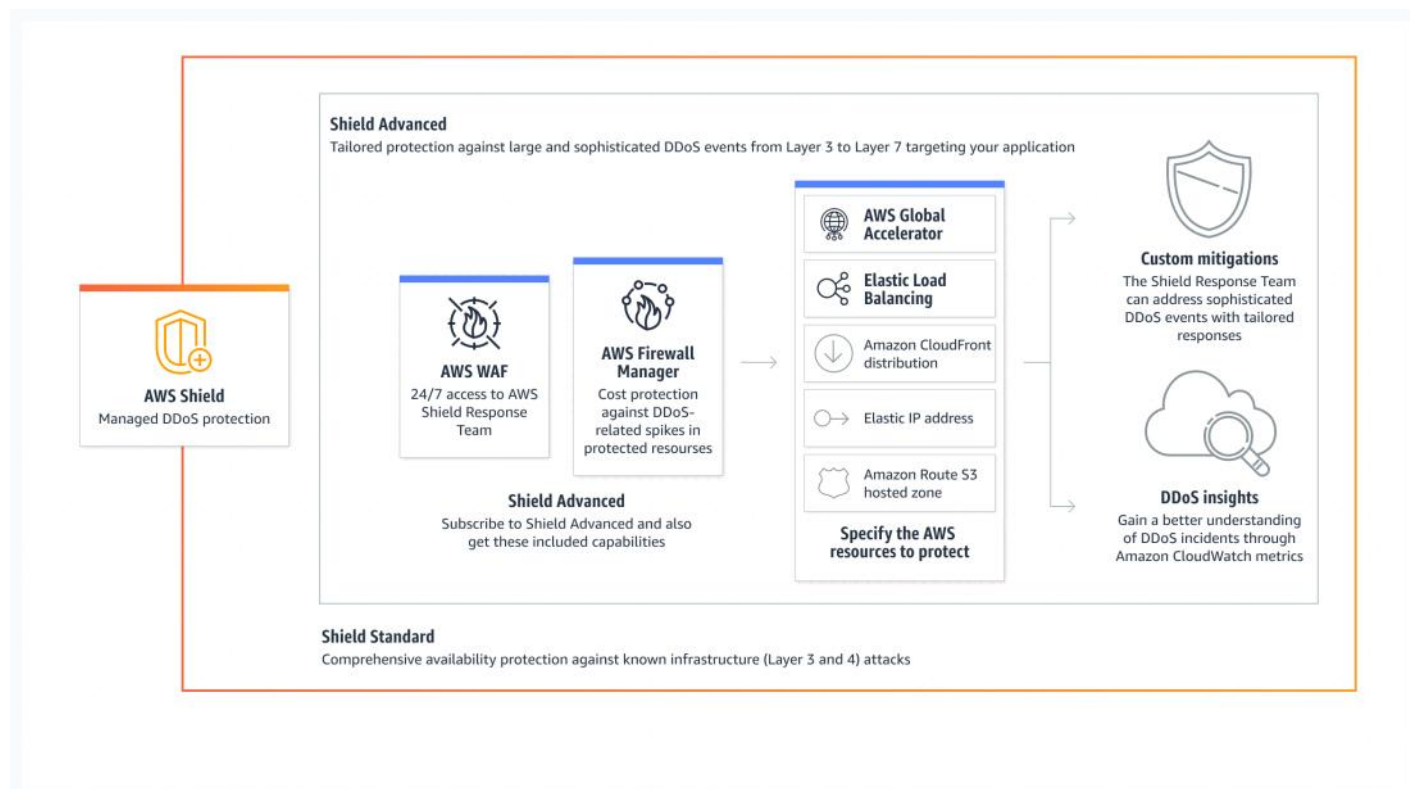
- Protection Against Web Exploits:
 - AWS WAF offers protection against common web exploits and vulnerabilities, helping organizations safeguard their web applications from malicious attacks.
- Customizable Security Policies:
 - Organizations can create custom security policies based on their unique requirements. This flexibility allows for tailored protection against specific threats relevant to the application.
- Scalable and Distributed Protection:
 - With integration with CloudFront and ALBs, AWS WAF provides scalable and distributed protection, allowing users to deploy web application firewall rules globally at the edge locations.
- Real-Time Monitoring and Logging:
 - WAF provides real-time monitoring and logging capabilities, allowing organizations to analyze and respond to security events promptly. CloudWatch integration enhances visibility into web traffic patterns.
- Automated Threat Intelligence:
 - Managed rules and IP reputation lists provided by AWS offer automated threat intelligence, helping organizations stay up-to-date with the latest security threats and vulnerabilities.
- Ease of Integration:
 - AWS WAF seamlessly integrates with other AWS services, making it easy for organizations to incorporate web application security into their existing AWS infrastructure.
- Cost-Effective:
 - AWS WAF follows a pay-as-you-go pricing model, allowing organizations to pay for the specific resources and rules they use. This cost-effective approach aligns with the scale and needs of web applications.

AWS Shield

AWS Shield is a managed Distributed Denial of Service (DDoS) protection service provided by Amazon Web Services (AWS). It is designed to safeguard web applications and websites from the impact of DDoS attacks, ensuring the availability and performance of applications even in the face of malicious attempts to disrupt service.

What it does?

- AWS Shield detects and mitigates DDoS attacks in real time, protecting applications and websites hosted on AWS from volumetric, state-exhaustion, and application layer attacks.
- By utilizing a combination of automated detection, machine learning, and global threat intelligence, AWS Shield provides a robust defense against a wide range of DDoS threats.



Use Cases:

- **Availability Protection:**
 - AWS Shield is commonly used to ensure the availability of web applications and websites by mitigating DDoS attacks.
 - This is crucial for organizations that rely on uninterrupted online services to maintain customer trust and business continuity.
- **DDoS Attack Mitigation:**
 - Organizations facing DDoS attacks, whether they are volumetric, network-layer, or application-layer attacks, can leverage AWS Shield to detect and automatically mitigate these attacks.
 - This proactive defense helps prevent service disruption.
- **Protection Against Application Layer Attacks:**
 - AWS Shield provides protection against sophisticated application layer attacks, including HTTP floods and other targeted attacks that aim to exploit vulnerabilities in web applications.
- **Securing E-commerce Platforms:**
 - E-commerce platforms handling transactions and sensitive customer data are prime targets for DDoS attacks.

- AWS Shield helps secure these platforms, ensuring that customers can access and transact on the site without interruption.
- Financial Services Security:
 - Financial services organizations, such as banks and payment processors, use AWS Shield to protect against DDoS attacks that could disrupt online banking services, financial transactions, and other critical operations.
- Media and Entertainment:
 - Websites and applications in the media and entertainment industry, especially those streaming live events or hosting high-traffic content, use AWS Shield to ensure continuous availability during peak periods and protect against targeted attacks.
- Online Gaming Protection:
 - Online gaming platforms face DDoS attacks that can disrupt gameplay and impact the user experience.
 - AWS Shield helps gaming companies maintain a stable and secure gaming environment.
- Protection for Public Sector Organizations:
 - Public sector organizations, including government agencies and educational institutions, use AWS Shield to protect their online services and ensure the availability of essential resources.

Key Features:

- Global Threat Environment Monitoring:
 - AWS Shield continuously monitors the global threat environment to detect and respond to emerging DDoS threats.
 - This includes analyzing traffic patterns, identifying malicious behavior, and adapting mitigation strategies.
- Automatic DDoS Attack Mitigation:
 - AWS Shield provides automated DDoS attack mitigation, dynamically adjusting mitigation strategies based on real-time traffic patterns.
 - This ensures that legitimate traffic is allowed while malicious traffic is blocked.
- Layer 3, 4, and 7 Protection:
 - AWS Shield offers protection at multiple layers of the OSI model, including network and application layers. This comprehensive approach safeguards against a variety of DDoS attack vectors.
- Managed Rules and Policies:
 - AWS Shield includes managed DDoS protection rules and policies that are updated based on evolving threat intelligence.
 - Users can also create custom rules to tailor protection to specific application needs.
- Advanced Threat Intelligence:
 - AWS Shield leverages advanced threat intelligence from AWS and external sources to identify and mitigate DDoS attacks proactively.
 - This intelligence helps improve the accuracy and effectiveness of attack detection.
- Rate-Based Mitigation:
 - AWS Shield employs rate-based mitigation to identify and block DDoS attacks based on unusual request rates.
 - This helps protect against volumetric attacks that attempt to overwhelm resources.
- Application Layer DDoS Protection:
 - Protection against application layer attacks is a key feature of AWS Shield. It helps prevent attacks targeting the application layer, such as SQL injection, cross-site scripting, and other sophisticated threats.
- Distributed Anycast Network:
 - AWS Shield operates on a global distributed Anycast network, allowing it to efficiently absorb and mitigate DDoS attacks close to the source.
 - This distributed infrastructure enhances the scalability and responsiveness of the

service.

Benefits:

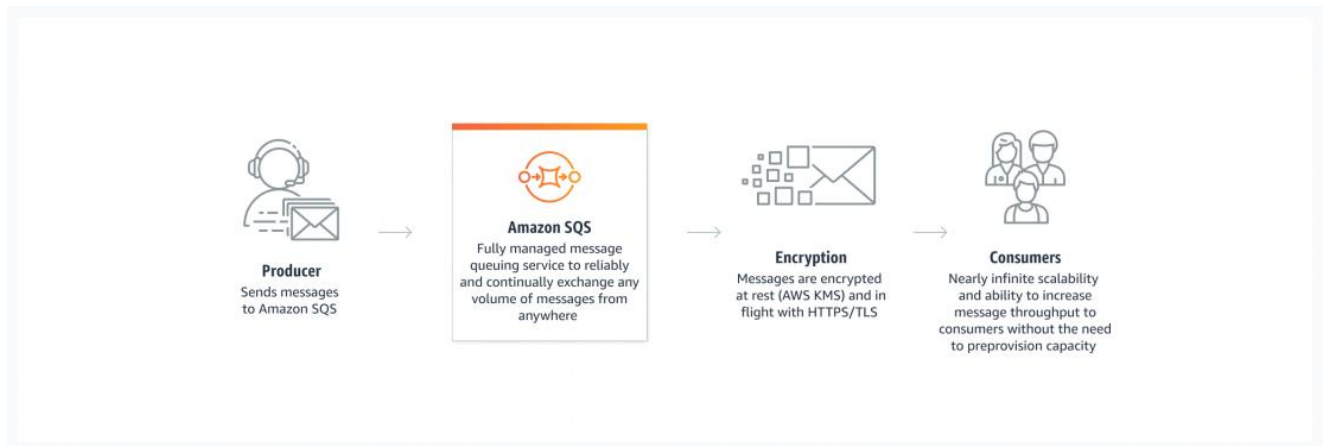
- **Continuous Availability:**
 - AWS Shield helps ensure the continuous availability of web applications and websites by mitigating DDoS attacks in real time.
 - This is crucial for maintaining business operations and user trust.
- **Automated Attack Detection and Mitigation:**
 - The automated nature of AWS Shield allows for rapid detection and mitigation of DDoS attacks without manual intervention.
 - This ensures a swift response to evolving threats.
- **Global Threat Intelligence:**
 - By leveraging global threat intelligence, AWS Shield stays ahead of emerging DDoS threats, providing users with proactive protection against the latest attack vectors and techniques.
- **Comprehensive DDoS Protection:**
 - AWS Shield offers protection against a wide range of DDoS attack vectors, including volumetric attacks, network-layer attacks, and application-layer attacks.
 - This comprehensive defense strategy safeguards against various threat scenarios.
- **Scalable Infrastructure:**
 - The distributed Anycast network architecture of AWS Shield ensures scalability, allowing it to absorb and mitigate DDoS attacks efficiently, even in the face of large-scale attacks.
- **Flexible Configuration:**
 - Users can configure AWS Shield to meet the specific needs of their applications, including the ability to create custom rules and policies.
 - This flexibility ensures that protection is tailored to the unique requirements of each environment.
- **Integration with AWS Services:**
 - AWS Shield seamlessly integrates with other AWS services, such as CloudFront and Route 53, allowing users to extend DDoS protection across their entire infrastructure.
- **Cost-Effective:**
 - AWS Shield follows a pay-as-you-go pricing model, allowing organizations to pay for the specific protection resources they use.
 - This cost-effective approach aligns with the scale and needs of applications facing DDoS threats.

Amazon Simple Queue Service (SQS)

- Amazon Simple Queue Service (SQS) is a fully managed message queuing service provided by Amazon Web Services (AWS).
- SQS enables decoupling of components in distributed systems by allowing one component to send messages to a queue, and another component to retrieve and process those messages asynchronously.
- This messaging system helps in building scalable, fault-tolerant, and loosely coupled architectures.

What it does?

- SQS provides a reliable and scalable platform for sending, storing, and receiving messages between software components.
- It acts as a buffer that decouples the components of a distributed system, allowing them to operate independently, improving resilience and scalability.



Use Cases:

Asynchronous Communication:

SQS facilitates asynchronous communication between microservices or distributed components. One component can send a message to a queue, and other components can consume and process those messages independently, allowing for efficient and loosely coupled interactions.

Load Leveling:

SQS helps in load leveling by acting as a buffer between components that produce messages and those that consume them. This prevents spikes in load from affecting the entire system and allows components to operate at their own pace.

Batch Processing:

Batch processing applications benefit from SQS by allowing them to process a large number of messages efficiently. Messages are queued up, and consumers can process them in batches, optimizing resource utilization and reducing processing times.

Event-Driven Architecture:

SQS is well-suited for event-driven architectures where components respond to events by producing or consuming messages. Events can be generated by user actions, system events, or external triggers, and SQS provides a reliable way to handle these events asynchronously.

Job Queues:

SQS is commonly used as a job queue for distributing and processing workloads. Jobs are submitted as messages to the queue, and worker components can pull and process these jobs independently, providing a scalable solution for background processing.

Decoupling of Services:

When building complex systems with multiple services, SQS helps in decoupling these services. Each service can communicate with others by sending messages to queues, allowing for better isolation, fault tolerance, and scalability.

Workflow Orchestration:

SQS is used in workflow orchestration systems where different stages of a workflow are handled by separate components. Messages in the queue represent tasks or workflow steps, and each component processes its designated part of the workflow.

Retry Mechanism:

SQS can be configured to implement a retry mechanism for failed messages. If a component fails to process a message, SQS can automatically redeliver the message after a specified delay, providing a built-in mechanism for handling failures.

Key Features:

- **Message Queues:**
 - SQS allows users to create message queues that act as buffers for storing messages.
 - Each message in the queue can be up to 256 KB in size.
- **Standard and FIFO Queues:**
 - SQS offers two types of queues: Standard Queues and FIFO (First-In-First-Out) Queues.
 - Standard Queues provide high throughput with best-effort ordering, while FIFO Queues provide exactly-once processing and maintain the order in which messages are sent and received.
- **Visibility Timeout:**
 - SQS provides a visibility timeout for messages. Once a message is retrieved by a consumer, it becomes temporarily invisible to other consumers for a specified duration.
 - If the consumer fails to process the message within the visibility timeout, the message becomes visible again for other consumers to retrieve.
- **Dead Letter Queues:**
 - SQS supports Dead Letter Queues (DLQs) where messages that cannot be processed successfully after a certain number of retries are moved.
 - This helps in identifying and handling messages that consistently fail processing.
- **Message Retention Period:**
 - Users can configure the retention period for messages in the queue.
 - After this period, messages are automatically deleted from the queue, whether they have been processed or not.
- **Delay Queues:**
 - SQS allows users to set a delay on messages before they become available for processing.
 - This feature is useful when messages should not be processed immediately, allowing time for changes or corrections.
- **Server-Side Encryption:**
 - SQS supports server-side encryption for messages stored in queues.
 - This helps in securing sensitive data as it moves through the messaging system.
- **Message Attributes:**
 - Users can attach custom metadata to messages in the form of message attributes.
 - These attributes can be used to provide additional information about the message or control its processing.

Benefits:

- Scalability:
 - SQS is designed to be highly scalable. It can handle a large number of messages and scale horizontally to accommodate varying workloads and messaging patterns.
- Reliability:
 - SQS provides a reliable and durable messaging service. Messages are stored redundantly across multiple servers, and the service is designed to withstand failures and ensure message delivery.
- Loose Coupling:
 - SQS promotes loose coupling between components in a system. Producers and consumers of messages are independent, allowing for flexibility and easier maintenance of distributed systems.
- Fault Tolerance:
 - The distributed nature of SQS and its redundant storage ensure fault tolerance. Even if some components or servers fail, messages are not lost, and the system can recover.
- Efficient Resource Utilization:
 - SQS allows for efficient resource utilization by providing load leveling and decoupling of components. This ensures that components operate at their optimal pace, preventing resource bottlenecks.
- Cost-Effective:
 - SQS follows a pay-as-you-go pricing model, where users are charged based on the number of requests and the amount of data transferred. This allows organizations to pay for the resources they use, making it a cost-effective solution.
- Easy Integration:
 - SQS seamlessly integrates with other AWS services, making it easy to incorporate messaging capabilities into various applications and architectures within the AWS ecosystem.
- Managed Service:
 - As a fully managed service, SQS handles operational tasks such as server provisioning, monitoring, and maintenance. This allows users to focus on building and scaling their applications rather than managing the underlying messaging infrastructure.

AWS Simple Notification Service (SNS)

- Amazon Simple Notification Service (SNS) is a fully managed pub/sub (publish/subscribe) messaging service provided by Amazon Web Services (AWS).
- SNS enables the easy and flexible distribution of messages or notifications to a large number of subscribers or endpoints, such as applications, web services, and distributed systems.
- It allows developers to decouple components of a system and send messages asynchronously.

What it does?

- SNS simplifies the process of sending messages or notifications to a distributed set of subscribers or endpoints.
- It operates on a publish/subscribe model, where publishers (message senders) send messages to topics, and subscribers (message receivers) subscribe to these topics to receive the messages.
- SNS handles the routing, delivery, and fan-out of messages to multiple subscribers.



Use Cases:

- **Event Notification Systems:**
 - SNS is often used to build event-driven architectures, where various components or services react to events by subscribing to SNS topics.
 - Events could include application updates, system alerts, or changes in data.
- **Microservices Communication:**
 - In a microservices architecture, SNS facilitates communication between microservices by allowing them to publish events to topics.
 - Subscribing microservices can then react to relevant events, promoting loose coupling and scalability.
- **Mobile Push Notifications:**
 - SNS is commonly used for sending push notifications to mobile devices (iOS, Android, etc.).
 - Mobile applications can subscribe to SNS topics, and when a relevant message is published to the topic, the notification is sent to all subscribed devices.
- **Email and SMS Messaging:**

- SNS supports sending messages to email addresses and SMS (Short Message Service) endpoints.
- This is useful for applications that need to notify users via email or text messages. Subscribers can be email addresses, phone numbers, or even other AWS services.
- Fanout Systems:
 - SNS facilitates fanout patterns where a single message is broadcast to multiple subscribers.
 - This is beneficial for scenarios where the same information needs to be delivered to multiple endpoints simultaneously.
- Application Health Monitoring:
 - SNS can be used to send alerts and notifications related to the health and status of applications or systems.
 - Monitoring services or components can publish messages to a health-related topic, and subscribers can react accordingly.
- Cross-Region Communication:
 - In multi-region setups, SNS can be used to communicate between applications or services in different regions.
 - Messages published in one region can be received by subscribers in other regions, enabling global communication.
- Workflow Coordination:
 - SNS is employed in workflow coordination systems where different stages of a process are triggered by events.
 - Subscribers can be components or services responsible for handling specific workflow steps.

Key Features:

- Topics:
 - SNS uses topics to group multiple subscribers interested in receiving the same type of message.
 - Publishers send messages to topics, and subscribers receive copies of these messages.
- Subscribers:
 - Subscribers are endpoints that receive messages published to topics.
 - Subscribers can include AWS Lambda functions, HTTP/HTTPS endpoints, email addresses, mobile devices, and more.
- Publishers:
 - Publishers are components or services that send messages to SNS topics.
 - Publishers do not need to know the specific subscribers; they simply publish messages to the relevant topic.
- Push and Pull Mechanisms:
 - SNS supports both push and pull mechanisms. Subscribers can receive messages via HTTP/HTTPS endpoints (push), or they can periodically poll SNS to retrieve messages (pull).
- Message Filtering:
 - SNS allows message filtering based on message attributes. Subscribers can specify filter policies to receive only messages that match certain criteria, enhancing the precision of message delivery.
- Delivery Retries:
 - SNS automatically retries message delivery to subscribers in case of failures.
 - This helps ensure the reliability of message delivery even in the presence of transient issues.

- **Dead Letter Queues:**
 - SNS supports the configuration of dead letter queues (DLQs) for failed message deliveries.
 - Messages that cannot be delivered to subscribers after a certain number of retries are sent to the DLQ for further analysis.
- **Encryption:**
 - SNS provides options for encrypting messages in transit using HTTPS and at rest using AWS Key Management Service (KMS).
 - This helps ensure the confidentiality and integrity of messages.

Benefits:

- **Scalability:**
 - SNS is designed for high throughput and can handle a large number of messages and subscribers, making it suitable for scalable and distributed systems.
- **Decoupling of Components:**
 - SNS promotes loose coupling between components by allowing them to communicate through messages without direct dependencies.
 - Publishers and subscribers can operate independently.
- **Flexible Message Delivery:**
 - SNS supports multiple delivery protocols, including HTTP/HTTPS, email, SMS, and more.
 - This flexibility enables developers to choose the most suitable delivery mechanism for their use case.
- **Reliability:**
 - SNS ensures reliable message delivery by handling retries, dead letter queues, and providing encryption options.
 - This reliability is crucial for applications that depend on timely and accurate message delivery.
- **Event-Driven Architectures:**
 - SNS facilitates the creation of event-driven architectures, where components respond to events by subscribing to relevant topics.
 - This approach enables dynamic and scalable systems that react to changing conditions.
- **Ease of Integration:**
 - SNS seamlessly integrates with other AWS services, making it easy to incorporate messaging capabilities into various applications and architectures within the AWS ecosystem.
- **Monitoring and Logging:**
 - SNS provides monitoring and logging capabilities, allowing users to track message deliveries, subscription status, and overall system health using Amazon CloudWatch.
- **Cost-Effective:**
 - SNS follows a pay-as-you-go pricing model, allowing organizations to pay for the specific resources they use.
 - This cost-effective approach aligns with the scale and needs of applications using SNS.

AWS Simple Workflow (SWF)

- Amazon Simple Workflow (SWF) is a fully managed workflow service provided by Amazon Web Services (AWS).
- SWF helps developers build, run, and scale distributed applications with ease by orchestrating and coordinating tasks across various components.
- It provides a programming framework for defining, executing, and managing workflows in the cloud.

What it does?

- SWF acts as a coordination service for distributed applications, managing the flow of tasks and the state of workflows.
- It allows developers to define complex workflows as a series of coordinated tasks, manage their execution, handle errors, and maintain the overall state of the workflow.
- SWF decouples the coordination logic from the individual tasks, providing a reliable and scalable solution for building distributed and asynchronous systems.

Use Cases:

- **Business Process Automation:**
 - SWF is used for automating and orchestrating business processes that involve multiple steps or tasks.
 - It allows developers to model and execute complex workflows in a reliable and scalable manner.
- **Media Processing Workflows:**
 - Media processing applications, such as transcoding videos, can benefit from SWF for orchestrating tasks like file upload, processing, and distribution.
 - SWF ensures the reliable execution of each step in the media processing workflow.
- **Order Fulfillment Systems:**
 - E-commerce platforms use SWF to manage order fulfillment workflows.
 - Tasks like order processing, inventory check, and shipment coordination can be orchestrated using SWF to ensure accurate and timely order fulfillment.
- **Data Pipeline Orchestration:**
 - SWF is used to orchestrate data processing pipelines that involve multiple stages, such as data extraction, transformation, and loading (ETL).
 - It helps coordinate the execution of tasks across distributed computing resources.
- **Financial Application Workflows:**
 - In financial applications, SWF can be employed to manage complex workflows related to transaction processing, risk assessment, and compliance checks.
 - It ensures that each step in the financial workflow is executed in the correct order.
- **Healthcare Workflow Management:**
 - Healthcare applications use SWF to coordinate workflows related to patient care, appointment scheduling, and medical data processing.
 - SWF ensures that healthcare workflows are executed efficiently and reliably.
- **Supply Chain Management:**
 - SWF is suitable for managing and optimizing supply chain processes.
 - It can coordinate tasks related to inventory management, order processing, and shipment tracking in a scalable and distributed environment.

- **Scientific Research Workflows:**
 - Scientific research projects often involve complex workflows with multiple steps.
 - SWF can be used to orchestrate tasks in scientific data analysis, simulation workflows, and other research processes.

Key Features:

- **Workflow Execution:**
 - SWF allows developers to define and execute workflows by specifying a series of tasks and their dependencies.
 - It handles the coordination of task execution, retries, and error handling.
- **Task Coordination:**
 - SWF provides coordination capabilities for tasks, allowing developers to specify the order and dependencies between tasks.
 - It ensures that tasks are executed in the correct sequence and manages parallelism when necessary.
- **Task Execution History:**
 - SWF maintains a complete history of task execution, allowing developers to monitor the progress of workflows, review completed tasks, and analyze the overall execution timeline.
- **State Management:**
 - SWF manages the state of workflows, allowing developers to store and retrieve workflow state information.
 - This is essential for maintaining the context and progress of long-running workflows.
- **Task Retries and Timeouts:**
 - SWF supports automatic retries for failed tasks and allows developers to set timeouts for task execution.
 - This ensures robustness and reliability in the face of transient failures.
- **Deciders and Workers:**
 - SWF uses the concepts of deciders and workers. Deciders are responsible for coordinating the execution of tasks, while workers perform the actual tasks.
 - This separation of concerns enables flexibility and scalability.
- **Asynchronous and Distributed Execution:**
 - SWF enables the asynchronous and distributed execution of tasks.
 - Tasks can be performed by different computing resources, and SWF ensures that they are coordinated and executed correctly.
- **Task Parallelism:**
 - SWF supports parallel execution of tasks, allowing developers to specify parallel branches in workflows.
 - This is useful for optimizing the execution of tasks that can be performed concurrently.

Benefits:

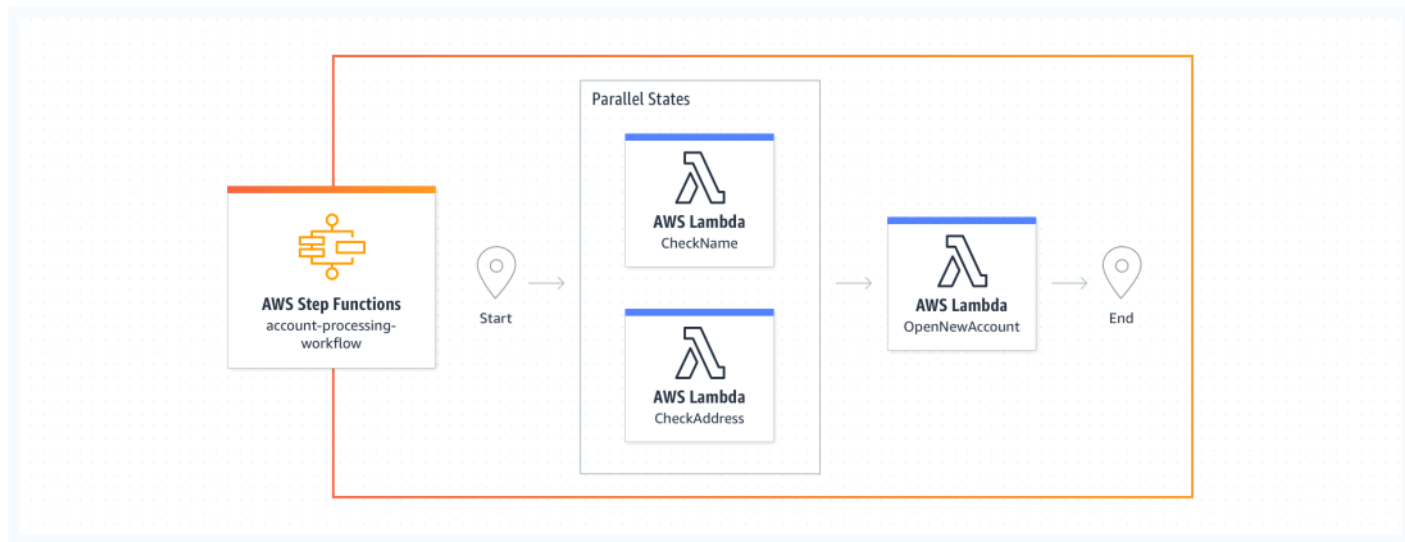
- **Distributed Workflow Orchestration:**
 - SWF enables the orchestration of workflows across distributed and scalable computing resources.
 - This is crucial for building applications that require coordination across multiple components.
- **Reliability and Fault Tolerance:**
 - SWF provides built-in features for handling task retries, timeouts, and error handling.
 - This contributes to the reliability and fault tolerance of workflows, ensuring that

tasks are executed successfully even in the presence of failures.

- Scalability:
 - SWF scales horizontally to handle a large number of workflows and tasks.
 - It can accommodate varying workloads and dynamically scale resources based on the demand for workflow execution.
- Workflow Monitoring and Analysis:
 - SWF offers tools for monitoring and analyzing workflow execution.
 - Developers can track the progress of workflows, review task execution history, and gain insights into the performance of their distributed applications.
- Flexibility in Task Execution:
 - SWF provides flexibility in choosing how tasks are executed.
 - Tasks can be performed by different types of workers, allowing developers to use the most suitable computing resources for each task.
- Maintaining Workflow State:
 - SWF maintains the state of workflows, enabling the persistence of workflow context and progress.
 - This is essential for long-running workflows that span extended periods.
- Integration with AWS Services:
 - SWF seamlessly integrates with other AWS services, allowing developers to build end-to-end solutions that leverage services like Amazon S3, Amazon Lambda, and more.
- Cost-Effective:
 - SWF follows a pay-as-you-go pricing model, allowing organizations to pay for the resources they use.
 - This cost-effective approach aligns with the scale and needs of applications leveraging SWF for workflow management.

AWS Step Functions

- AWS Step Functions is a fully managed service provided by Amazon Web Services (AWS) that allows developers to coordinate and orchestrate multiple AWS services into serverless workflows.
- It enables the creation of workflows that automate the execution of a series of tasks, manage state transitions, and handle error scenarios.
- Step Functions simplifies the development of distributed applications by providing a visual workflow editor and handling the complexities of workflow execution.



What it does:

- AWS Step Functions enables developers to define, run, and scale workflows by orchestrating the execution of various tasks and services.
- Workflows, known as state machines, are defined using a JSON-based specification.
- Step Functions then executes these workflows, managing the flow of tasks and handling state transitions based on the outcomes of individual tasks.

Use Cases:

- **Microservices Orchestration:**
 - Step Functions can be used to orchestrate microservices in a serverless architecture.
 - It helps manage the flow of tasks between different microservices, ensuring coordination and communication in a scalable manner.
- **E-commerce Order Processing:**
 - In e-commerce applications, Step Functions can be used to automate order processing workflows.
 - Tasks such as order validation, payment processing, inventory check, and shipment coordination can be orchestrated seamlessly.
- **Media Processing Pipelines:**
 - Media processing workflows, such as video transcoding or image processing, benefit from Step Functions.
 - It allows developers to define and manage the sequence of tasks involved in processing media files.
- **Data Ingestion and ETL:**
 - Step Functions are suitable for orchestrating data ingestion and ETL (Extract, Transform, Load) processes.
 - It helps coordinate tasks involved in extracting data from various sources,

transforming it, and loading it into a target system.

- **Batch Processing Workflows:**
 - Batch processing applications that involve multiple stages of computation can be orchestrated using Step Functions.
 - It helps manage the execution of tasks in a specified order, handling dependencies and ensuring reliability.
- **Workflow Automation:**
 - Step Functions can be used for general-purpose workflow automation in various domains.
 - It helps automate repetitive tasks, manage dependencies between tasks, and handle complex workflows in a reliable manner.
- **DevOps Automation:**
 - In a DevOps context, Step Functions can automate deployment workflows, infrastructure provisioning, and other operational processes.
 - It allows for the coordination of various tasks involved in software delivery and infrastructure management.
- **Stateful Applications:**
 - Step Functions are suitable for managing stateful applications where the state transitions depend on the outcomes of tasks.
 - This is particularly useful in applications with complex business logic and multiple decision points.

Key Features:

- **Visual Workflow Editor:**
 - Step Functions provides a visual workflow editor that allows developers to design and visualize workflows using a graphical interface.
 - This simplifies the creation and understanding of complex workflows.
- **State Machines:**
 - Workflows in Step Functions are represented as state machines, defined using the Amazon States Language (ASL).
 - State machines define the sequence of tasks, their dependencies, and the flow of control.
- **Built-in Retry and Error Handling:**
 - Step Functions automatically handles retries and error handling for tasks.
 - Developers can specify retry policies and error catchers in the state machine definition, ensuring robust and resilient workflows.
- **Choice State:**
 - Step Functions includes a Choice state that allows developers to define conditional branching within workflows.
 - This is useful for decision-making based on the outcomes of previous tasks.
- **Parallel State:**
 - The Parallel state in Step Functions allows for the parallel execution of multiple tasks.
 - This is beneficial for scenarios where tasks can be executed independently and concurrently.
- **Wait State:**
 - Step Functions supports a Wait state, allowing developers to introduce delays in workflows.
 - This is useful for scenarios where tasks need to wait for a specified duration or until a certain condition is met.
- **Lambda Integration:**
 - Step Functions seamlessly integrates with AWS Lambda, enabling the execution of serverless functions as tasks within workflows.
 - This facilitates the creation of serverless architectures with event-driven workflows.
- **Integration with AWS Services:**

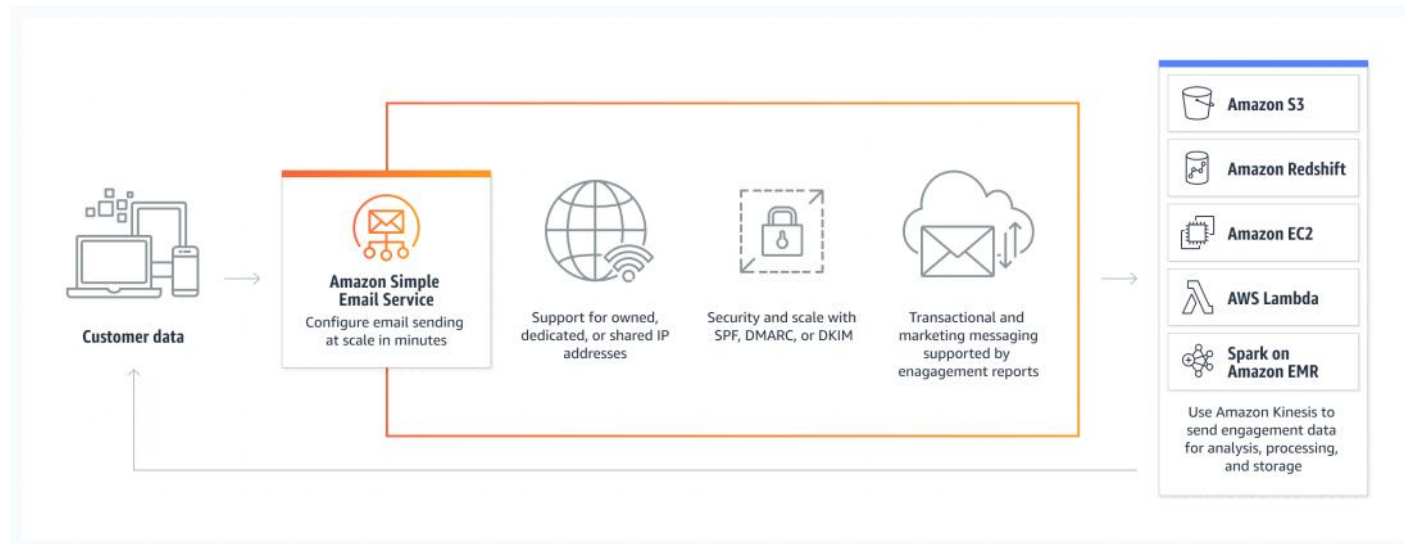
- Step Functions can integrate with various AWS services, including AWS Batch, Amazon ECS (Elastic Container Service), AWS Fargate, AWS Glue, and more.
- This allows developers to leverage a wide range of services within their workflows.

Benefits:

- **Simplified Workflow Development:**
 - The visual workflow editor in Step Functions simplifies the development of workflows, making it easier for developers to design, modify, and understand complex orchestration logic.
- **Error Handling and Retries:**
 - Step Functions automates error handling and retries for tasks, reducing the complexity of dealing with transient failures.
 - Developers can focus on defining tasks and let Step Functions manage reliability.
- **Scalability:**
 - Step Functions scales automatically to handle the execution of workflows, making it suitable for applications with varying workloads.
 - It ensures that workflows can scale up or down based on demand.
- **State Management:**
 - Step Functions manages the state of workflows, enabling the persistence of workflow context and progress.
 - This is crucial for long-running workflows that may span extended periods.
- **Integration with Serverless Computing:**
 - Step Functions seamlessly integrates with AWS Lambda, allowing developers to execute serverless functions as tasks within workflows.
 - This enables the creation of serverless architectures with event-driven workflows.
- **Flexible Task Execution:**
 - Step Functions supports the execution of tasks across various AWS services, providing flexibility in choosing the most suitable computing resources for each task within the workflow.
- **Parallel and Concurrent Execution:**
 - The Parallel state in Step Functions allows for concurrent execution of tasks, improving the efficiency of workflows by enabling parallelism where applicable.
- **Monitoring and Logging:**
 - Step Functions provides tools for monitoring and logging workflow executions.
 - Developers can track the progress of workflows, review task execution history, and gain insights into the performance of their orchestrated applications.
- **Cost-Effective:**
 - Step Functions follows a pay-as-you-go pricing model, allowing organizations to pay for the resources they use.
 - This cost-effective approach aligns with the scale and needs of applications leveraging Step Functions for workflow orchestration.

Amazon Simple Email Service (SES)

Amazon Simple Email Service (SES) is a fully managed email sending service provided by Amazon Web Services (AWS). SES enables businesses and developers to send transactional and marketing emails, manage email campaigns, and handle incoming email through a scalable and reliable platform. SES is designed to be easy to use, cost-effective, and integrated with other AWS services.



What it does?

SES simplifies the process of sending and receiving emails, whether they are transactional messages, marketing campaigns, or handling inbound emails. It provides a scalable and reliable infrastructure for sending high-volume emails while ensuring deliverability and compliance with email standards.

Use Cases:

- **Transactional Emails:**
 - SES is commonly used for sending transactional emails, such as order confirmations, shipping notifications, and account verifications. It ensures that important, time-sensitive information reaches recipients' inboxes reliably.
- **Marketing Campaigns:**
 - Businesses use SES to send marketing emails and newsletters to a large audience. SES allows for the creation and management of email campaigns, tracking of delivery metrics, and optimizing email content for better engagement.
- **Automated Notifications:**
 - SES is suitable for sending automated notifications to users based on specific events or triggers. This includes password reset emails, subscription confirmations, and alerts for account activities.
- **Application-generated Emails:**
 - Applications and services can integrate with SES to send emails programmatically. This is useful for applications that need to notify users about events, updates, or changes in status.
- **B2B and B2C Communication:**
 - SES supports both business-to-business (B2B) and business-to-consumer (B2C) communication. It is used for establishing and maintaining communication with customers, clients, partners, and other stakeholders.
- **Feedback Loops:**
 - SES provides feedback loops that allow businesses to receive notifications about bounce rates, complaints, and other metrics. This feedback helps organizations identify and address issues that could impact email deliverability.

- Email Forwarding and Processing:
 - SES can be used to receive emails, process them, and forward them to other destinations. This is valuable for scenarios where businesses need to handle incoming emails, such as support ticket creation or automated response systems.
- Transactional Email for Websites:
 - Websites and web applications often use SES to handle transactional email communications. This includes sending emails for user registrations, password resets, and order confirmations.

Key Features:

- Email Sending:
 - SES provides a reliable infrastructure for sending emails. It supports both plain text and HTML emails and includes features for embedding images, creating links, and customizing email headers.
- High Deliverability:
 - SES is designed to ensure high deliverability of emails by implementing best practices and adhering to industry standards. It includes features like dedicated IP addresses, DKIM (DomainKeys Identified Mail) signing, and SPF (Sender Policy Framework) records.
- Email Templates:
 - SES allows users to create and manage email templates for consistent branding and content across email campaigns. Templates can be customized, and placeholders can be used to personalize emails.
- Custom Sending Domains:
 - Organizations can configure custom sending domains in SES, allowing them to send emails from their own domains rather than using default SES domains. This contributes to better brand identity and trust.
- Bounce and Complaint Tracking:
 - SES provides tracking for bounced emails and complaints, enabling businesses to identify issues that could affect email deliverability. This information is crucial for maintaining a positive sender reputation.
- Email Analytics:
 - SES includes features for tracking email delivery metrics, such as open rates, click-through rates, and bounce rates. Analytics help organizations assess the effectiveness of their email campaigns and make data-driven decisions.
- Email Receiving:
 - SES can be configured to receive emails, acting as a mail server for processing incoming messages. This is useful for scenarios where organizations need to handle replies, support tickets, or other inbound communication.
- Email Forwarding:
 - SES supports email forwarding, allowing organizations to receive emails and automatically forward them to other destinations. This feature is often used for creating email processing workflows.

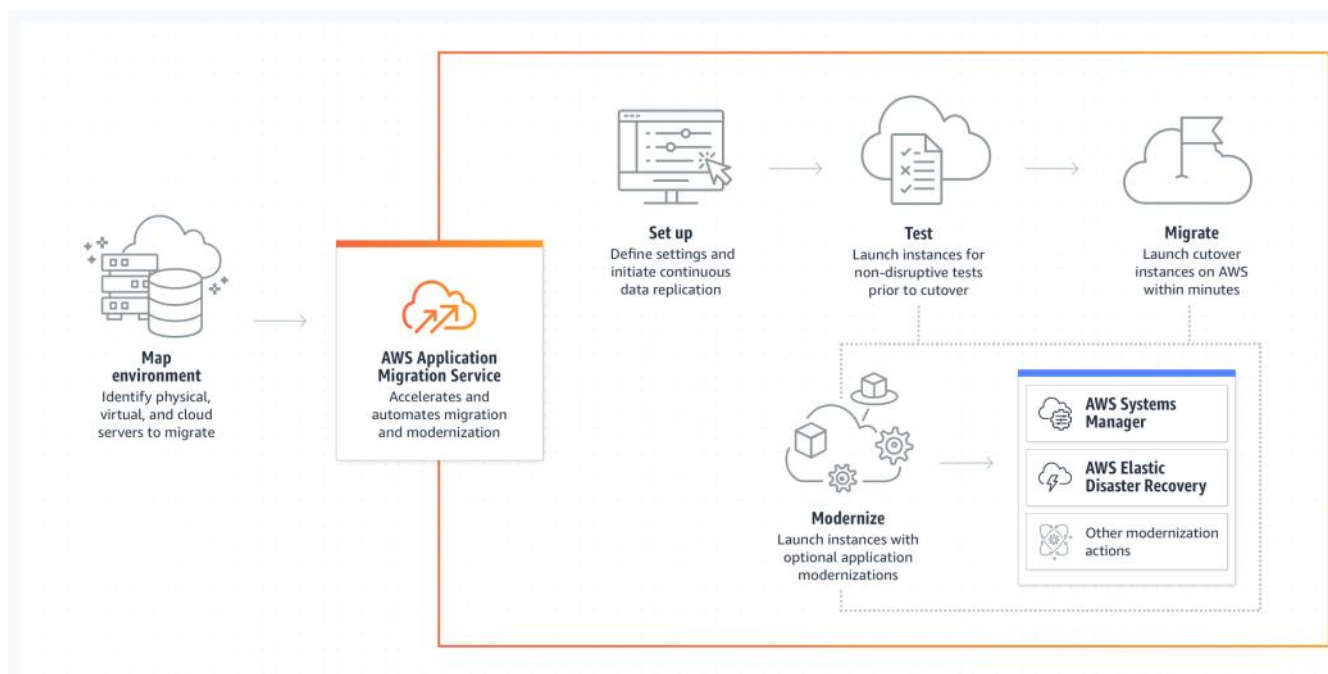
Benefits:

- Scalability:
 - SES is designed to scale horizontally to handle large volumes of emails. It can accommodate varying workloads, making it suitable for both small-scale applications and high-volume email campaigns.
- Reliability:
 - SES provides a reliable infrastructure for sending and receiving emails. It leverages the global AWS network and redundancies to ensure that emails are delivered consistently and efficiently.
- Cost-Effective:
 - SES follows a pay-as-you-go pricing model, allowing organizations to pay for the resources they use. This cost-effective approach makes SES suitable for businesses of all sizes, from startups to large enterprises.
- Integration with AWS Services:

- SES seamlessly integrates with other AWS services, such as Lambda, S3, and CloudWatch. This integration allows businesses to build end-to-end solutions for email processing, analytics, and automation.
- Developer-Friendly APIs:
 - SES provides APIs and SDKs (Software Development Kits) for easy integration with applications. Developers can use the programming language of their choice to send and receive emails using SES.
- Feedback Loops:
 - SES offers feedback loops that provide insights into email delivery issues. This allows organizations to proactively address bounce rates and complaints, maintaining a positive sender reputation.
- Flexible Configuration:
 - SES allows for flexible configuration of email sending and receiving settings. Organizations can customize sending domains, configure DKIM signing, and set up rules for processing incoming emails.
- Compliance and Security:
 - SES complies with email authentication standards and implements security features to protect against spam and phishing. It helps organizations maintain a good sender reputation and ensures the security of email communications.

AWS Simple Migration Service (SMS)

AWS Server Migration Service (SMS) is a service that assists in migrating on-premises workloads to AWS. It simplifies and automates the process of migrating virtualized servers to Amazon EC2 instances. SMS is particularly useful for large-scale migration projects, enabling organizations to efficiently move their applications to the AWS cloud.



What it does?

SMS helps automate the migration of virtualized workloads by replicating server volumes to Amazon Machine Images (AMIs) and then launching EC2 instances based on those AMIs. It provides a reliable and automated way to migrate applications without the need for significant manual intervention.

Use Cases:

- **Data Center Migration:**
 - Organizations looking to migrate their on-premises data centers to AWS can use SMS to automate the migration of virtualized servers. This includes applications, databases, and other workloads.
- **Application Modernization:**
 - For organizations undergoing application modernization initiatives, SMS facilitates the migration of existing virtualized applications to AWS, where they can take advantage of cloud-native services and architectures.
- **Data Migration to the Cloud:**
 - SMS is used for migrating virtual machines and associated data to AWS. This is relevant for scenarios where organizations want to move their existing workloads to the cloud while preserving the data and configurations.
- **Disaster Recovery:**
 - SMS can be utilized for setting up disaster recovery solutions by replicating on-premises virtual machines to AWS. This ensures that there is a failover environment in the cloud in case of a disaster at the primary data center.
- **Application Consolidation:**
 - In cases where organizations want to consolidate servers or data centers, SMS

can be employed to migrate virtual machines to AWS and optimize resource usage through consolidation.

- Cloud Adoption:
 - Companies transitioning to the cloud can use SMS to smoothly migrate their existing workloads to AWS, enabling a phased approach to cloud adoption and minimizing disruption to ongoing operations.

Key Features:

- Automated Replication:
 - SMS automatically replicates on-premises virtual machine volumes to Amazon Machine Images (AMIs). This ensures that the server configurations and data are replicated accurately in the AWS environment.
- Incremental Replication:
 - SMS supports incremental replication, only transferring changes made to the virtual machine since the last replication. This reduces the time and bandwidth required for ongoing replication.
- Server Grouping:
 - Servers that are part of an application or have dependencies on each other can be grouped together in SMS. This allows for coordinated and consistent migration of related servers.
- Application Dependency Mapping:
 - SMS provides application dependency mapping features, helping organizations understand the relationships and dependencies between different servers. This information is crucial for planning and executing migrations.
- Server Migration Plans:
 - Migration plans in SMS allow organizations to define the order in which servers are migrated and specify any dependencies or constraints. This helps orchestrate the migration process for complex applications.
- Server Validation:
 - SMS includes server validation features to check if the configuration of the virtual machines is compatible with AWS. It helps identify potential issues before the migration process begins.
- Scheduling:
 - Organizations can schedule migration events using SMS, allowing them to plan migrations during periods of lower activity or in accordance with maintenance windows.
- Integration with AWS Migration Hub:
 - SMS integrates with AWS Migration Hub, providing a centralized view of migration progress and status. This allows organizations to monitor and track the status of their entire migration project.

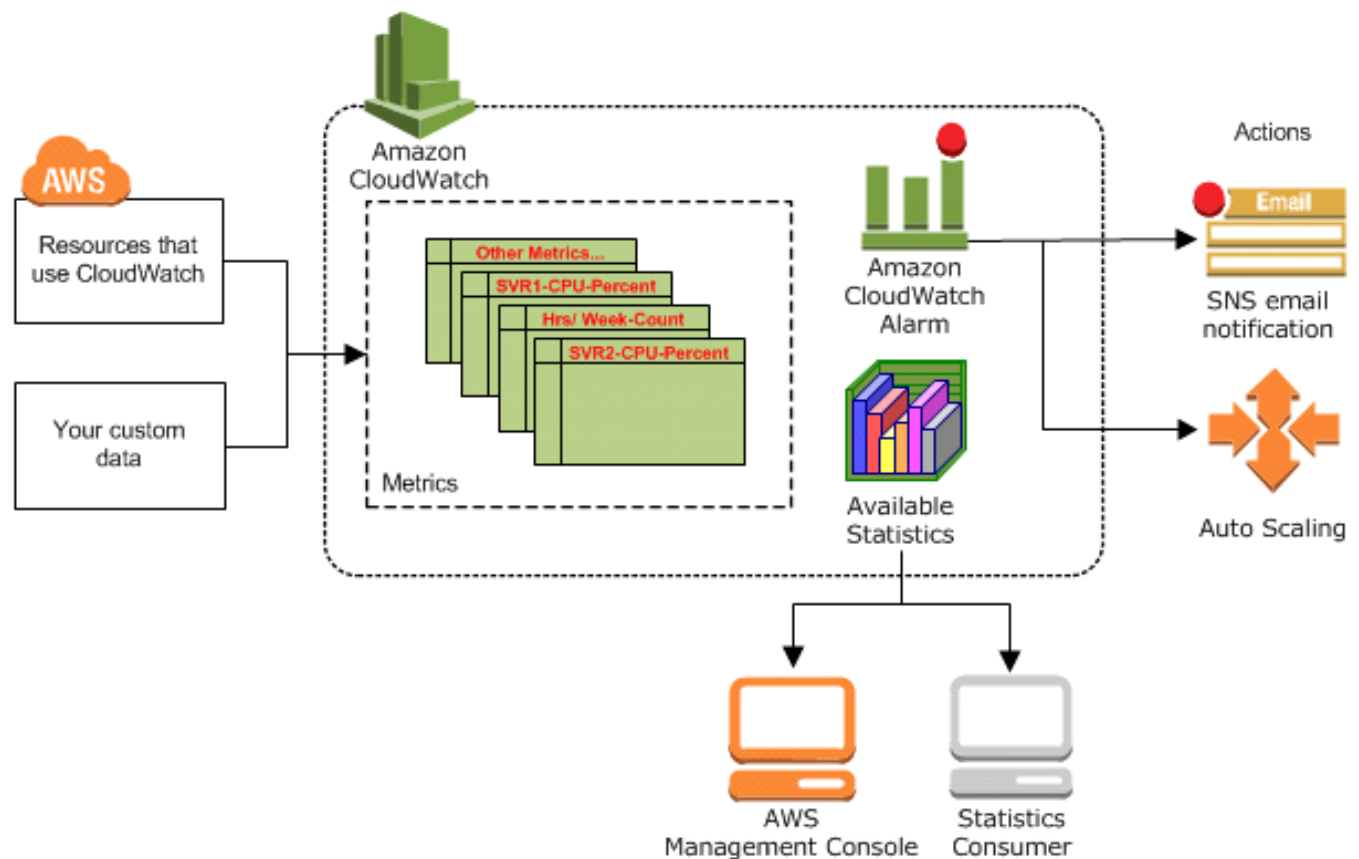
Benefits:

- Automation:
 - SMS automates many aspects of the migration process, reducing the manual effort required for moving virtualized workloads to AWS.
- Minimized Downtime:
 - By using features like incremental replication and server grouping, SMS helps minimize downtime during migrations, allowing organizations to maintain business continuity.
- Consistency:
 - SMS ensures consistency in the migration process, replicating configurations and data accurately to AWS instances.
- Application Visibility:

- The application dependency mapping and migration plans in SMS provide visibility into the relationships between servers, helping organizations plan and execute migrations more effectively.
- Scalability:
 - SMS is designed to handle large-scale migrations, making it suitable for enterprises with diverse and extensive IT environments.
- Cost-Effective:
 - By automating and streamlining the migration process, SMS helps organizations achieve cost-effective migrations with reduced manual effort and associated expenses.
- Integrated Monitoring:
 - Integration with AWS Migration Hub allows organizations to monitor the progress of their migration project in a centralized manner.

Amazon CloudWatch

Amazon CloudWatch is a fully managed monitoring and observability service provided by Amazon Web Services (AWS). It allows users to collect, monitor, and analyze data from various AWS resources, applications, and services in real-time. CloudWatch provides a comprehensive set of tools for tracking the performance, health, and operational status of resources within the AWS cloud environment.



What it does?

CloudWatch captures and stores metrics, collects log files, sets alarms, and automatically reacts to changes in AWS resources. It enables users to gain insights into the behavior of their applications and infrastructure, facilitating better decision-making, troubleshooting, and optimization of resources.

Use Cases:

- Performance Monitoring:
 - CloudWatch is extensively used for monitoring the performance of AWS resources. Users can track key metrics such as CPU utilization, network performance, and disk I/O for EC2 instances, RDS databases, and other AWS services.
- Resource Utilization:
 - Users can monitor the utilization of resources to ensure optimal performance and identify potential bottlenecks.
 - This includes monitoring the usage of compute resources, storage, and other infrastructure components.
- Auto Scaling:
 - CloudWatch plays a crucial role in AWS Auto Scaling.

- Users can set up Auto Scaling policies based on CloudWatch alarms, allowing the automatic adjustment of resources to handle changes in demand.
- Application Monitoring:
 - CloudWatch monitors custom metrics and logs from applications running on AWS.
 - This is beneficial for tracking application-specific performance metrics, errors, and other relevant information.
- Log Monitoring:
 - CloudWatch Logs allows users to collect, monitor, and store log files from various AWS services and applications.
 - This is useful for debugging, troubleshooting, and auditing purposes.
- Infrastructure Health Monitoring:
 - Users can set up CloudWatch alarms to monitor the health of infrastructure components.
 - For example, alarms can be triggered based on high error rates, low disk space, or other indicators of potential issues.
- Cost Management:
 - CloudWatch helps users understand and manage costs by providing insights into resource utilization.
 - By analyzing metrics related to resource consumption, organizations can optimize their infrastructure for cost efficiency.
- Security Monitoring:
 - CloudWatch can be used to monitor security-related events by capturing and analyzing logs generated by AWS CloudTrail.
 - This helps organizations detect and respond to security incidents.
- Custom Metric Monitoring:
 - Users can publish custom metrics to CloudWatch, allowing for the monitoring of specific application or business metrics not covered by default AWS metrics.
- Scheduled Events:
 - CloudWatch Events enables users to schedule automated actions, such as starting or stopping EC2 instances, based on specified time intervals or events within the AWS environment.
- Dashboard Creation:
 - CloudWatch Dashboards allow users to create custom dashboards to visualize and monitor metrics from multiple AWS resources on a single screen.
 - This is useful for gaining a consolidated view of the environment.

Key Features:

- Metrics:
 - CloudWatch collects and stores metrics, which are numerical data points representing the performance of AWS resources over time.
 - These metrics can be visualized using graphs and used for setting up alarms.
- Alarms:
 - Users can set up CloudWatch alarms to receive notifications or take automated actions when certain thresholds are breached.
 - Alarms are based on metrics, and they help users respond promptly to performance or health issues.
- Logs:
 - CloudWatch Logs allows users to collect, store, and analyze log files from various AWS services.
 - Logs can be searched, filtered, and monitored to gain insights into the behavior of applications and resources.

- **Dashboards:**
 - CloudWatch Dashboards provide a customizable and consolidated view of metrics, logs, and alarms from different AWS resources.
 - Dashboards help users monitor the overall health and performance of their environment.
- **Events:**
 - CloudWatch Events enables users to respond to changes in AWS resources or custom events.
 - Users can define rules that trigger actions, such as starting an EC2 instance or invoking a Lambda function, in response to events.
- **Agent-Based Monitoring:**
 - CloudWatch supports the installation of agents on EC2 instances, allowing for more detailed monitoring of system-level metrics.
 - This includes monitoring memory usage, disk space, and other operating system metrics.
- **Cross-Account and Cross-Region Monitoring:**
 - CloudWatch supports cross-account and cross-region monitoring, allowing users to collect and visualize metrics and logs from multiple AWS accounts and regions in a centralized location.
- **Integration with AWS Services:**
 - CloudWatch seamlessly integrates with various AWS services, including EC2, RDS, Lambda, S3, and more.
 - This integration enables users to monitor and analyze the performance of a wide range of AWS resources.

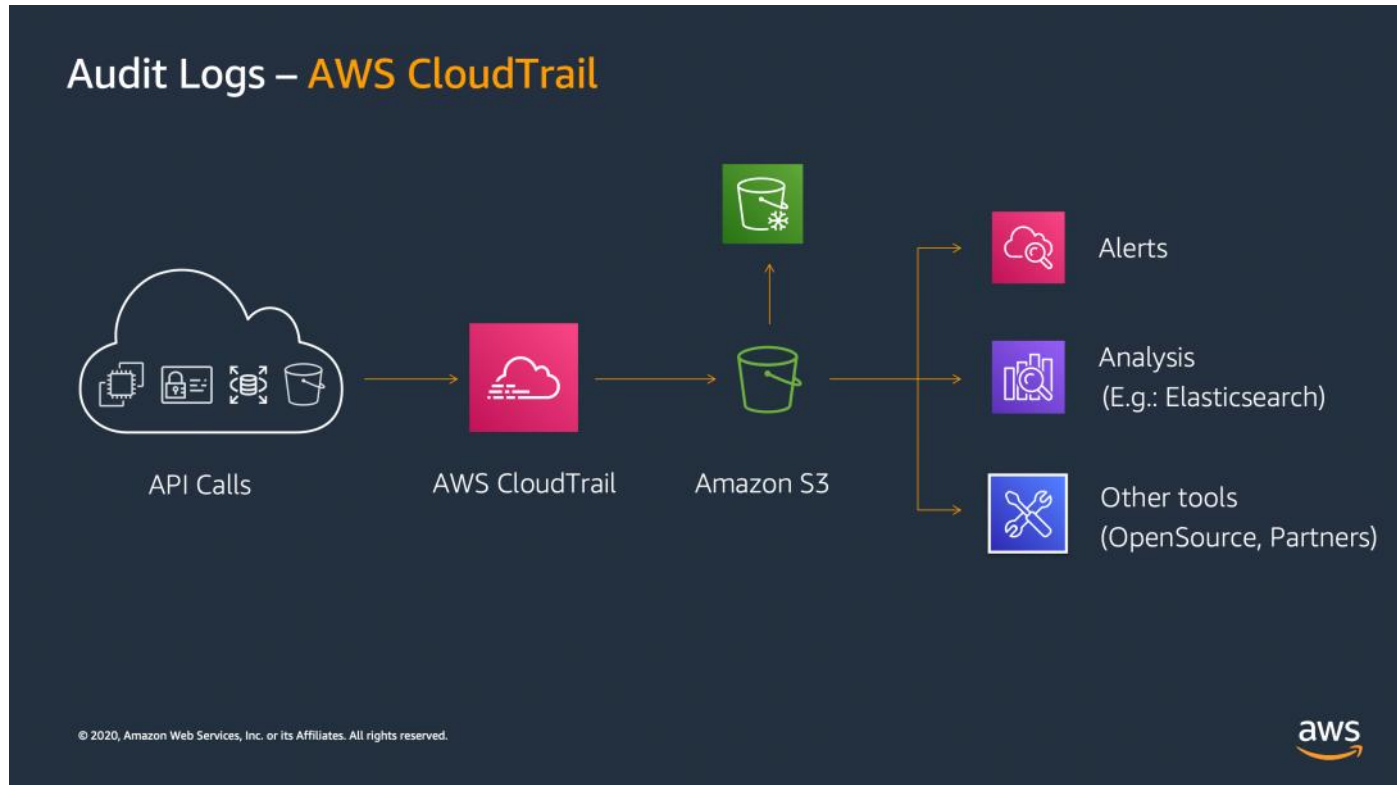
Benefits:

- **Real-Time Visibility:**
 - CloudWatch provides real-time visibility into the performance and health of AWS resources, allowing users to quickly identify and respond to issues.
- **Automation and Auto Scaling:**
 - CloudWatch integrates with AWS Auto Scaling, allowing for automated adjustments to resources based on performance metrics.
 - This helps optimize resource utilization and maintain application performance.
- **Cost Optimization:**
 - By monitoring resource utilization and performance metrics, organizations can identify opportunities for cost optimization and ensure that resources are provisioned efficiently.
- **Troubleshooting and Debugging:**
 - CloudWatch Logs and metrics facilitate troubleshooting and debugging by providing detailed information about system behavior, errors, and application performance.
- **Scalability and Elasticity:**
 - CloudWatch supports the scalability and elasticity of applications by providing metrics and alarms that can trigger automated actions based on changes in demand or resource usage.
- **Centralized Monitoring:**
 - CloudWatch allows organizations to centralize the monitoring of AWS resources, making it easier to manage and gain insights into the entire environment.
- **Customization:**
 - Users can customize CloudWatch to monitor and collect metrics specific to their applications and business requirements, providing flexibility in tracking relevant data points.

- Security Monitoring:
 - CloudWatch supports security monitoring by capturing and analyzing logs generated by AWS CloudTrail.
 - This helps organizations detect and respond to security-related events.
- Operational Insights:
 - CloudWatch provides operational insights by offering a comprehensive view of metrics, logs, and alarms.
 - This helps organizations make informed decisions about their AWS resources.

AWS CloudTrail

Amazon CloudTrail is a service provided by Amazon Web Services (AWS) that enables governance, compliance, operational auditing, and risk auditing of AWS accounts. It records and logs AWS API calls made on the account, providing a detailed history of actions taken by users, applications, or AWS services. CloudTrail logs capture essential details, such as the identity of the entity making the request, the request parameters, and the response elements returned by the AWS service.



What it does:

CloudTrail tracks API calls across an AWS environment, storing the resulting event history in an Amazon S3 bucket. These logs can be analyzed for security and compliance purposes, allowing organizations to monitor account activity, detect unusual behavior, and maintain an audit trail.

Use Cases:

- **Security and Compliance Auditing:**
 - CloudTrail is a crucial tool for security and compliance auditing. It provides a detailed record of all API calls, including who made the call, what actions were taken, and when they occurred.
 - This information is valuable for demonstrating compliance with regulatory requirements and identifying potential security issues.
- **Incident Response and Forensics:**
 - In the event of a security incident or suspected unauthorized activity, CloudTrail logs can be used for forensic analysis.
 - Security teams can review the logs to understand the scope of the incident, identify the origin, and determine the actions taken.
- **Governance and Policy Enforcement:**
 - CloudTrail assists in governance and policy enforcement by allowing

- organizations to monitor whether users are adhering to established policies.
 - It helps ensure that AWS resources are accessed and modified in accordance with organizational guidelines.
- Change Tracking:
 - CloudTrail logs provide a historical record of changes made to AWS resources.
 - This is beneficial for tracking modifications to configurations, security groups, access policies, and other settings.
 - It helps organizations understand how their infrastructure evolves over time.
- Identity and Access Management (IAM) Analysis:
 - CloudTrail logs capture details about IAM roles, users, and permissions, aiding in IAM analysis.
 - Organizations can review the logs to understand who is making changes to IAM policies and roles, ensuring proper access controls are maintained.
- Resource Utilization Monitoring:
 - By monitoring API calls related to resource creation and utilization, CloudTrail helps organizations understand how AWS resources are being used.
 - This information can be used for optimizing resource allocation and cost management.
- Integration with Security Information and Event Management (SIEM) Systems:
 - CloudTrail logs can be integrated with SIEM systems, allowing security teams to correlate AWS activity with broader security events.
 - This enhances the overall security monitoring and incident response capabilities.
- Cross-Account and Cross-Region Monitoring:
 - CloudTrail supports cross-account and cross-region logging, enabling organizations to centralize logs from multiple AWS accounts and regions.
 - This is valuable for managing a diverse and distributed AWS infrastructure.

Key Features:

- Event Logging:
 - CloudTrail records API calls as events, capturing details such as the time of the event, the source IP address, the identity of the requester, and the outcome of the request.
- Log File Integrity Validation:
 - CloudTrail logs are cryptographically signed and can be validated for integrity.
 - This ensures that the logs have not been tampered with and provides an additional layer of security.
- S3 Bucket Logging:
 - CloudTrail logs are stored in an Amazon S3 bucket, providing durable and scalable storage.
 - Organizations can configure their own S3 bucket to receive CloudTrail logs.
- CloudWatch Events Integration:
 - CloudTrail integrates with Amazon CloudWatch Events, allowing organizations to set up rules and automated responses based on CloudTrail events.
 - This can include triggering Lambda functions or notifying teams of specific activities.
- Identity Resolution:
 - CloudTrail logs include information about the entity making the API call, whether it's an IAM user, IAM role, AWS service, or an anonymous user.
- Encryption of Log Files:
 - CloudTrail supports server-side encryption for log files stored in Amazon S3, providing an additional layer of protection for sensitive information.
- Log Filtering and Analysis:
 - Organizations can filter CloudTrail logs based on specific criteria, allowing for

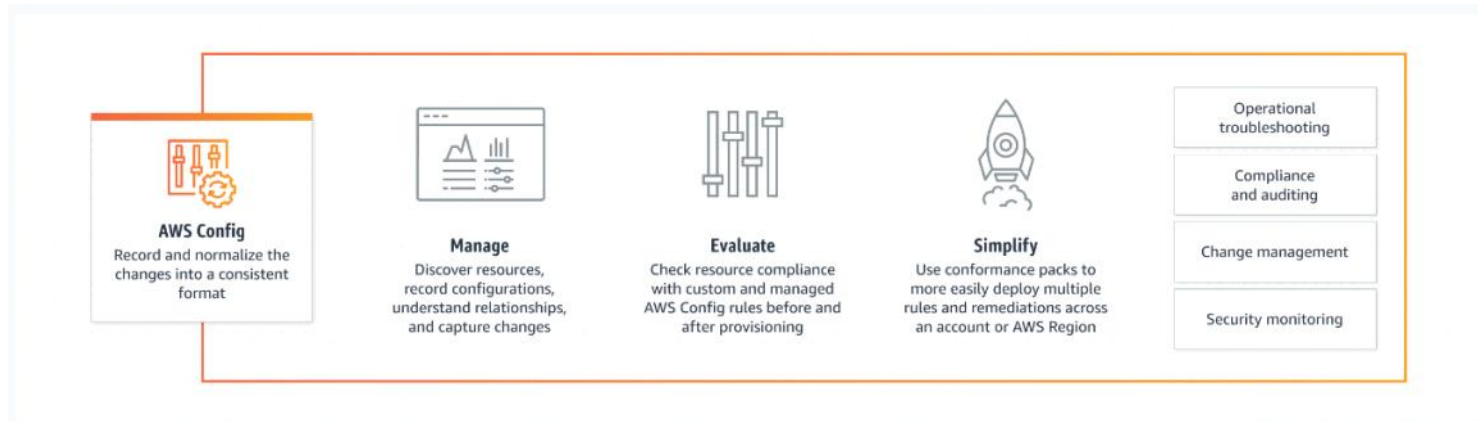
- targeted analysis.
 - This is useful for focusing on specific activities or users during audits.
- Integration with AWS Organizations:
 - CloudTrail integrates with AWS Organizations, enabling organizations to consolidate logs from multiple AWS accounts into a central S3 bucket.

Benefits:

- Comprehensive Auditing:
 - CloudTrail provides a comprehensive audit trail of API calls made within an AWS environment, supporting governance, compliance, and security auditing requirements.
- Visibility into AWS Activity:
 - Organizations gain visibility into AWS activity, allowing them to understand who is performing actions, what actions are being taken, and when these actions occur.
- Security Incident Detection:
 - CloudTrail logs help detect security incidents by providing detailed information about changes and activities within the AWS environment.
 - Security teams can use the logs for threat detection and incident response.
- Compliance Reporting:
 - CloudTrail logs contribute to compliance reporting by providing evidence of adherence to security and governance policies.
 - This is valuable for regulatory audits and certifications.
- Forensic Analysis:
 - In the event of a security incident, CloudTrail logs can be used for forensic analysis to understand the scope of the incident, identify the source, and determine the impact.
- Policy Enforcement:
 - CloudTrail supports policy enforcement by allowing organizations to monitor and ensure that users adhere to established policies and guidelines for accessing and managing AWS resources.
- Change Management:
 - CloudTrail assists with change management by providing a historical record of changes made to AWS resources.
 - This is beneficial for tracking modifications, understanding resource evolution, and troubleshooting issues.
- Centralized Logging and Monitoring:
 - CloudTrail supports centralized logging and monitoring by allowing organizations to consolidate logs from multiple AWS accounts and regions. This centralization simplifies the management and analysis of logs.

AWS Config

AWS Config is a service provided by Amazon Web Services (AWS) that enables continuous monitoring and assessment of AWS resource configurations. It provides a detailed inventory of resources and records configuration changes over time. AWS Config helps organizations maintain compliance, security, and governance by allowing them to track resource configurations, identify drift, and receive notifications for changes.



What it does?

AWS Config continuously monitors and records configurations of AWS resources in an AWS account. It captures configuration snapshots, known as configuration items, and provides a historical view of resource states. Users can define rules to evaluate the configurations against desired states and receive alerts if there is any deviation or non-compliance.

Use Cases:

- **Configuration Visibility:**
 - AWS Config provides a centralized view of the configurations of AWS resources.
 - This visibility allows organizations to understand the state of their infrastructure, track changes, and maintain an up-to-date inventory.
- **Change Management:**
 - Organizations can use AWS Config to track changes made to AWS resources over time.
 - This includes modifications to resource attributes, security group rules, access policies, and other configuration settings.
- **Security and Compliance Monitoring:**
 - AWS Config helps organizations monitor security and compliance by allowing them to assess whether their AWS resources comply with established security and compliance policies.
 - It provides visibility into security group configurations, encryption settings, and more.
- **Drift Detection:**
 - AWS Config detects configuration drift, which occurs when the actual resource configurations deviate from the desired configurations.
 - Drift detection helps organizations identify and rectify configuration inconsistencies to maintain a secure and compliant environment.
- **Resource Relationship Mapping:**
 - AWS Config captures relationships between resources, providing a map of how different resources are connected.
 - This is valuable for understanding dependencies and potential impacts before making changes to the infrastructure.
- **Change Auditing:**
 - AWS Config enables organizations to audit changes made to AWS resources.
 - This is useful for compliance audits, troubleshooting, and understanding the impact of configuration modifications on system behavior.
- **Automated Remediation:**
 - Organizations can use AWS Config rules to automate the remediation of non-compliant resources.
 - When a rule detects a deviation from the desired state, automated actions can be triggered to bring the resource back into compliance.
- **Inventory Management:**
 - AWS Config serves as an inventory management tool by providing a comprehensive list of resources and their configurations.
 - This is beneficial for asset tracking, license management, and resource optimization.
- **Custom Rule Creation:**
 - Users can create custom AWS Config rules to evaluate resource configurations against specific criteria.
 - This allows organizations to enforce custom policies and ensure that resources adhere to internal standards.

Key Features:

- **Configuration History:**
 - AWS Config maintains a detailed history of resource configurations, allowing users to view changes made to resources over time.
 - This includes information about attribute changes, relationships, and more.
- **Configuration Items:**
 - Configuration items are snapshots of resource configurations captured by AWS Config.
 - These items include metadata, relationships, and details about the resource state at a specific point in time.
- **Config Rules:**
 - AWS Config rules are customizable rules that evaluate resource configurations against desired states.
 - Users can choose from predefined rules or create custom rules to enforce

specific policies.

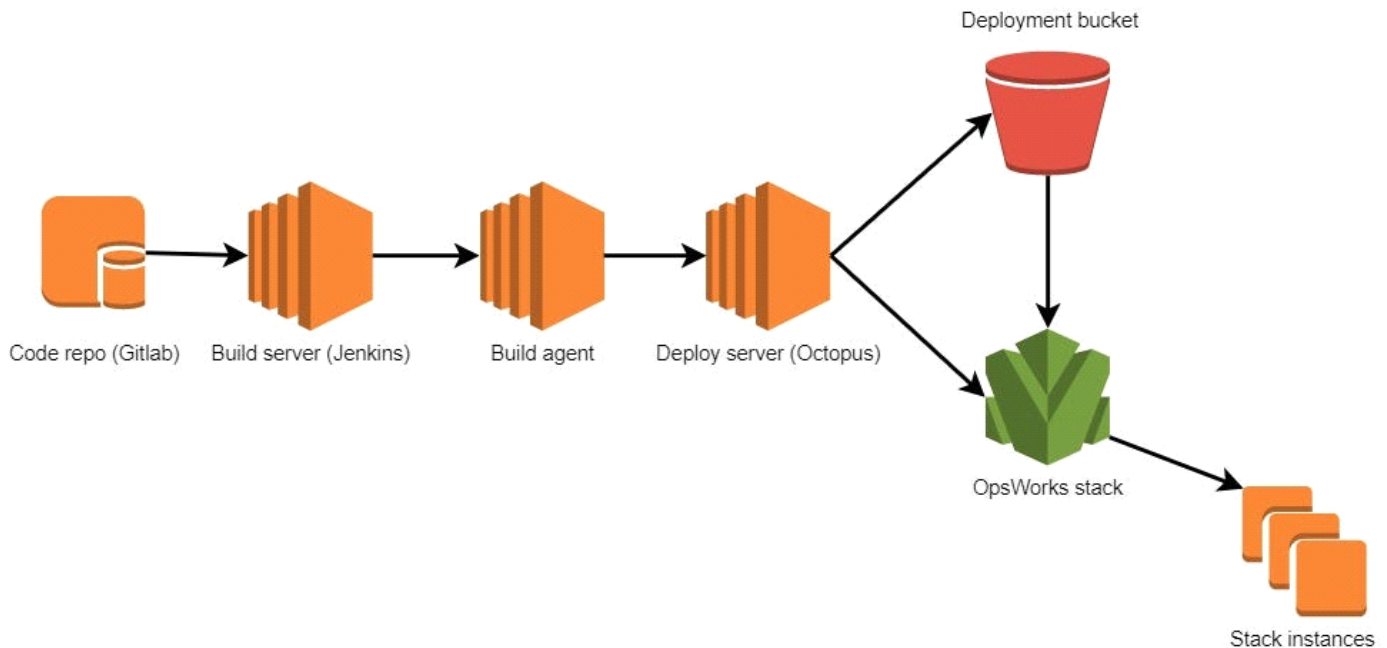
- **Configuration Snapshot:**
 - Users can take manual configuration snapshots using AWS Config to capture the current state of resources.
 - This is useful before making significant changes to the environment or as part of auditing processes.
- **Drift Detection:**
 - Drift detection in AWS Config helps identify and visualize differences between desired and actual resource configurations.
 - It enables organizations to understand where configurations have deviated and take corrective actions.
- **Resource Relationships:**
 - AWS Config captures relationships between resources, providing insights into how resources are interconnected.
 - This relationship mapping is helpful for understanding dependencies and impacts.
- **Multi-Account and Multi-Region Support:**
 - AWS Config supports multi-account and multi-region configurations, allowing organizations to centralize configuration data from multiple AWS accounts and regions.
- **Event Notifications:**
 - AWS Config provides event notifications that can be configured to alert users about changes to configurations, rule evaluations, and compliance status.
 - Notifications can be delivered via Amazon SNS (Simple Notification Service).
- **Resource Exclusion:**
 - Users can exclude specific resources from AWS Config evaluations, allowing for customization based on the organization's requirements.
 - This is useful when certain resources should be exempt from specific rules.
- **Access Control:**
 - AWS Config integrates with AWS Identity and Access Management (IAM) for access control.
 - Organizations can define policies to control who can view configurations, create rules, or access other AWS Config features.

Benefits:

- **Continuous Monitoring:**
 - AWS Config provides continuous monitoring of AWS resources, offering a real-time view of configurations and changes made to the environment.
- **Compliance Assurance:**
 - Organizations can use AWS Config to enforce compliance with security and governance policies. Config rules help identify and remediate non-compliant resources automatically.
- **Change Tracking:**
 - AWS Config tracks changes made to AWS resources, offering a comprehensive change history. This is beneficial for change management, auditing, and understanding the evolution of the infrastructure.
- **Configuration Drift Detection:**
 - AWS Config detects configuration drift, helping organizations identify and rectify inconsistencies between desired and actual resource configurations.
- **Resource Relationship Mapping:**
 - The ability to map relationships between resources provides valuable insights into the structure of the AWS environment, assisting with troubleshooting and planning.
- **Automation and Remediation:**
 - AWS Config supports automation through custom rules and remediation actions. This allows organizations to automatically enforce policies and correct non-compliant resources.
- **Multi-Account and Multi-Region Support:**
 - AWS Config is scalable and supports configurations from multiple AWS accounts and regions, making it suitable for complex and distributed environments.
- **Integration with Other AWS Services:**
 - AWS Config integrates with other AWS services, including CloudWatch, CloudTrail, and SNS, enhancing its capabilities and providing additional features for monitoring and alerting.
- **Audit Trail for Compliance:**
 - AWS Config serves as an audit trail for compliance purposes, providing evidence of adherence to configurations and policies. This is valuable for regulatory audits and reporting.

AWS OpsWorks

- AWS OpsWorks is a configuration management service provided by Amazon Web Services (AWS).
- It helps automate the deployment and management of applications on AWS infrastructure.
- OpsWorks supports both Chef and Puppet, two popular automation frameworks, allowing users to define, deploy, and manage the entire application stack, including application code, runtime settings, and infrastructure configuration.



What it does?

OpsWorks enables users to model and manage their applications and infrastructure as code. It provides a flexible and scalable platform for deploying and managing applications, allowing users to focus on application development and leave the infrastructure management to OpsWorks.

Use Cases:

- **Application Deployment:**
 - OpsWorks is used for deploying applications by defining the entire application stack, including web servers, application servers, and databases, as code.
 - This allows for consistent and repeatable deployments.
- **Configuration Management:**
 - With support for configuration management tools like Chef and Puppet, OpsWorks facilitates the definition and enforcement of infrastructure configurations.
 - This includes server settings, application settings, and dependencies.
- **Auto Scaling:**
 - OpsWorks supports auto scaling, allowing users to dynamically adjust the number of instances in a layer based on demand.
 - This is particularly useful for handling varying workloads and optimizing resource utilization.
- **Infrastructure as Code:**
 - OpsWorks enables the practice of infrastructure as code (IaC), where the entire infrastructure is defined in a code format.
 - This approach enhances repeatability, version control, and collaboration in the

deployment process.

- Continuous Integration and Continuous Deployment (CI/CD):
 - OpsWorks integrates with CI/CD tools and services, allowing for the automation of build and deployment pipelines.
 - This ensures that application changes are automatically deployed to the infrastructure.
- Customization and Configuration:
 - OpsWorks allows users to customize the configuration of instances in their stacks.
 - This includes specifying software packages, configuring services, and defining security settings.
- Application Stacks and Layers:
 - OpsWorks organizes resources into stacks, each representing a different application, and layers, representing different components of the application stack (e.g., web layer, app layer).
 - This provides a structured way to manage complex applications.
- Monitoring and Logging:
 - OpsWorks integrates with Amazon CloudWatch for monitoring and provides logs for each instance, making it easier to troubleshoot issues, monitor performance, and gain insights into application behavior.
- Compliance and Security:
 - OpsWorks helps enforce compliance and security policies by allowing users to define and manage security settings, permissions, and access controls for instances and applications.

Key Features:

- Stacks and Layers:
 - OpsWorks organizes resources into stacks, representing different applications or environments, and layers, representing different components of the application stack.
 - This allows for modular and scalable management.
- Chef and Puppet Integration:
 - OpsWorks supports integration with Chef and Puppet, two popular configuration management tools.
 - Users can leverage existing Chef or Puppet scripts or choose from predefined configurations.
- Custom Recipes and Cookbooks:
 - Users can define custom recipes and cookbooks (for Chef) or manifests and modules (for Puppet) to configure instances.
 - This allows for flexibility and customization of the deployment process.
- Auto Healing:
 - OpsWorks provides auto healing, which automatically replaces instances that become unhealthy.
 - This ensures high availability by automatically recovering from failures.
- Auto Scaling:
 - OpsWorks supports auto scaling based on time or load. Users can define scaling rules to automatically adjust the number of instances in a layer, ensuring that the application can handle varying levels of demand.
- Integration with Other AWS Services:
 - OpsWorks integrates with various AWS services, including Amazon RDS, Elastic Load Balancing (ELB), and CloudWatch.
 - This integration enhances the functionality of OpsWorks and allows for a more comprehensive application stack.
- Custom AMIs:
 - OpsWorks allows users to use custom Amazon Machine Images (AMIs) for instances.
 - This is beneficial for organizations that have specific requirements for the base configuration of their instances.
- Permission Management:

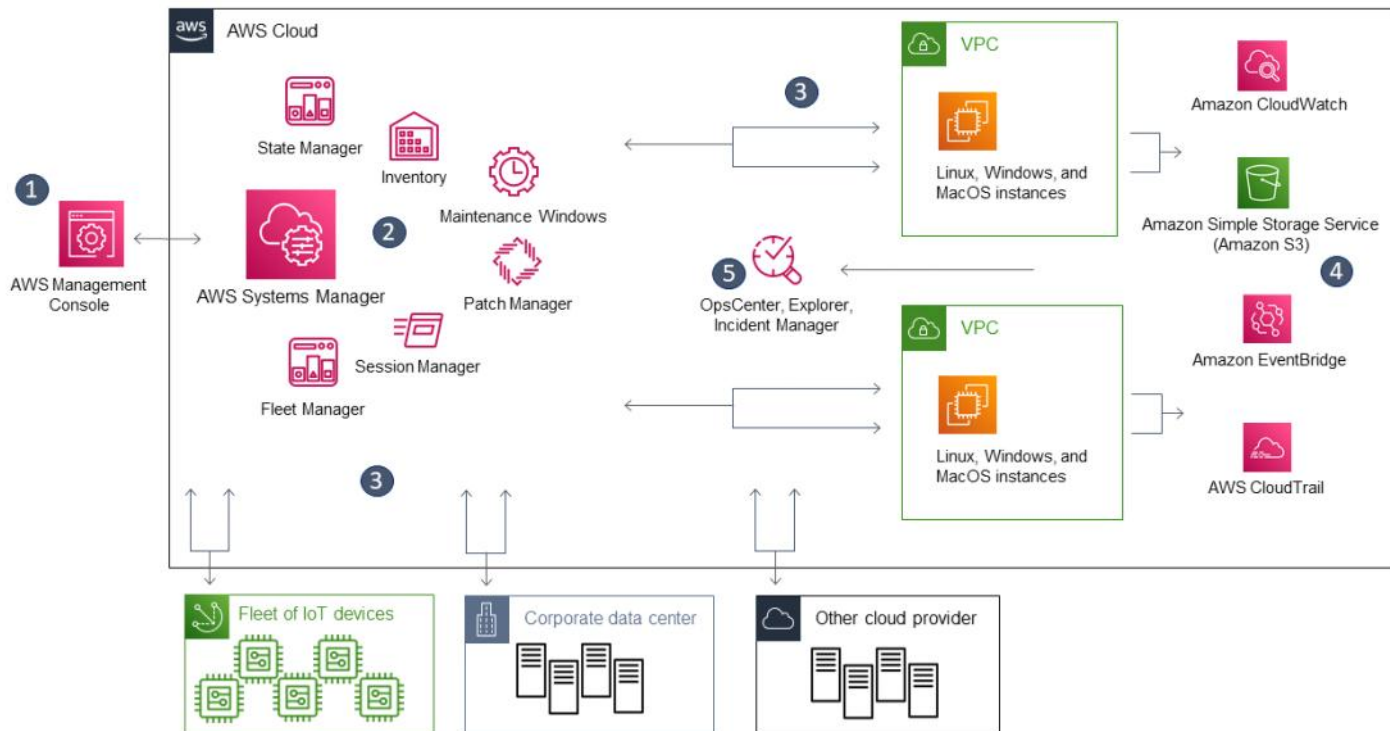
- OpsWorks provides role-based access control, allowing users to define permissions for different users or groups.
- This helps enforce security policies and restrict access to sensitive operations.
- Layer Recipes:
 - Users can define recipes at the layer level, allowing for specific configurations for each layer.
 - This is useful for managing the unique requirements of different components within the application stack.

Benefits:

- Automation and Consistency:
 - OpsWorks automates the deployment and management of applications, ensuring consistency across different environments and reducing the risk of configuration errors.
- Scalability:
 - OpsWorks supports auto scaling, allowing applications to scale based on demand.
 - This ensures that the infrastructure can handle varying workloads efficiently.
- Flexibility:
 - With support for Chef and Puppet, OpsWorks provides flexibility in defining and managing configurations.
 - Users can leverage existing automation scripts or choose from predefined configurations.
- Infrastructure as Code (IaC):
 - OpsWorks enables the practice of IaC, allowing users to define and manage their infrastructure using code.
 - This enhances repeatability, version control, and collaboration in the deployment process.
- Monitoring and Troubleshooting:
 - OpsWorks integrates with CloudWatch for monitoring and provides logs for each instance, making it easier to troubleshoot issues, monitor performance, and gain insights into application behavior.
- Customization:
 - Users can customize configurations at both the stack and instance levels, allowing for fine-grained control over the deployment and runtime settings of applications.
- Integrated CI/CD:
 - OpsWorks integrates with CI/CD tools and services, streamlining the continuous integration and continuous deployment processes.
 - This helps automate the release pipeline and ensure rapid and reliable application deployments.
- Security and Compliance:
 - OpsWorks provides tools for defining and enforcing security policies, permissions, and access controls.
 - This supports security best practices and compliance requirements.

AWS Systems Manager

AWS Systems Manager is a comprehensive management service provided by Amazon Web Services (AWS) that enables centralized visibility and control over AWS resources. Systems Manager helps automate operational tasks, enhance security and compliance, and simplify resource management across AWS environments. It provides a unified user interface for managing infrastructure, applications, and system configurations.



How Systems Manager works?

The following diagram describes how some Systems Manager capabilities perform actions on your resources. The diagram doesn't cover all capabilities. Each enumerated interaction is described before the diagram.

1. **Access Systems Manager** – Use one of the available options for accessing Systems Manager.
2. **Choose a Systems Manager capability** – Determine which capability can help you perform the action you want to perform on your resources. The diagram shows only a few of the capabilities that IT administrators and DevOps personnel use to manage their applications and resources.
3. **Verification and processing** – Systems Manager verifies that your user, group, or role has the required AWS Identity and Access Management (IAM) permissions to perform the action you specified. If the target of your action is a managed node, the Systems Manager Agent (SSM Agent) running on the node performs the action. For other types of resources, Systems Manager performs the specified action or communicates with other AWS services to perform the action on behalf of Systems Manager.
4. **Reporting** – Systems Manager, SSM Agent, and other AWS services that performed an action on behalf of Systems Manager report status. Systems Manager can send status details to other AWS services, if configured.
5. **Systems Manager operations management capabilities** – If enabled, Systems Manager operations management capabilities such as Explorer, OpsCenter, and Incident Manager aggregate operations data or create artifacts in response to events or errors with your resources. These artifacts include operational work items (OpsItems) and incidents. Systems Manager operations management capabilities provide operational insight into your applications and resources and automated remediation solutions to help troubleshoot problems.

What it does?

AWS Systems Manager offers a set of tools and features that allow users to manage their

AWS resources at scale. It includes capabilities for configuration management, patch management, automation, inventory tracking, monitoring, and more. Systems Manager helps organizations streamline operational processes and maintain a consistent and secure environment.

Use Cases:

- **Patch Management:**
 - Systems Manager automates the patching process for Amazon EC2 instances, making it easier to keep systems up to date with the latest security patches.
 - Users can define patch baselines, schedule patching operations, and track compliance.
- **Configuration Management:**
 - Users can define and enforce configurations for their instances using Systems Manager.
 - This includes defining policies for operating system configurations, application settings, and security configurations.
- **Automation Workflows:**
 - Systems Manager allows users to create and run automation workflows for common operational tasks.
 - This can include tasks such as updating AMIs, scaling instances, or executing custom scripts across multiple instances.
- **Run Command:**
 - The Run Command feature enables users to remotely execute commands on a fleet of instances, making it easier to perform operational tasks, troubleshoot issues, or deploy updates across multiple servers.
- **State Manager:**
 - Systems Manager State Manager helps users define and enforce desired state configurations for instances.
 - It allows users to ensure that instances adhere to specific configurations, automatically remediating any drift from the desired state.
- **Inventory Management:**
 - Systems Manager collects and maintains an inventory of software, applications, and infrastructure configurations across instances.
 - This provides visibility into the environment and assists with compliance and auditing.
- **Session Manager:**
 - Session Manager allows users to establish secure connections to instances without the need for opening inbound ports or managing SSH keys.
 - This feature simplifies remote access for troubleshooting and maintenance tasks.
- **Parameter Store:**
 - Parameter Store is a secure and scalable storage solution within Systems Manager for managing configuration data, secrets, and other information.
 - It can be used to centralize and secure sensitive information used by applications.
- **Distributor:**
 - Systems Manager Distributor allows users to centrally store and distribute software packages to instances.
 - It simplifies the process of deploying and updating applications across fleets of instances.
- **OpsCenter:**
 - OpsCenter within Systems Manager provides a central location for managing and resolving operational issues.
 - It helps teams collaborate on solving problems and tracks the resolution of incidents.
- **Application Management:**
 - Systems Manager provides capabilities for managing applications, including integration with Application Manager and insights into application performance.

Key Features:

- **Patch Manager:**
 - Patch Manager automates the process of keeping instances up to date with the latest security patches. Users can define patch baselines, schedule patching operations, and track compliance.
- **Automation:**
 - Automation enables users to create, schedule, and execute workflows for common operational tasks. This includes tasks such as updating software,

configuring instances, and executing scripts.

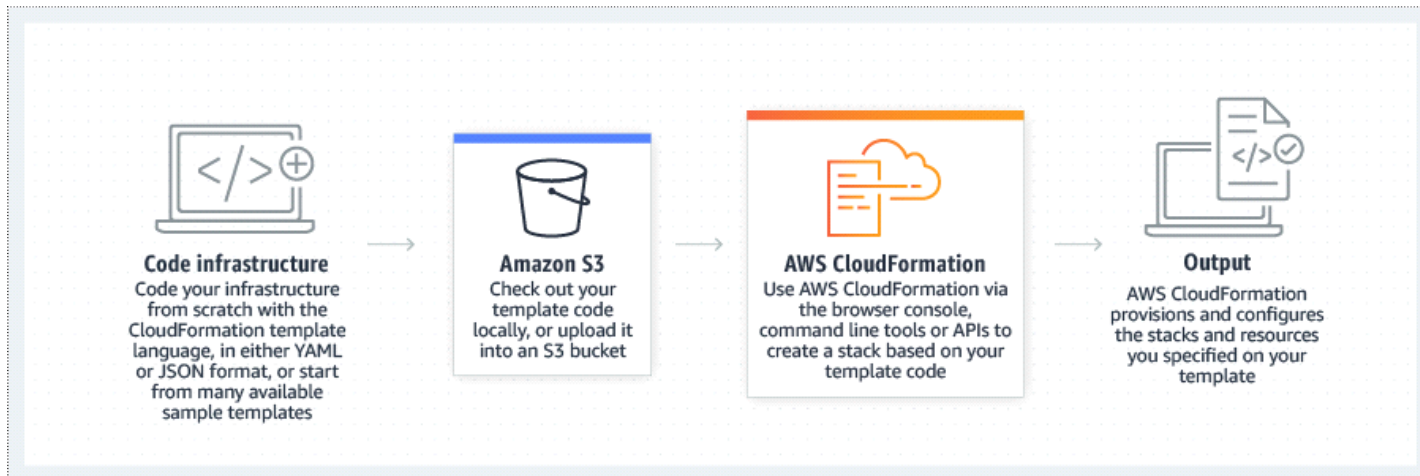
- **Run Command:**
 - Run Command allows users to remotely execute commands on instances, providing a secure and efficient way to perform administrative tasks across multiple servers.
- **State Manager:**
 - State Manager helps users define and enforce desired configurations for instances, ensuring consistency and automatically remediating any configuration drift.
- **Inventory:**
 - Inventory Management collects and maintains an inventory of software, applications, and configurations on instances. This provides visibility into the environment and supports compliance and auditing.
- **Session Manager:**
 - Session Manager provides secure and auditable remote access to instances without the need for open ports or direct access keys. It simplifies the process of troubleshooting and maintenance.
- **Parameter Store:**
 - Parameter Store is a secure storage solution for managing configuration data, secrets, and other information. It supports versioning and hierarchies, making it suitable for managing sensitive data.
- **Distributor:**
 - Distributor simplifies the distribution and deployment of software packages to instances. It allows users to centrally store and manage software artifacts.
- **OpsCenter:**
 - OpsCenter provides a central location for managing and resolving operational issues. It facilitates collaboration, tracks incidents, and helps teams streamline incident response.
- **Application Manager Integration:**
 - Systems Manager integrates with Application Manager to provide capabilities for managing applications, including deployment, scaling, and monitoring.

Benefits:

- **Operational Efficiency:**
 - Systems Manager automates routine operational tasks, reducing manual intervention and improving overall efficiency in managing AWS resources.
- **Consistency and Compliance:**
 - With features like Patch Manager, State Manager, and Inventory Management, Systems Manager helps ensure that instances adhere to desired configurations and remain compliant with organizational policies.
- **Security:**
 - Systems Manager provides secure and auditable remote access to instances through Session Manager. It also supports secure storage of sensitive information in Parameter Store.
- **Visibility and Control:**
 - Systems Manager provides centralized visibility into the environment, allowing users to track configurations, inventory, and operational tasks. This enhances control and decision-making.
- **Scalability:**
 - Systems Manager is designed to scale with the number of instances in an environment. It supports large-scale automation, patching, and management operations across fleets of instances.
- **Automation Workflows:**
 - Automation enables users to define and execute workflows for common operational tasks, reducing the time and effort required for manual intervention.
- **Application Management:**
 - Integration with Application Manager and features like Distributor and OpsCenter provide additional capabilities for managing applications, enhancing the overall application lifecycle management.
- **Cost Optimization:**
 - By automating tasks, ensuring consistent configurations, and optimizing operational processes, Systems Manager contributes to cost optimization by reducing the manual effort required for managing infrastructure.

AWS CloudFormation

- AWS CloudFormation is a service provided by Amazon Web Services (AWS) that enables the provisioning and management of AWS infrastructure as code (IaC).
- It allows users to define and deploy a collection of AWS resources in a predictable and repeatable manner.
- CloudFormation uses templates, written in either JSON or YAML format, to define the architecture and configuration of AWS resources.



What it does?

CloudFormation simplifies the process of creating and managing AWS resources by allowing users to declare the desired state of their infrastructure in a template. These templates are then used to provision and update resources, making it easier to maintain consistency and manage the entire lifecycle of applications and infrastructure.

Use Cases:

- Infrastructure as Code (IaC):
 - CloudFormation is widely used for practicing IaC, where infrastructure configurations are defined in a declarative template.
 - This approach enhances automation, version control, and reproducibility.
- Application Stacks:
 - Users can define entire application stacks in CloudFormation templates, including compute resources, databases, storage, networking, and more.
 - This is useful for deploying and managing complex, multi-tier applications.
- Environment Provisioning:
 - CloudFormation allows users to provision entire environments, including development, testing, and production environments, with a single template.
 - This helps maintain consistency across different stages of the application lifecycle.
- Resource Orchestration:
 - CloudFormation provides orchestration capabilities, allowing users to specify dependencies and relationships between resources.
 - This ensures that resources are created and configured in the correct order.
- Change Management:
 - Users can use CloudFormation to make changes to their infrastructure in a controlled and predictable way.
 - CloudFormation templates support updates, allowing users to modify existing resources without disrupting the entire stack.
- Reusable Templates:
 - CloudFormation templates can be reused across different projects and environments.
 - This promotes standardization and reduces the effort required to create and manage infrastructure configurations.
- Rollback and Rollback Triggers:
 - CloudFormation supports automatic rollback in case of stack creation or update

- failures.
 - Users can define rollback triggers to initiate specific actions when a rollback occurs.
- Parameterization:
 - CloudFormation templates support parameterization, allowing users to define input parameters that customize the behavior of the template.
 - This makes it easy to reuse templates with different configurations.

Key Features:

- Templates:
 - CloudFormation templates are JSON or YAML files that define the architecture and configuration of AWS resources. They serve as the blueprint for provisioning and managing infrastructure.
- Stacks:
 - A CloudFormation stack is a collection of AWS resources created and managed as a single unit. Stacks are created based on templates and can include a variety of resources, such as EC2 instances, S3 buckets, databases, etc.
- Resources:
 - Resources are the individual components defined in a CloudFormation template. Examples include EC2 instances, Amazon RDS databases, S3 buckets, and more. Resources are created, updated, or deleted based on the template.
- Change Sets:
 - Change sets allow users to preview the changes that will be made to a stack before applying those changes. This helps users understand the impact of modifications and assess potential risks.
- Outputs:
 - CloudFormation templates can define outputs, which are values that are returned after the stack is created or updated. Outputs can be used to retrieve information such as resource identifiers or connection details.
- Rollback and Rollback Triggers:
 - CloudFormation supports automatic rollback in case of errors during stack creation or updates. Users can define rollback triggers to perform specific actions when a rollback occurs.
- Parameterization:
 - Templates can include parameters that allow users to input custom values at the time of stack creation or update. Parameterization makes templates reusable across different scenarios.
- Stack Policies:
 - Stack policies allow users to control updates to specific resources in a stack. This provides an additional layer of control over the modification of critical resources.

Benefits:

- Automation:
 - CloudFormation automates the process of provisioning and managing AWS resources.
 - This reduces manual intervention, minimizes errors, and ensures consistency.
- Consistency:
 - With CloudFormation, infrastructure configurations are defined in templates, promoting consistency across different environments.
 - This helps avoid configuration drift and ensures that environments are identical.
- Efficiency:
 - CloudFormation makes it easy to create and manage stacks of resources, allowing users to provision and update infrastructure efficiently.
 - This is particularly beneficial for managing large and complex environments.
- Scalability:
 - CloudFormation scales with the complexity and size of deployments.
 - It is suitable for small projects with a few resources as well as large enterprise-scale applications with extensive infrastructure requirements.
- Version Control:
 - CloudFormation templates can be version-controlled using standard version control systems.
 - This allows teams to track changes, collaborate on infrastructure configurations, and roll back to previous versions if needed.
- Change Management:

- CloudFormation supports controlled and predictable change management.
- Users can preview changes using change sets, ensuring that updates are well-understood before being applied.
- Reusable Templates:
 - CloudFormation templates can be reused across different projects and environments, saving time and effort in defining infrastructure configurations.
- Rollback Mechanism:
 - The automatic rollback mechanism in CloudFormation ensures that if a stack creation or update encounters errors, the stack is rolled back to its previous state, preventing incomplete or inconsistent configurations.
- Integration with Other AWS Services:
 - CloudFormation integrates with other AWS services, such as AWS Identity and Access Management (IAM), AWS CloudTrail, and AWS CloudWatch.
 - This allows for enhanced security, auditing, and monitoring capabilities.

Cloud formation template:

<pre>AWSTemplateFormatVersion: '2010-09-09' Resources: MyS3Bucket: Type: 'AWS::S3::Bucket' Properties: BucketName: my-unique-s3-bucket-name</pre>	<pre>AWSTemplateFormatVersion: '2010-09-09' Resources: MyS3Bucket: Type: 'AWS::S3::Bucket' Properties: BucketName: my-unique-s3-bucket-name</pre>
---	---

In this template:

- AWSTemplateFormatVersion specifies the version of the CloudFormation template language.
- Resources is the section where you define the AWS resources you want to create. In this case, it creates an S3 bucket named MyS3Bucket.
- Type specifies the type of AWS resource to create. In this case, it's an AWS::S3::Bucket.
- Properties contains the configuration details for the resource. For an S3 bucket, you can specify properties such as BucketName.

To use this template, you would typically save it to a file with a .yaml or .yml extension, and then you can create a stack using the AWS Management Console, AWS CLI, or an SDK.

Here's an example using the AWS CLI:

- `aws cloudformation create-stack --stack-name MyS3Stack --template-body file:///path/totemplate.yaml`

AWS Documentation URLs

S.No	Content Name	Official URL to Read more
1	Amazon Aurora	Read More
2	Amazon CloudFront	Read More
3	Amazon CloudWatch	Read More
4	Amazon DynamoDB	Read More
5	Amazon EBS	Read More
6	Amazon EC2	Read More
7	Amazon ElastiCache	Read More
8	Amazon Glacier	Read More
9	Amazon RDS	Read More
10	Amazon Redshift	Read More
11	Amazon Route 53	Read More
12	Amazon S3	Read More
13	Amazon Simple Email Service (SES)	Read More
14	Amazon Simple Notification Service (SNS)	Read More
15	Amazon Simple Queue Service (SQS)	Read More
16	Amazon Simple Workflow Service (SWF)	Read More
17	Amazon VPC	Read More
18	AWS Auto Scaling	Read More
19	AWS Cloud Best Practices	Read More
20	AWS CloudFormation	Read More
21	AWS CloudHSM	Read More
22	AWS CloudTrail	Read More
23	AWS Config	Read More
24	AWS Direct Connect	Read More
25	AWS Identity and Access Management	Read More
26	AWS Key Management Service	Read More
27	AWS Lambda	Read More
28	AWS Management Console	Read More
29	AWS OpsWorks	Read More
30	AWS Server Migration Service (SMS)	Read More
31	AWS Shield	Read More
32	AWS Snowball	Read More
33	AWS Step Functions	Read More
34	AWS Storage Gateway	Read More
35	AWS Systems Manager	Read More
36	AWS VPN	Read More
37	AWS WAF	Read More
38	AWS Elastic Load Balancing (ELB)	Read More
39	AWS Well-Architected Framework Whitepapers	Read More

AWS Cloud Practitioner

Monday, April 14, 2025 12:00 AM

Episode 1: Overview and Cloud Concepts

Dive into foundational AWS Cloud concepts and understand exam expectations, as well as AWS benefits, design principles, migration strategies, and cloud economics.

Resources

- [4-Step Exam Preparation Plan](#)
- [AWS Certified Cloud Practitioner exam page](#)
- [Cloud Computing with AWS](#)
- [AWS Well-Architected Framework](#)
- [AWS Cloud Quest: Cloud Practitioner](#)

Episode 2: Security and Compliance

Deep dive into AWS security fundamentals, including the Shared Responsibility Model, access management, and compliance frameworks.

Resources

- [4-Step Exam Preparation Plan](#)
- [What is IAM?](#)
- [What is IAM Identity Center?](#)
- [IAM Identities](#)
- [AWS Security Documentation](#)

Episode 3: Cloud Technology and Services

Explore core AWS services across compute, storage, database, networking, and more. Learn how these services work together in real-world solutions.

Resources

- [4-Step Exam Preparation Plan](#)
- [AWS Compute](#)
- [AWS Storage](#)
- [AWS Databases](#)
- [AWS Networking](#)

Episode 4: Billing, Pricing, and Support; Review and Practice

Review AWS pricing models, cost management tools, and support options. Plus, get final exam preparation tips and practice with sample questions.

Resources

- [4-Step Exam Preparation Plan](#)
- [What is AWS Billing and Cost Management?](#)
- [Select the best pricing model](#)
- [Getting started with AWS Support](#)

AWS CLI command reference

Setting up AWS CLI account.

1. Create a new user with proper admin rights.
2. Create access key for the user and store the access & secret keys safely.
3. Install AWS CLI utility on your system.
4. Then open command prompt and run below command.

```
PS E:\PowerShell> aws configure
AWS Access Key ID [*****EFRO]: AKIAY376KZPMABFK2RMW
AWS Secret Access Key [*****r5TB]: pkx5J9TzvJj4boc47W2d8jpyKv7YwzwhGH04MwOF
Default region name [Ohio]:
Default output format [None]: json
PS E:\PowerShell>
PS E:\PowerShell>
```

If you get error "Could not connect to the endpoint URL: "<https://ec2.Ohio.amazonaws.com/>"

Then, use method 2:

Open powershell, and add the following as Environment Variables.

```
$Env:AWS_ACCESS_KEY_ID="AKIAY376KZPMABFK2RMW"
$Env:AWS_SECRET_ACCESS_KEY="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"
$Env:AWS_DEFAULT_REGION="us-east-2"
```

```
PS E:\PowerShell> aws configure list
      Name      Value      Type      Location
      ----      -
profile        <not set>  None      None
access_key     *****2RMW  env
secret_key     *****MwOF  env
region         us-east-2   env      ['AWS_REGION', 'AWS_DEFAULT_REGION']
PS E:\PowerShell>
```

To validate:

```
PS E:\PowerShell> aws ec2 describe-vpcs
{
  "Vpcs": [
    {
      "OwnerId": "609881869272",
      "InstanceTenancy": "default",
      "CidrBlockAssociationSet": [
        {
          "AssociationId": "vpc-cidr-assoc-051e04e83b8cb9b8c",
          "CidrBlock": "192.168.0.0/16",
          "CidrBlockState": {
            "State": "associated"
          }
        }
      ],
      "IsDefault": false,
      "Tags": [
        {
          "Key": "Name",
          "Value": "DemoVPC"
        }
      ],
      "BlockPublicAccessStates": {
        "InternetGatewayBlockMode": "off"
      },
      "VpcId": "vpc-0c3826f0da00181ca",
      "State": "available",
      "CidrBlock": "192.168.0.0/16",
      "DhcpOptionsId": "dopt-06ecfec896a7e68ab"
    }
  ],
}
```

Service	Command Example	Description
---------	-----------------	-------------

General	aws configure	Configure access keys, region, and output format
	aws help	Get help on AWS CLI usage
	aws <service> help	Help for a specific service
S3	aws s3 ls	List all S3 buckets
	aws s3 ls s3://<bucket-name>	List contents of a bucket
	aws s3 cp file.txt s3://<bucket-name>/	Copy file to S3
	aws s3 cp s3://<bucket-name>/file.txt .	Download file from S3
	aws s3 mv source.txt s3://<bucket-name>/	Move file to S3
	aws s3 rm s3://<bucket-name>/file.txt	Delete file from S3
EC2	aws ec2 describe-instances	List all EC2 instances
	aws ec2 start-instances --instance-ids i-123456	Start an EC2 instance
	aws ec2 stop-instances --instance-ids i-123456	Stop an EC2 instance
	aws ec2 terminate-instances --instance-ids i-123456	Terminate instance
IAM	aws iam list-users	List IAM users
	aws iam create-user --user-name <username>	Create a new IAM user
	aws iam delete-user --user-name <username>	Delete an IAM user
VPC	aws ec2 describe-vpcs	List all VPCs
	aws ec2 create-vpc --cidr-block 10.0.0.0/16	Create a new VPC
Lambda	aws lambda list-functions	List Lambda functions
	aws lambda invoke --function-name MyFunction out.json	Invoke a Lambda function
CloudWatch	aws logs describe-log-groups	List all log groups
	aws logs get-log-events --log-group-name <group> --log-stream-name <stream>	Get logs
CloudFormation	aws cloudformation list-stacks	List CloudFormation stacks
	aws cloudformation describe-stacks	Describe all stacks
ECS	aws ecs list-clusters	List ECS clusters
	aws ecs list-tasks --cluster <cluster-name>	List ECS tasks in a cluster
EKS	aws eks list-clusters	List EKS clusters
	aws eks describe-cluster --name <cluster-name>	Describe an EKS cluster
STS (Security Token Service)	aws sts get-caller-identity	Shows the identity of the current credentials

Creating a new subnet in an existing VPC:

Cmd:

```
aws ec2 create-subnet --vpc-id vpc-0c3826f0da00181ca --cidr-block 192.168.3.0/24 --availability-zone us-east-2c --region us-east-2
PS E:\PowerShell> aws ec2 create-subnet --vpc-id vpc-0c3826f0da00181ca --cidr-block 192.168.3.0/24 --availability-zone us-east-2c --region us-east-2
{
  "Subnet": {
    "AvailabilityZoneId": "use2-az3",
    "OwnerId": "609881869272",
    "AssignIpv6AddressOnCreation": false,
    "Ipv6CidrBlockAssociationSet": [],
    "SubnetArn": "arn:aws:ec2:us-east-2:609881869272:subnet/subnet-0c004c5245b485e58",
    "EnableDns64": false,
    "Ipv6Native": false,
    "PrivateDnsNameOptionsOnLaunch": {
      "HostnameType": "ip-name",
      "EnableResourceNameDnsARecord": false,
      "EnableResourceNameDnsAAAARecord": false
    },
    "SubnetId": "subnet-0c004c5245b485e58",
    "State": "available",
    "VpcId": "vpc-0c3826f0da00181ca",
    "CidrBlock": "192.168.3.0/24",
    "AvailableIpAddressCount": 251,
    "AvailabilityZone": "us-east-2c",
    "DefaultForAz": false,
    "MapPublicIpOnLaunch": false
  }
}
```

Adding name (tag) to this new subnet:

```
aws ec2 create-tags --resources subnet-0abf97ed0aca9c9bb --tags Key=Name,Value=subnet3 --region us-east-2
```

Verify:

Subnets (6) [Info](#)

Last updated
1 minute ago  [Acti](#)

 Find resources by attribute or tag

☐	Name ▲	Subnet ID ▼	State ▼	VPC ▼	Block Public... ▼	IPv4 CIDR
☐	DefaultSubnet1	subnet-0ed9737014f9f93fc	✔ Available	vpc-0a83901806b68edd9 DefaultVPC	⊖ Off	172.31.16.0/20
☐	DefaultSubnet2	subnet-082aac0ce09b47845	✔ Available	vpc-0a83901806b68edd9 DefaultVPC	⊖ Off	172.31.32.0/20
☐	DefaultSubnet3	subnet-070aa0fee0880a5fd	✔ Available	vpc-0a83901806b68edd9 DefaultVPC	⊖ Off	172.31.0.0/20
☐	subnet1	subnet-056b28474d03c25a9	✔ Available	vpc-0c3826f0da00181ca DemoVPC	⊖ Off	192.168.1.0/24
☐	subnet2	subnet-08813ae14d076febf	✔ Available	vpc-0c3826f0da00181ca DemoVPC	⊖ Off	192.168.2.0/24
☐	subnet3	subnet-0abf97ed0aca9c9bb	✔ Available	vpc-0c3826f0da00181ca DemoVPC	⊖ Off	192.168.3.0/24

AWS Elastic Container Service (ECS)

Wednesday, April 23, 2025 9:13 PM

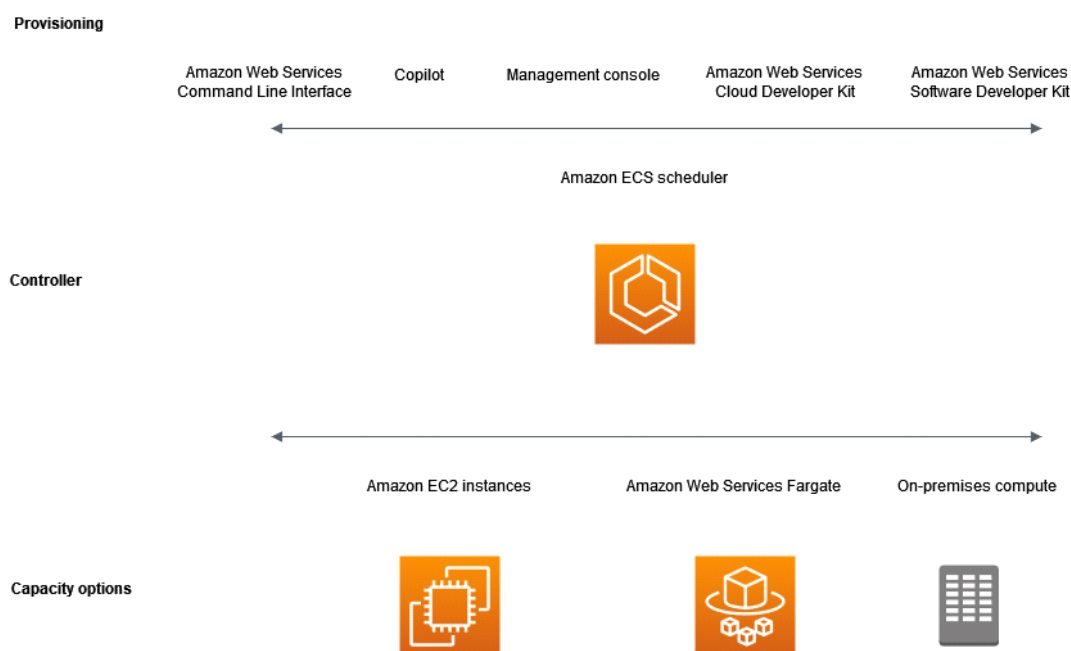
- Amazon Elastic Container Service (Amazon ECS) is a fully managed container orchestration service that helps you easily deploy, manage, and scale containerized applications.
- As a fully managed service, Amazon ECS comes with AWS configuration and operational best practices built-in.
- It's integrated with both AWS tools, such as Amazon Elastic Container Registry, and third-party tools, such as Docker.
- This integration makes it easier for teams to focus on building the applications, not the environment.
- You can run and scale your container workloads across AWS Regions in the cloud, and on-premises, without the complexity of managing a control plane.

Terminology and components

There are three layers in Amazon ECS:

- **Capacity** - The infrastructure where your containers run
- **Controller** - Deploy and manage your applications that run on the containers
- **Provisioning** - The tools that you can use to interface with the scheduler to deploy and manage your applications and containers

Amazon Elastic Container Service Layers



The capacity is the infrastructure where your containers run. The following is an overview of the capacity options:

Amazon EC2 instances in the AWS cloud You choose the instance type, the number of instances, and manage the capacity.

- Serverless (AWS Fargate) in the AWS cloud
- Fargate is a serverless, pay-as-you-go compute engine. With Fargate you don't need to manage servers, handle capacity planning, or isolate container workloads for security.
- On-premises virtual machines (VM) or servers

Amazon ECS Anywhere provides support for registering an external instance such as an on-

premises server or virtual machine (VM), to your Amazon ECS cluster.

Set up to use Amazon ECS:

1. Sign up for an AWS account
2. Create a user with Administrative access (root equivalent)
3. Create a virtual private cloud
 - Resources to create: VPC only
 - Name: Optionally provide a name for your VPC.
 - IPv4 CIDR block: IPv4 CIDR manual input
 - Tenancy: Default
4. Create a security group
 - HTTP
 - HTTPS
 - SSH
5. Create a key pair.
6. Install AWS CLI

Creating a container image for use on Amazon ECS:

Prerequisites

- Before you begin, ensure the following prerequisites are met.
- Ensure you have completed the Amazon ECR setup steps. For more information, see *Moving an image through its lifecycle in Amazon ECR in the Amazon Elastic Container Registry User Guide*.
- Your user has the required IAM permissions to access and use the Amazon ECR service. For more information, see *Amazon ECR managed policies*.
- You have Docker installed. For Docker installation steps for Amazon Linux 2, see *Installing Docker on AL2023*. For all other operating systems, see the *Docker documentation at Docker Desktop overview*.
- You have the AWS CLI installed and configured. For more information, see *Installing or updating to the latest version of the AWS CLI in the AWS Command Line Interface User Guide*.

Create a Docker image

Create a file called Dockerfile:

```
# touch Dockerfile
```

Edit the Dockerfile:

```
FROM public.ecr.aws/amazonlinux/amazonlinux:latest

# Update installed packages and install Apache
RUN yum update -y && \
    yum install -y httpd

# Write hello world message
RUN echo 'Hello World!' > /var/www/html/index.html

# Configure Apache
RUN echo 'mkdir -p /var/run/httpd' >> /root/run_apache.sh && \
    echo 'mkdir -p /var/lock/httpd' >> /root/run_apache.sh && \
    echo '/usr/sbin/httpd -D FOREGROUND' >> /root/run_apache.sh && \
    chmod 755 /root/run_apache.sh
```

```
CMD /root/run_apache.sh
```

Build the Docker image from your Dockerfile

```
# docker build -t hello-world .
# docker images --filter reference=hello-world
```

Run the newly built image.

```
# docker run -t -i -p 80:80 hello-world
```

Push your image to Amazon Elastic Container Registry

Create an Amazon ECR repository to store your hello-world image.

```
# aws ecr create-repository --repository-name hello-repository --region region
```

Tag the hello-world image with the repositoryUri value

```
# docker tag hello-world aws_account_id.dkr.ecr.region.amazonaws.com/hello-repository
```

Run the aws ecr get-login-password command:

```
# aws ecr get-login-password --region region | docker login --username AWS --password-stdin aws_account_id.dkr.ecr.region.amazonaws.com
```

Push the image to Amazon ECR:

```
# docker push aws_account_id.dkr.ecr.region.amazonaws.com/hello-repository
```

Commands

```
> sudo docker info
> touch Dockerfile
> vim Dockerfile
> docker build -t hello-world .
> sudo docker build -t hello-world .
> sudo docker images
> sudo docker run -t -i -p 80:80 hello-world
> curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
> unzip awscliv2.zip
> sudo ./aws/install
> aws --version
> aws configure
> aws ecr create-repository --repository-name hello-repository --region us-east-2
> docker tag hello-world 609881869272.dkr.ecr.us-east-2.amazonaws.com/hello-repository
> sudo docker tag hello-world 609881869272.dkr.ecr.us-east-2.amazonaws.com/hello-repository
> sudo docker push 609881869272.dkr.ecr.us-east-2.amazonaws.com/hello-repository:latest
> sudo docker images
> sudo docker push 609881869272.dkr.ecr.us-east-2.amazonaws.com/hello-repository:latest
> aws ecr get-login-password --region us-east-2 | sudo docker login --username AWS --password-stdin 609881869272.dkr.ecr.us-east-2.amazonaws.com
```

```
> sudo docker images  
> sudo docker push 609881869272.dkr.ecr.us-east-2.amazonaws.com/hello-  
repository:latest
```

Week-wise planner

Saturday, January 20, 2024 11:52 AM

Weeks 1-2: Foundations and Overview

- Understand the AWS Certified Solutions Architect - Associate exam guide.
- Set up an AWS account and explore the AWS Management Console.
- Read AWS whitepapers on the Well-Architected Framework and the AWS Cloud best practices.

Weeks 3-4: Compute Services

- Dive into EC2 (Elastic Compute Cloud) and understand instance types, AMIs, and security groups.
- Learn about Auto Scaling, Elastic Load Balancing (ELB), and launch configurations.
- Explore AWS Lambda and its use cases.

Weeks 5-6: Storage Services

- Study Amazon S3 (Simple Storage Service), including buckets, versioning, and permissions.
- Explore Amazon EBS (Elastic Block Store) and understand volume types.
- Learn about AWS Glacier, storage gateway, and Snowball.

Weeks 7-8: Database Services

- Study Amazon RDS (Relational Database Service) and understand database engines.
- Explore Amazon DynamoDB and understand its use cases.
- Learn about Amazon Redshift, Amazon Aurora, and ElastiCache.

Weeks 9-10: Networking and Content Delivery

- Understand Amazon VPC (Virtual Private Cloud) and subnetting.
- Explore Route 53 (DNS service) and CloudFront (content delivery network).
- Study Direct Connect, VPN, and Elastic Load Balancer (ELB).

Weeks 11-12: Security and Identity

- Learn about AWS Identity and Access Management (IAM).
- Study AWS Key Management Service (KMS) and CloudHSM.
- Understand AWS WAF, Shield, and best practices for securing your AWS environment.

Weeks 13-14: Application Integration and Messaging

- Explore Amazon Simple Queue Service (SQS) and Simple Notification Service (SNS).
- Learn about AWS Simple Workflow (SWF) and Step Functions.
- Study Amazon Simple Email Service (SES) and Simple Migration Service (SMS).

Weeks 15-16: Monitoring and Management Tools

- Dive into Amazon CloudWatch and CloudTrail.
- Explore AWS Config and OpsWorks.
- Learn about AWS Systems Manager and CloudFormation.

Weeks 17-18: Exam Review and Practice

- Review key concepts and notes.
- Take practice exams and identify areas for improvement.
- Review any remaining weak areas and brush up on exam-taking strategies.

Week 19: Final Review and Exam Day

- Final review of all topics.
- Take the AWS Certified Solutions Architect - Associate (SAA-C03) exam.

Official TOC

Saturday, January 20, 2024 11:52 AM

TOC:

Domain 1: Design Secure Architectures

Task Statement 1.1: Design secure access to AWS resources.

- Applying AWS security best practices to IAM users and root users (for example, multi-factor authentication [MFA])
- Designing a flexible authorization model that includes IAM users, groups, roles, and policies
- Designing a role-based access control strategy (for example, AWS Security Token Service [AWS STS], role switching, cross-account access)
- Designing a security strategy for multiple AWS accounts (for example, AWS Control Tower, service control policies [SCPs])
- Determining the appropriate use of resource policies for AWS services
- Determining when to federate a directory service with IAM roles

Task Statement 1.2: Design secure workloads and applications.

- Designing VPC architectures with security components (for example, security groups, route tables, network ACLs, NAT gateways)
- Determining network segmentation strategies (for example, using public subnets and private subnets)
- Integrating AWS services to secure applications (for example, AWS Shield, AWS WAF, IAM Identity Center, AWS Secrets Manager)
- Securing external network connections to and from the AWS Cloud (for example, VPN, AWS Direct Connect)

Task Statement 1.3: Determine appropriate data security controls.

- Aligning AWS technologies to meet compliance requirements
- Encrypting data at rest (for example, AWS Key Management Service [AWS KMS])
- Encrypting data in transit (for example, AWS Certificate Manager [ACM] using TLS)
- Implementing access policies for encryption keys
- Implementing data backups and replications
- Implementing policies for data access, lifecycle, and protection
- Rotating encryption keys and renewing certificates

Domain 2: Design Resilient Architectures

Task Statement 2.1: Design scalable and loosely coupled architectures.

- Designing event-driven, microservice, and/or multi-tier architectures based on requirements
- Determining scaling strategies for components used in an architecture design
- Determining the AWS services required to achieve loose coupling based on requirements
- Determining when to use containers
- Determining when to use serverless technologies and patterns
- Recommending appropriate compute, storage, networking, and database technologies based on requirements
- Using purpose-built AWS services for workloads

Task Statement 2.2: Design highly available and/or fault-tolerant architectures.

- Determining automation strategies to ensure infrastructure integrity
- Determining the AWS services required to provide a highly available and/or fault-tolerant architecture across AWS Regions or Availability Zones
- Identifying metrics based on business requirements to deliver a highly available solution
- Implementing designs to mitigate single points of failure

- Implementing strategies to ensure the durability and availability of data (for example, backups)
- Selecting an appropriate DR strategy to meet business requirements
- Using AWS services that improve the reliability of legacy applications and applications not built for the cloud (for example, when application changes are not possible)
- Using purpose-built AWS services for workloads

Domain 3: Design High-Performing Architectures

Task Statement 3.1: Determine high-performing and/or scalable storage solutions.

- Determining storage services and configurations that meet performance demands
- Determining storage services that can scale to accommodate future needs

Task Statement 3.2: Design high-performing and elastic compute solutions.

- Decoupling workloads so that components can scale independently
- Identifying metrics and conditions to perform scaling actions
- Selecting the appropriate compute options and features (for example, EC2 instance types) to meet business requirements
- Selecting the appropriate resource type and size (for example, the amount of Lambda memory) to meet business requirements

Task Statement 3.3: Determine high-performing database solutions.

- Configuring read replicas to meet business requirements
- Designing database architectures
- Determining an appropriate database engine (for example, MySQL compared with PostgreSQL)
- Determining an appropriate database type (for example, Amazon Aurora, Amazon DynamoDB)
- Integrating caching to meet business requirements

Task Statement 3.4: Determine high-performing and/or scalable network architectures.

- Creating a network topology for various architectures (for example, global, hybrid, multi-tier)
- Determining network configurations that can scale to accommodate future needs
- Determining the appropriate placement of resources to meet business requirements
- Selecting the appropriate load balancing strategy

Task Statement 3.5: Determine high-performing data ingestion and transformation solutions.

- Building and securing data lakes
- Designing data streaming architectures
- Designing data transfer solutions
- Implementing visualization strategies
- Selecting appropriate compute options for data processing (for example, Amazon EMR)
- Selecting appropriate configurations for ingestion
- Transforming data between formats (for example, .csv to .parquet)

Domain 4: Design Cost-Optimized Architectures

Task Statement 4.1: Design cost-optimized storage solutions.

- Designing appropriate storage strategies (for example, batch uploads to Amazon S3 compared with individual uploads)
- Determining the correct storage size for a workload
- Determining the lowest cost method of transferring data for a workload to AWS storage
- Determining when storage auto scaling is required
- Managing S3 object lifecycles
- Selecting the appropriate backup and/or archival solution

- Selecting the appropriate service for data migration to storage services
- Selecting the appropriate storage tier
- Selecting the correct data lifecycle for storage
- Selecting the most cost-effective storage service for a workload

Task Statement 4.2: Design cost-optimized compute solutions.

- Determining an appropriate load balancing strategy (for example, Application Load Balancer [Layer 7] compared with Network Load Balancer [Layer 4] compared with Gateway Load Balancer)
- Determining appropriate scaling methods and strategies for elastic workloads (for example, horizontal compared with vertical, EC2 hibernation)
- Determining cost-effective AWS compute services with appropriate use cases (for example, Lambda, Amazon EC2, Fargate)
- Determining the required availability for different classes of workloads (for example, production workloads, non-production workloads)
- Selecting the appropriate instance family for a workload
- Selecting the appropriate instance size for a workload

Task Statement 4.3: Design cost-optimized database solutions.

- Designing appropriate backup and retention policies (for example, snapshot frequency)
- Determining an appropriate database engine (for example, MySQL compared with PostgreSQL)
- Determining cost-effective AWS database services with appropriate use cases (for example, DynamoDB compared with Amazon RDS, serverless)
- Determining cost-effective AWS database types (for example, time series format, columnar format)
- Migrating database schemas and data to different locations and/or different database engines

Task Statement 4.4: Design cost-optimized network architectures.

- Configuring appropriate NAT gateway types for a network (for example, a single shared NAT gateway compared with NAT gateways for each Availability Zone)
- Configuring appropriate network connections (for example, Direct Connect compared with VPN compared with internet)
- Configuring appropriate network routes to minimize network transfer costs (for example, Region to Region, Availability Zone to Availability Zone, private to public, Global Accelerator, VPC endpoints)
- Determining strategic needs for content delivery networks (CDNs) and edge caching
- Reviewing existing workloads for network optimizations
- Selecting an appropriate throttling strategy
- Selecting the appropriate bandwidth allocation for a network device (for example, a single VPN compared with multiple VPNs, Direct Connect speed)